Lecture 7

Binary Search Trees and Red-Black Trees

Announcements

- Midterm Thursday!
  - Keep an eye out for an Ed post today about **seating charts** and other logistics.
    - Please check the seating chart **before** coming to the exam.
  - Reminder: Exam is **60 minutes** so that there's time at the end for exam collecting and ID-checking.
    - Please be patient and follow CA instructions so that we can distribute/collect exams in an orderly fashion!
  - **Note:** if you start taking the exam and find it difficult, then it's probably difficult for lots of people.
    - There's a curve for a reason! Keep calm and do your best.
- No Section this week!
  - Take a break after the exam ☺

A few notes on High-Res Feedback

- Thanks to those who have responded!
- A few themes:
  - "More examples please!"
    - **Okay!** Going forward, we'll put some extra practice problems on Section materials.
    - Also, in addition to Lecture/Section/HW/Practice Exam, you can check out all the problems in the textbook, and also in CLRS!
  - More resources for how to design algorithms
    - We'll have a bit on this at the beginning of today's lecture
    - Also check out the Problem-Solving Guide on the course website! (In the "Resources" tab)

A few notes on High-Res Feedback

- A few themes (ctd):
  - How detailed should pseudocode be?
    - We have guidance on this – and an example solution set – on the course website! (At the top of the Homework page)
  - More resources for induction/formal proofs?
    - Check out the handouts for Lectures 2 and 4
    - Check out the "Induction" overview on the Resources tab on the course website. (Under "Pre-requisites")
    - Check out Appendix A of your textbook
    - Come to office hours!

Real quick, leftover from last time

Question on RadixSort

- Running time was $O\big(d \cdot (n + r)\big)$

Number of base-r-digits (r-dits?)

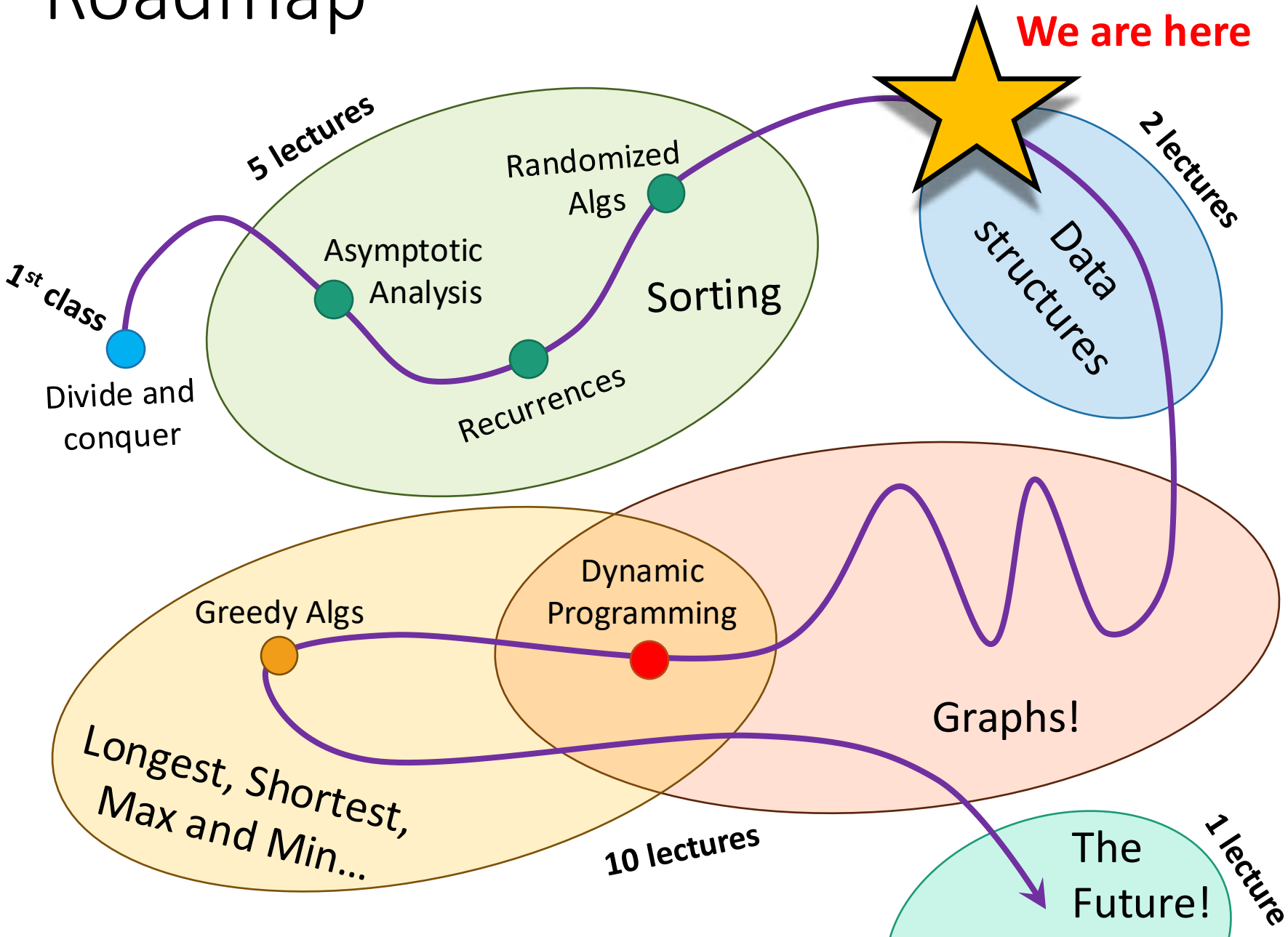
Number of integers we are sorting

Base for radix-sort, also the number of buckets

- **Question:** why not $O(dn + r)$ since we only need to initialize the buckets once?
- **Answer:** we also need to go through all the buckets each time to empty them into the final list.

More detailed schedule on the website!

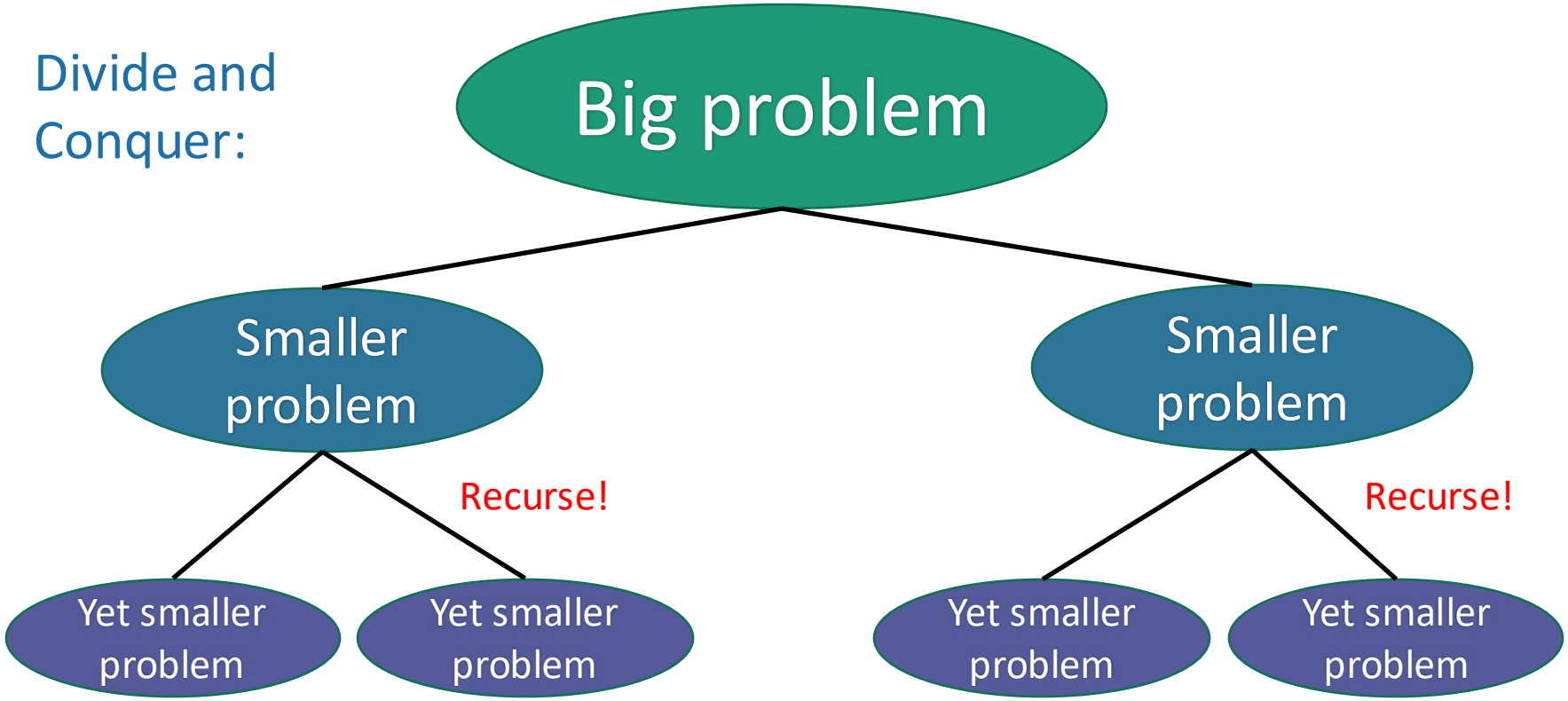
Roadmap



But first!

- A brief wrap-up of divide and conquer.

Divide and Conquer:



How do we design divide-and-conquer algorithms?

- So far we've seen lots of examples.
  - Karatsuba
  - MergeSort
  - Select
  - QuickSort
  - Sorting flocks of ducks (HW1)
  - Finding closest pairs of fish (HW2)
  - (plus more in Section)

- Let's take a minute to zoom out and look at some general strategies.

One Strategy

1. Identify natural sub-problems
   - Arrays of half the size
   - Things smaller/larger than a pivot
   - All the fish on the left/right of a cut-off
2. Imagine you had the magical ability to solve those natural sub-problems…what would you do?
   - Just try it with all of the natural sub-problems you can come up with! Anything look helpful?
3. Work out the details
   - Write down pseudocode
   - Figure out the running time
     - Usually for divide-and-conquer this starts with writing down a recurrence relation.

Fish Friends

- Goal: Find the closest pair of fish

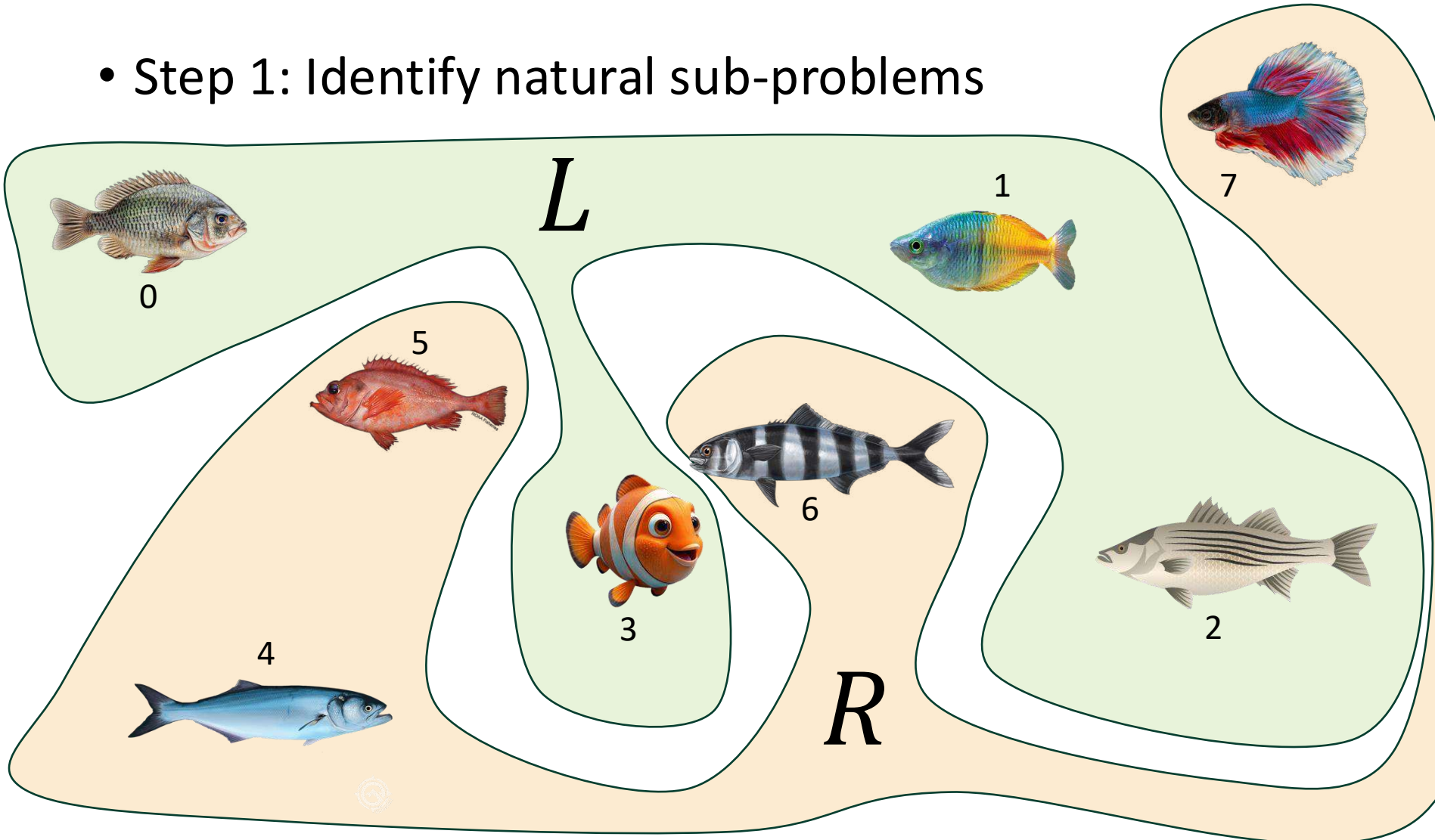
Fish Friends

- Step 1: Identify natural sub-problems

Fish Friends

Maybe let's try something like MergeSort? If the fish are numbered 0 through n-1, take the first n/2 and the second n/2?

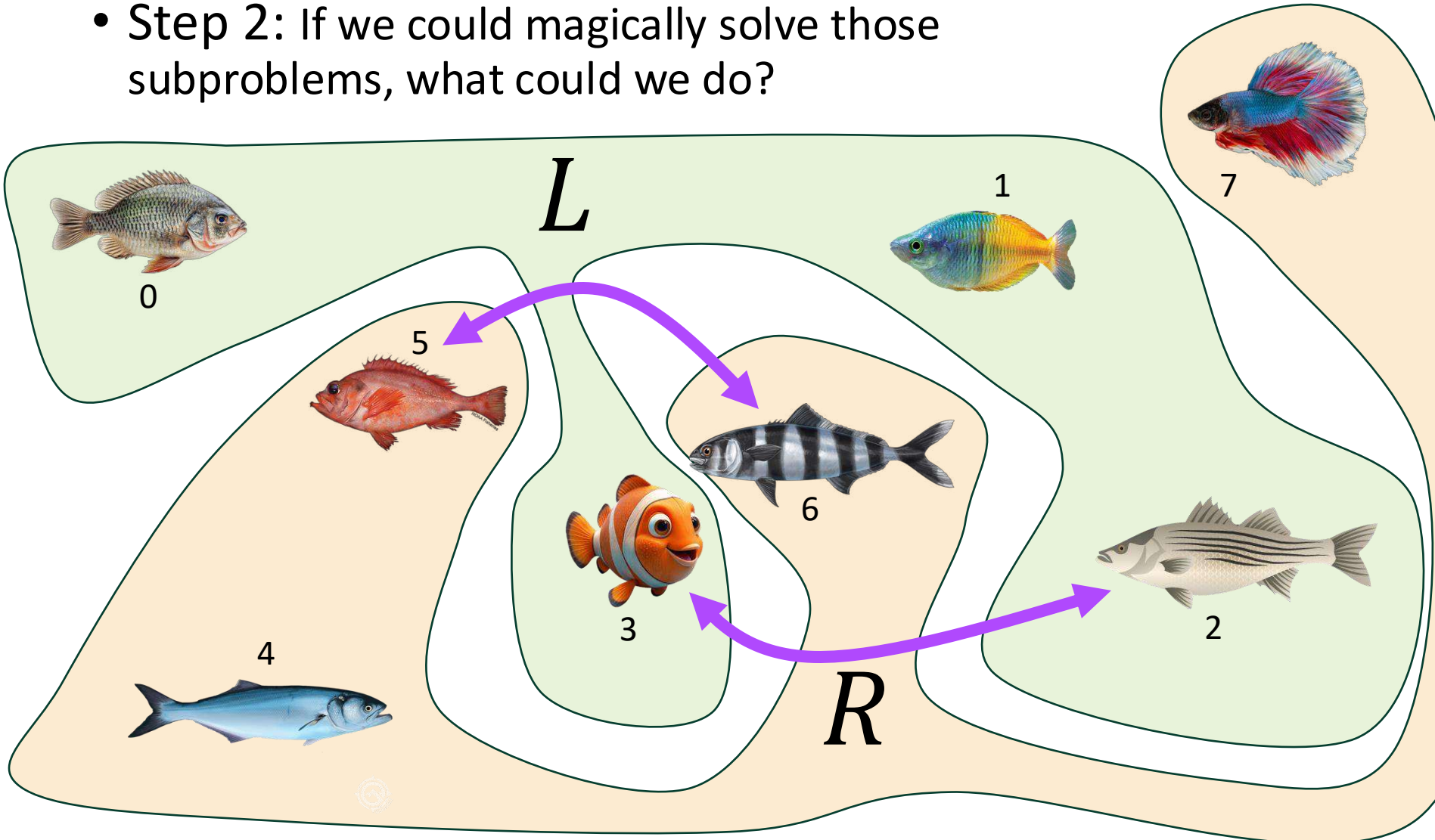
- Step 1: Identify natural sub-problems



Fish Friends

Maybe let's try something like MergeSort? If the fish are numbered 0 through n-1, take the first n/2 and the second n/2?

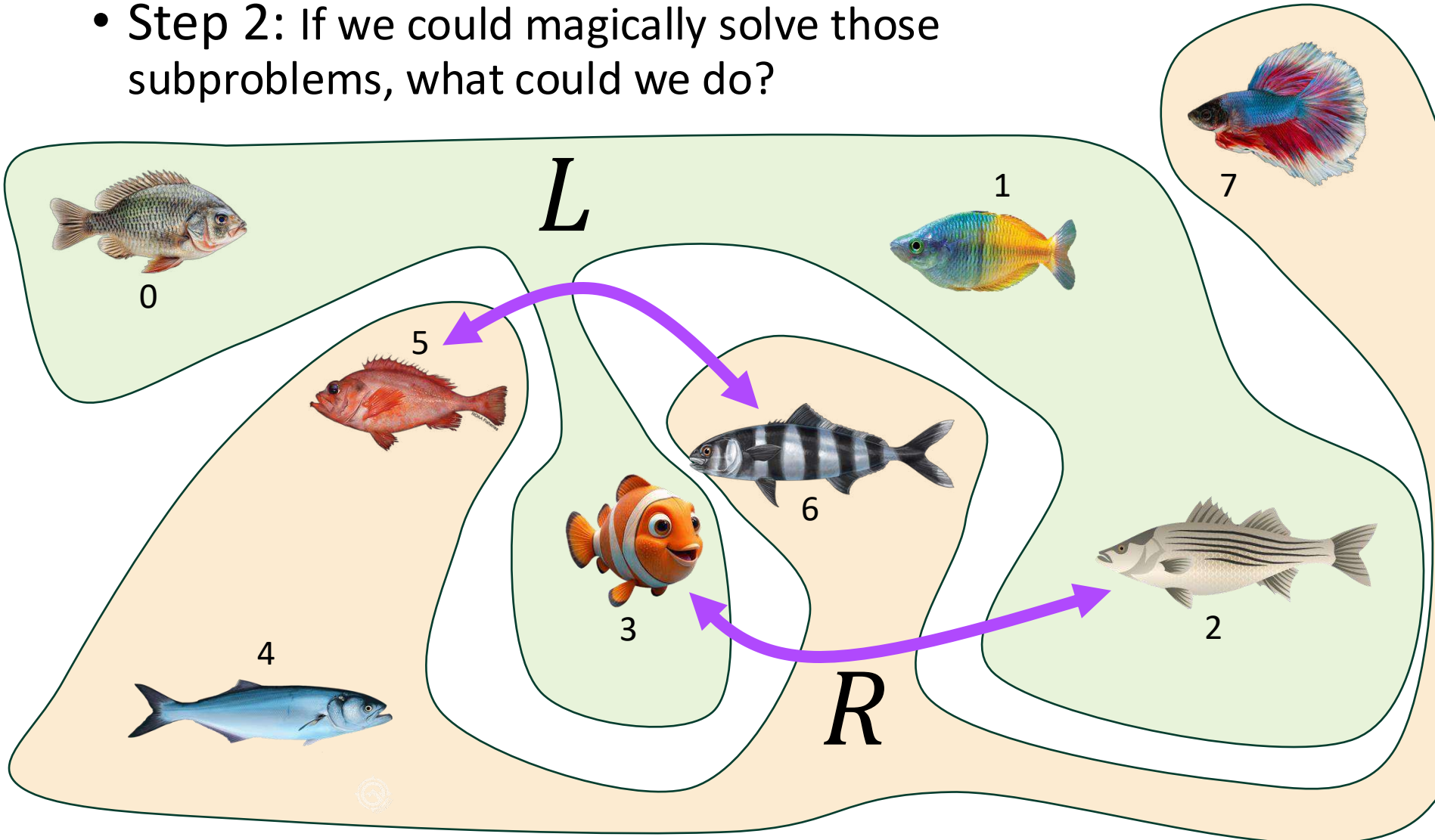
- Step 2: If we could magically solve those subproblems, what could we do?



Huh… that doesn't actually seem so useful…

Fish Friends

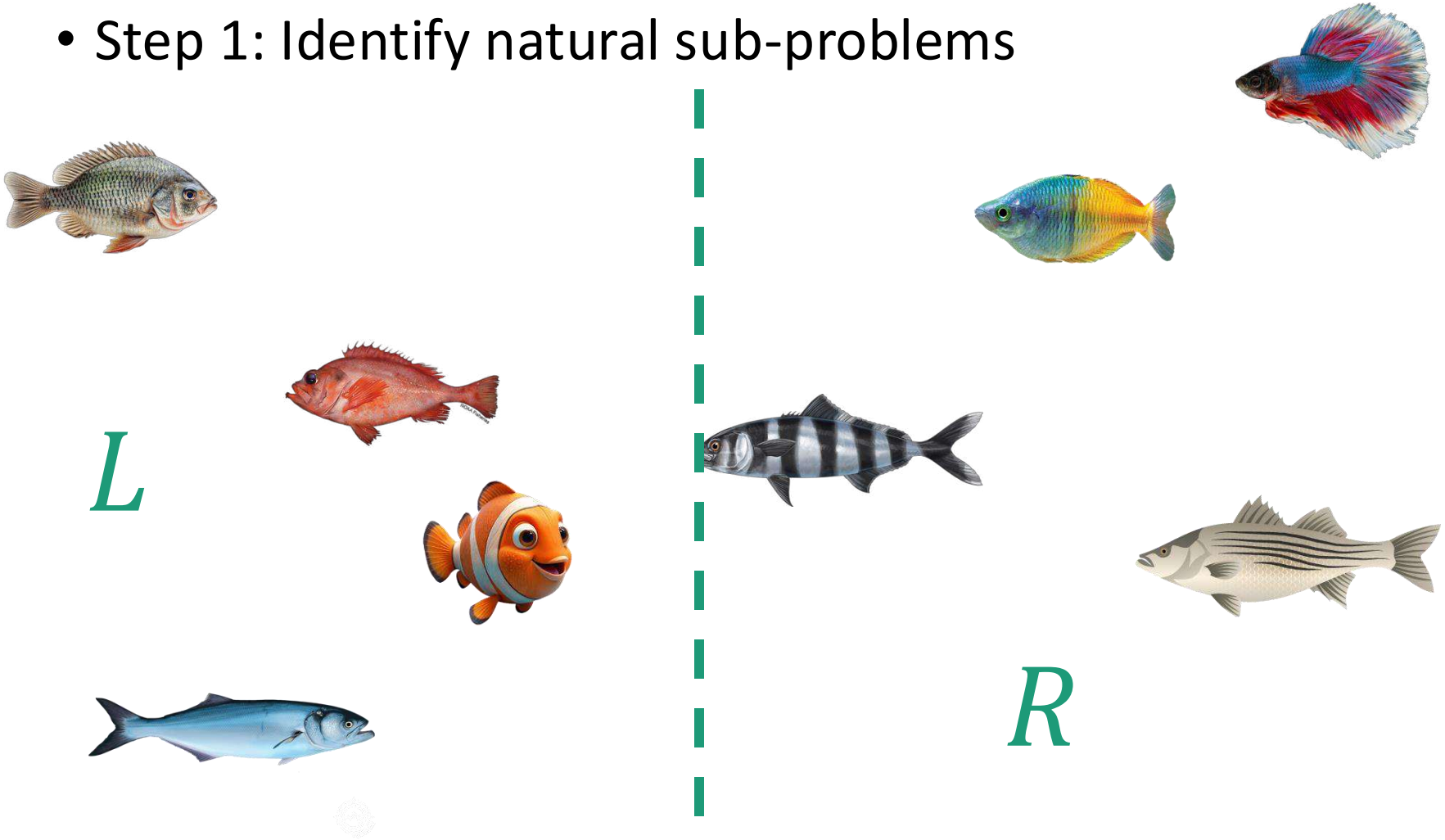
- **Step 2:** If we could magically solve those subproblems, what could we do?



Fish Friends

Maybe let's try something like QuickSort? Partition the fish relative to some "pivot"…

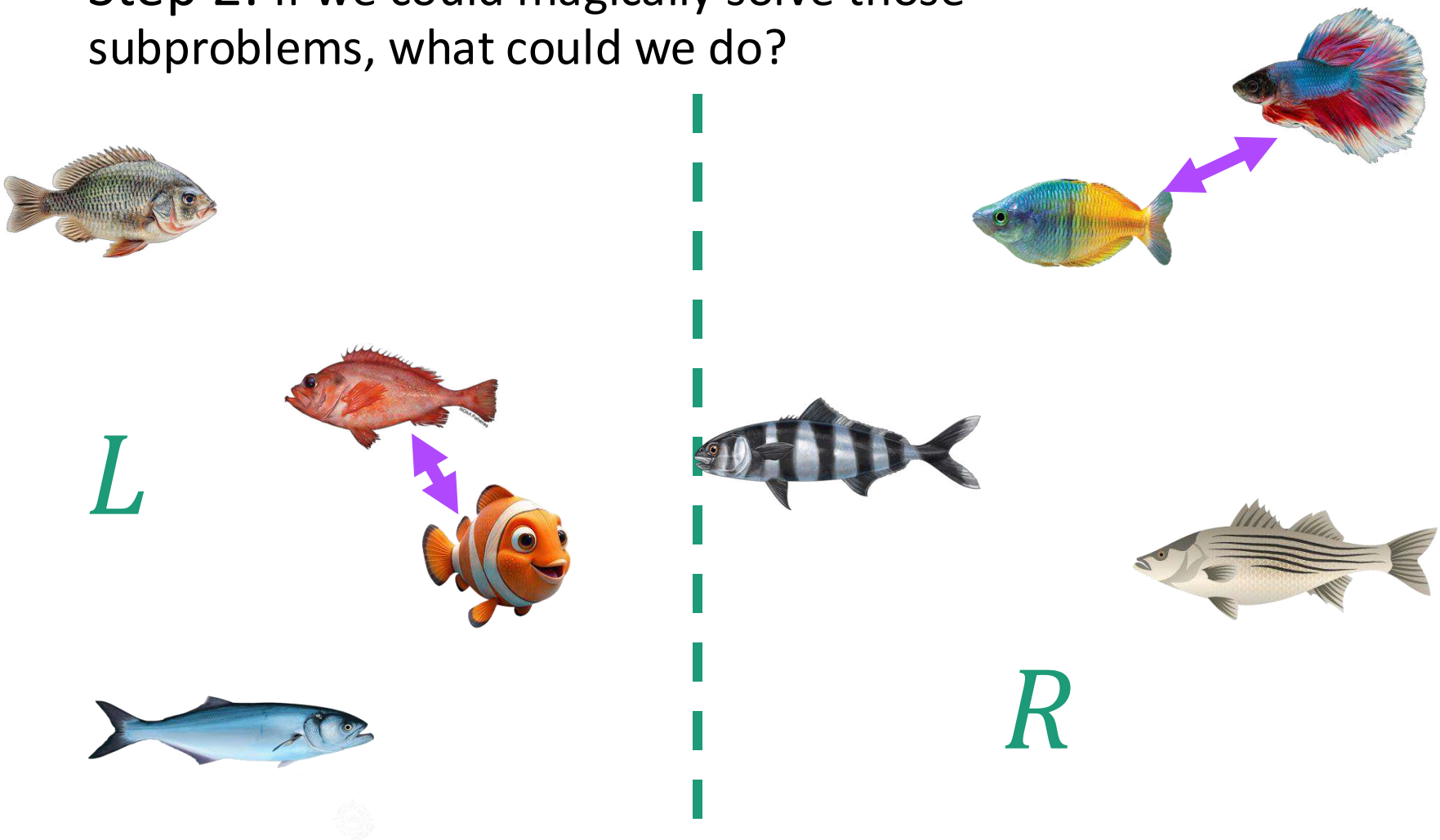
- Step 1: Identify natural sub-problems



Fish Friends

Maybe let's try something like QuickSort? Partition the fish relative to some "pivot"…

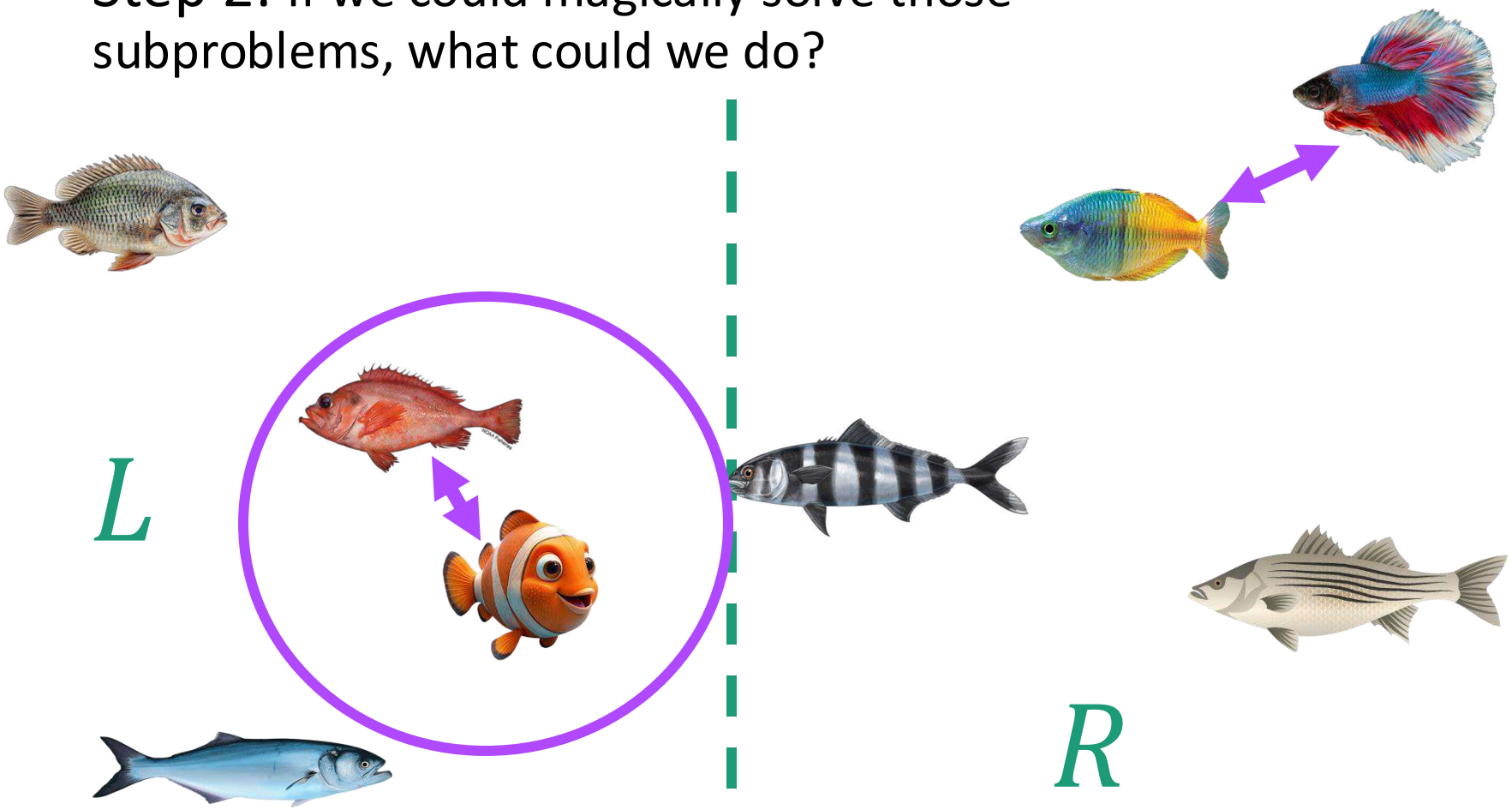
- Step 2: If we could magically solve those subproblems, what could we do?



Fish Friends

Hey, that seems useful! If the closest pair is either on the left or the right, we'd be done!

- Step 2: If we could magically solve those subproblems, what could we do?

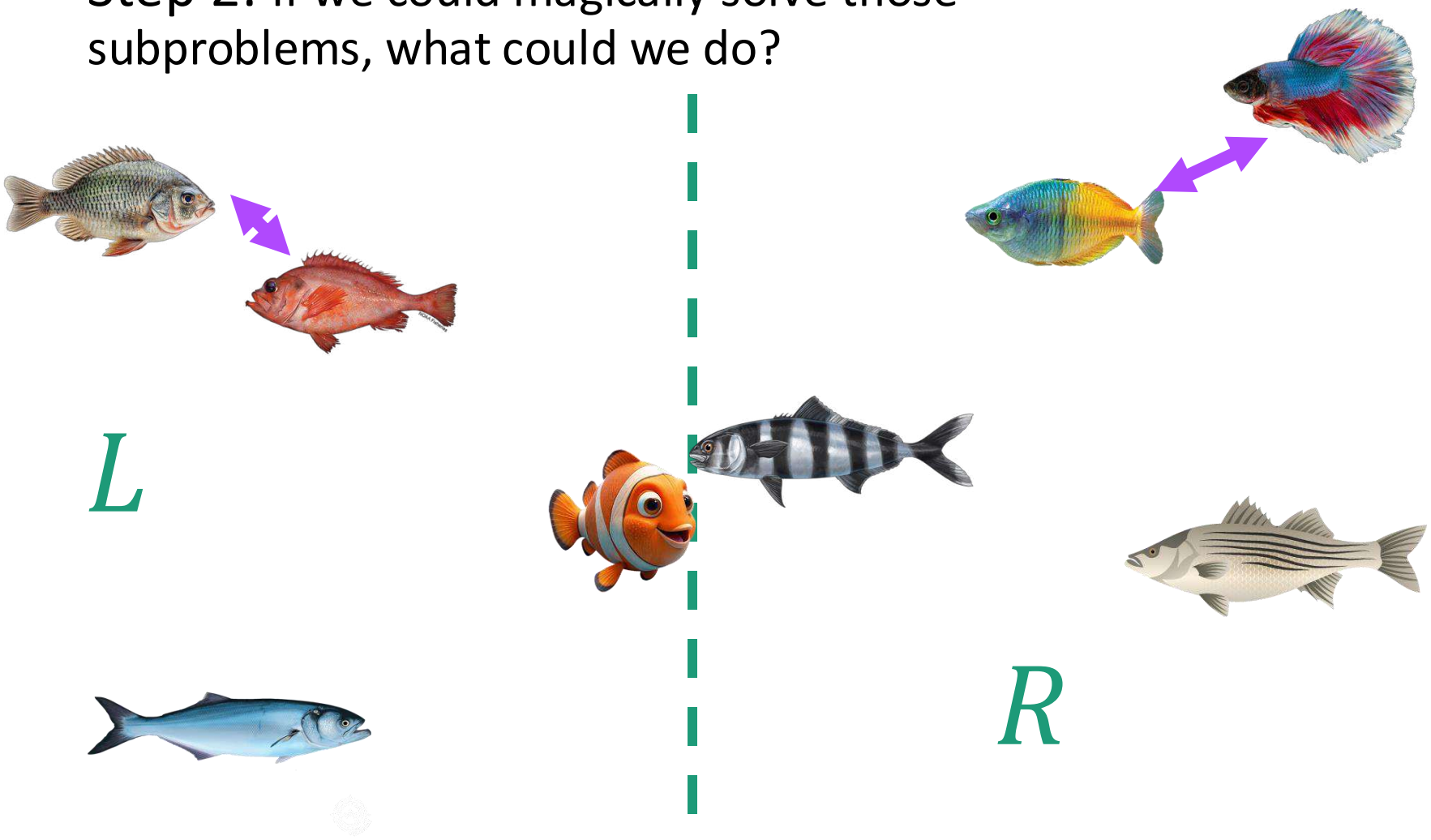


Fish Friends

But what if the true closest pair crosses the boundary?

We need to think about how to find "close pairs near the boundary" quickly…

- Step 2: If we could magically solve those subproblems, what could we do?

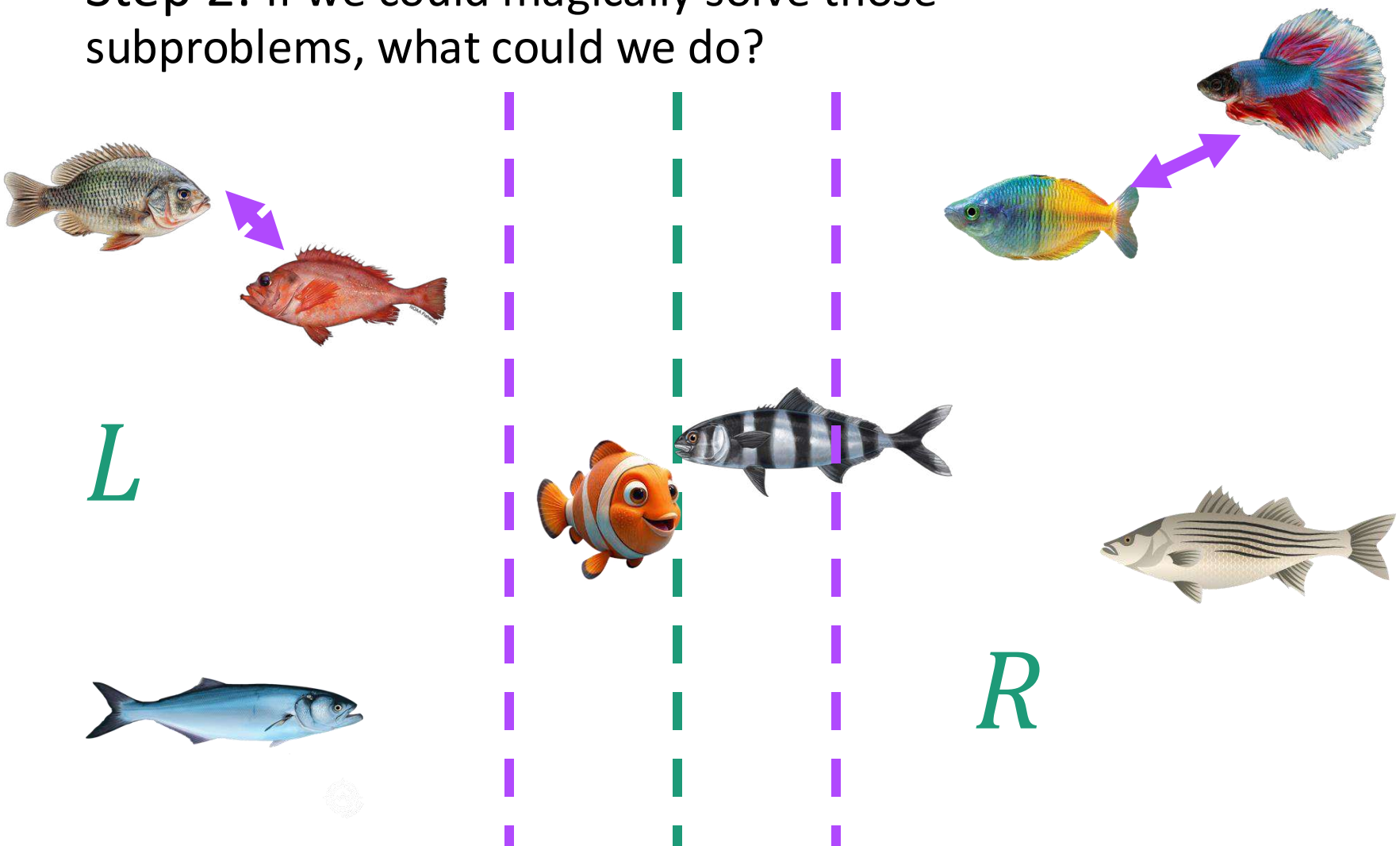


Fish Friends

But what if the true closest pair crosses the boundary?

What does “near the boundary” mean? Closer than either of the two two pairs in L or R…

- Step 2: If we could magically solve those subproblems, what could we do?

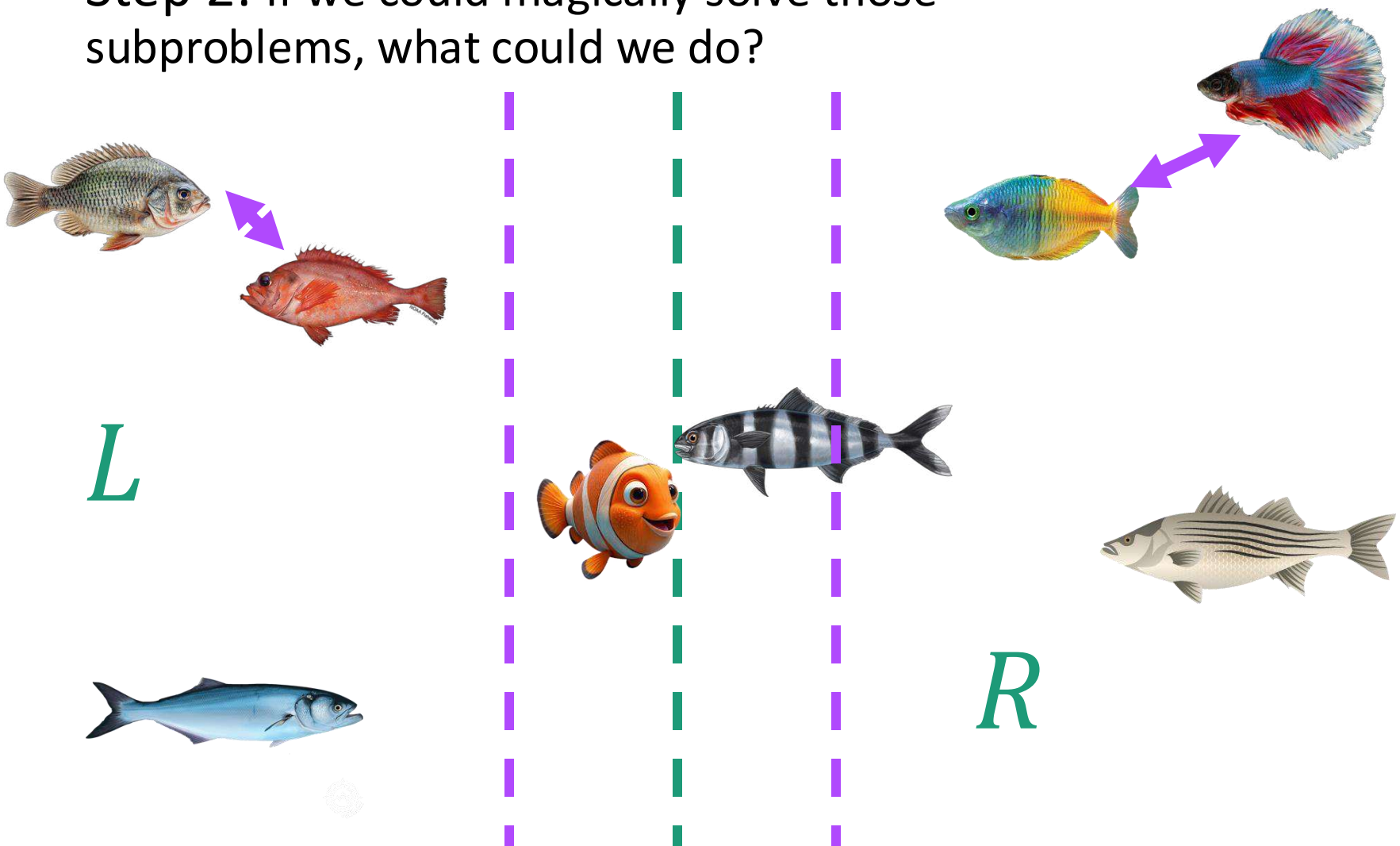


Fish Friends

But what if the true closest pair crosses the boundary?

Hey, if we sort just those "boundary fish" by y-coordinate, it looks like we won't have to consider **that** many pairs… (Thanks, hint!)

- Step 2: If we could magically solve those subproblems, what could we do?

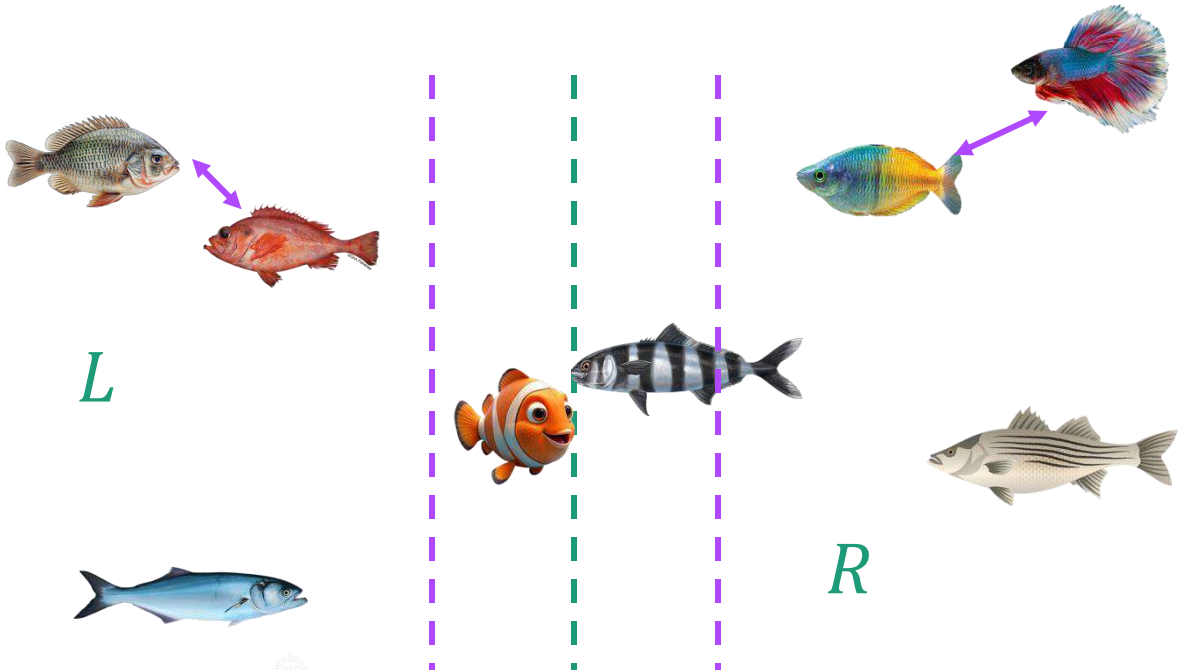


Fish Friends

- Step 3: Put it all together and write some pseudocode

My time to shine!

Probably we'll have to go back and forth a bit to nail this down ☺



Fish Friends

- Step 3': Figure out the running time

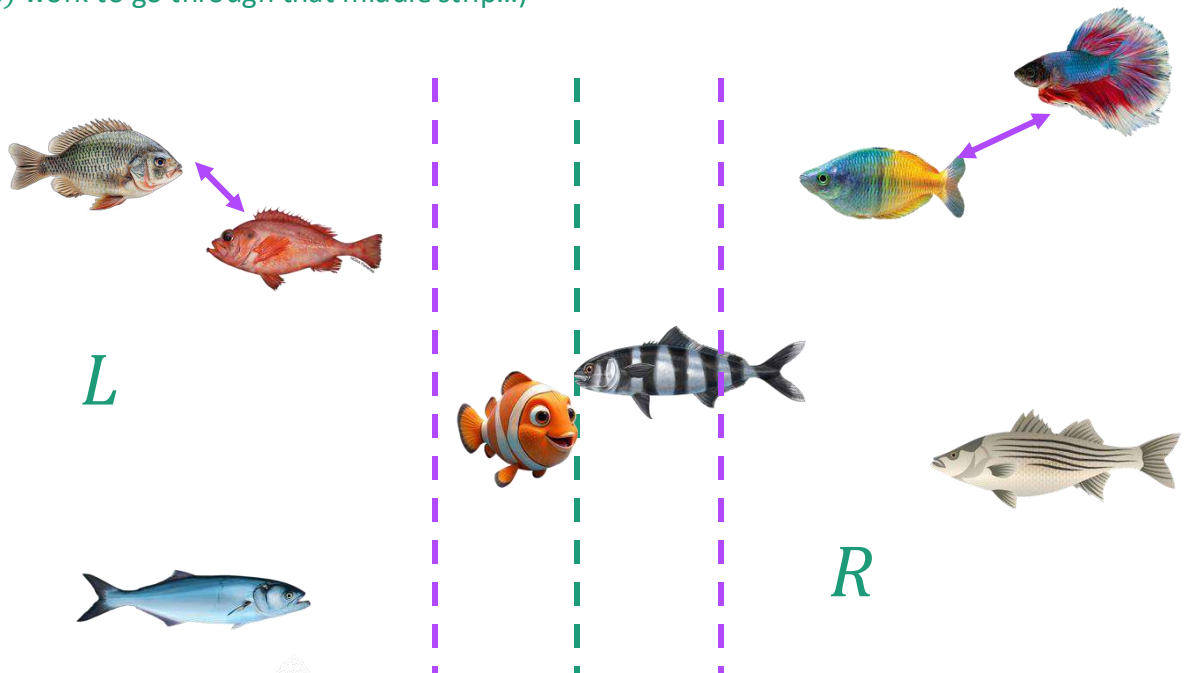
We can actually do this before the pseudocode is fully nailed down, to make sure we're not barking up the wrong tree…

If everything works out, the running time would satisfy $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

(Since we split one size-n problem into two size-n/2 problems, and do $O(n)$ work to go through that middle strip…)

We've seen that before! It's $O(n \log n)$. Yay!

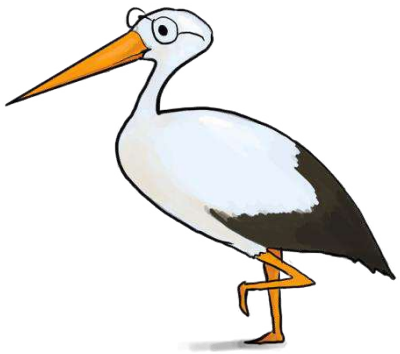
(Okay, **now** we can finish writing it up carefully).



One Strategy

1. Identify natural sub-problems
2. Imagine you had the magical ability to solve those natural sub-problems…what would you do?
3. Work out the details

Think about how you could arrive at MergeSort or QuickSort via this strategy!

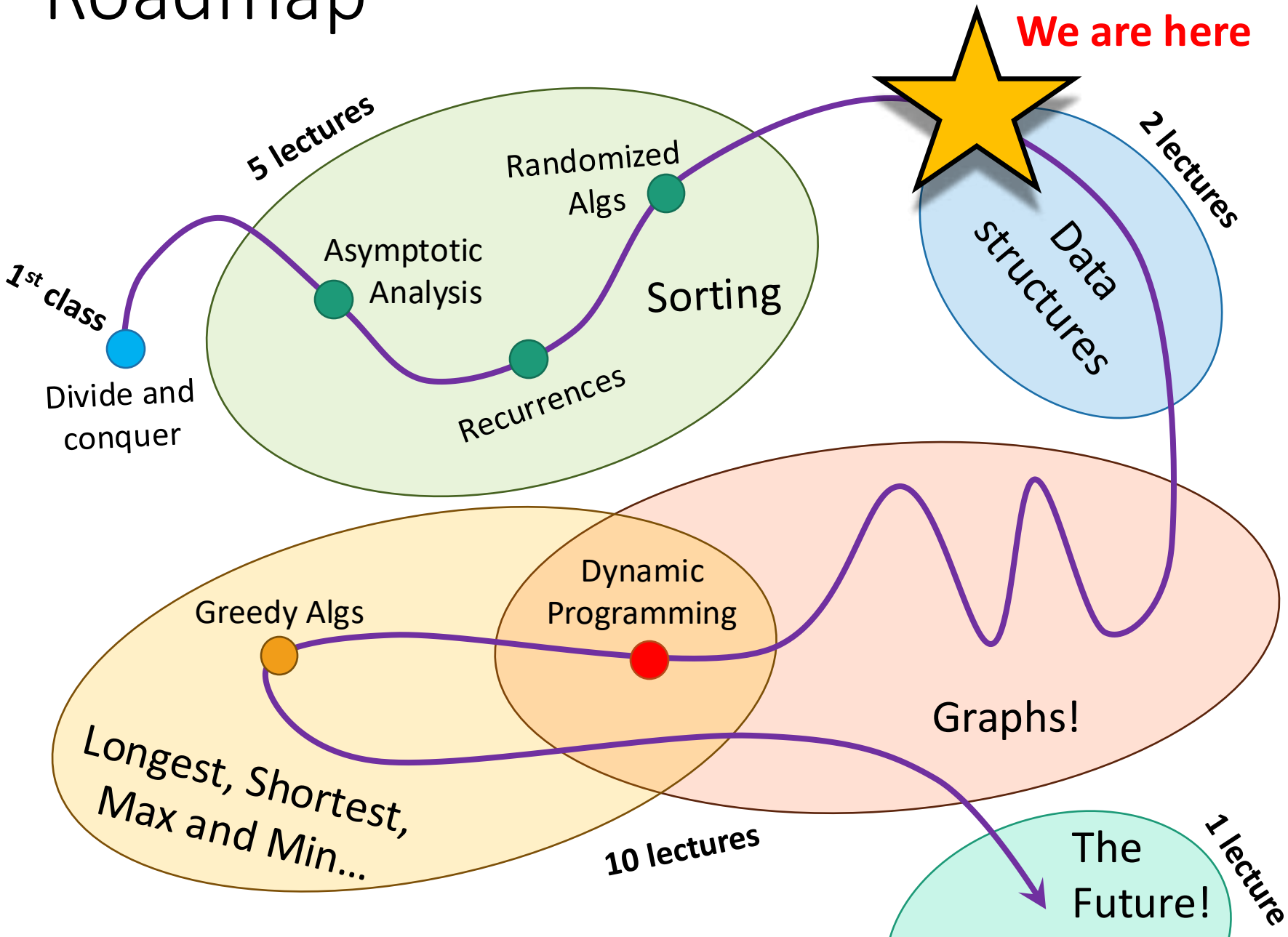


Other tips

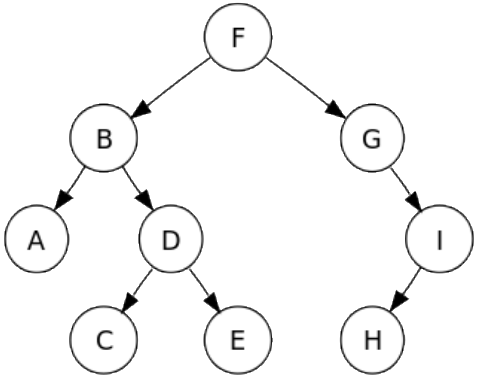
- Small examples.
  - If you have an idea but are having trouble working out the details, try it on a small example by hand.
- Gee, that looks familiar…
  - The more algorithms you see, the easier it will get to come up with new algorithms!
- Bring in your analysis tools.
  - E.g., if I'm doing divide-and-conquer with 2 subproblems of size n/2 and I want an O(n logn) time algorithm, I know that I can afford O(n) work combining my sub-problems.
- Iterate.
  - Darn, that approach didn't work! But, if I tweaked this aspect of it, maybe it works better?
- Everyone approaches problem-solving differently…find the way that works best for you.
  - Check out our **problem-solving guide** on the course website!

More detailed schedule on the website!

Roadmap

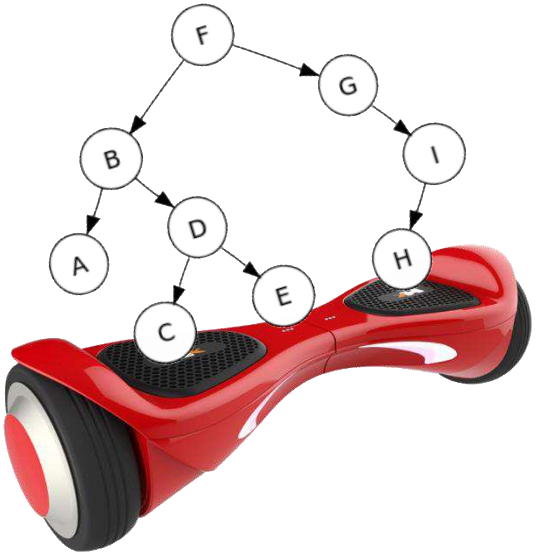


Today



- Begin a brief foray into data structures!
  - See CS 166 for more!
- Binary search trees
  - You may remember these from CS 106B
  - They are better when they're balanced.

this will lead us to…



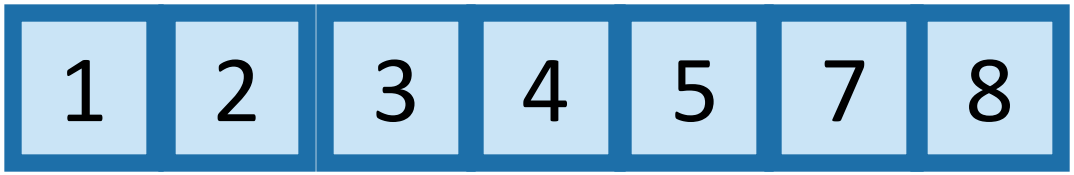
- Self-Balancing Binary Search Trees
  - **Red-Black** trees.

Some data structures

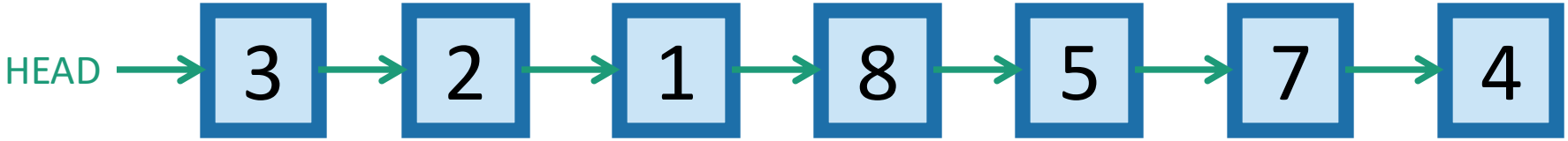
for storing objects like

(aka, nodes with keys)

- (Sorted) arrays:



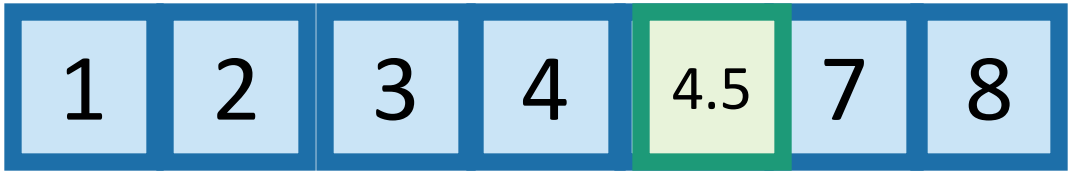
- Linked lists:



- Some basic operations:
  - INSERT, DELETE, SEARCH

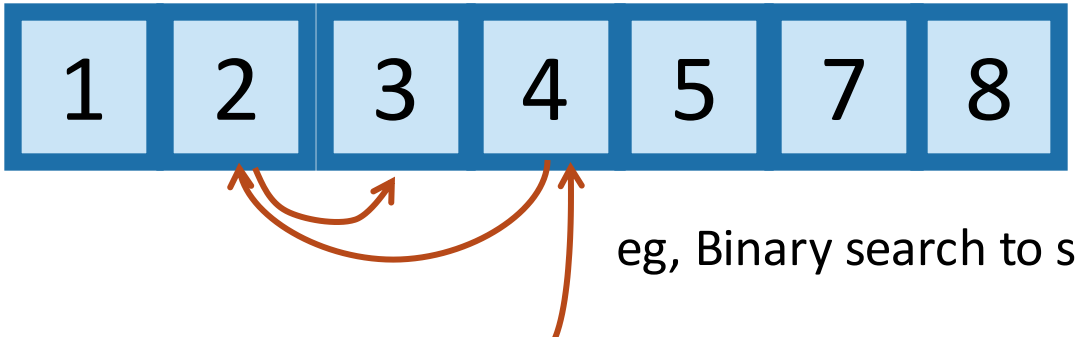
Sorted Arrays

- O(n) INSERT/DELETE:
  - First, find the relevant element (we'll see how below), and then move a bunch elements in the array:



eg, insert 4.5

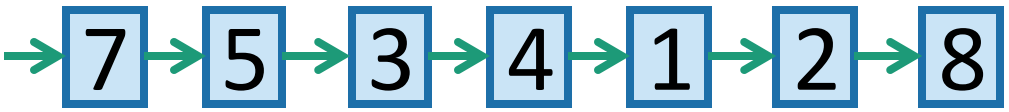
- O(log(n)) SEARCH:



eg, Binary search to see if 3 is in A.

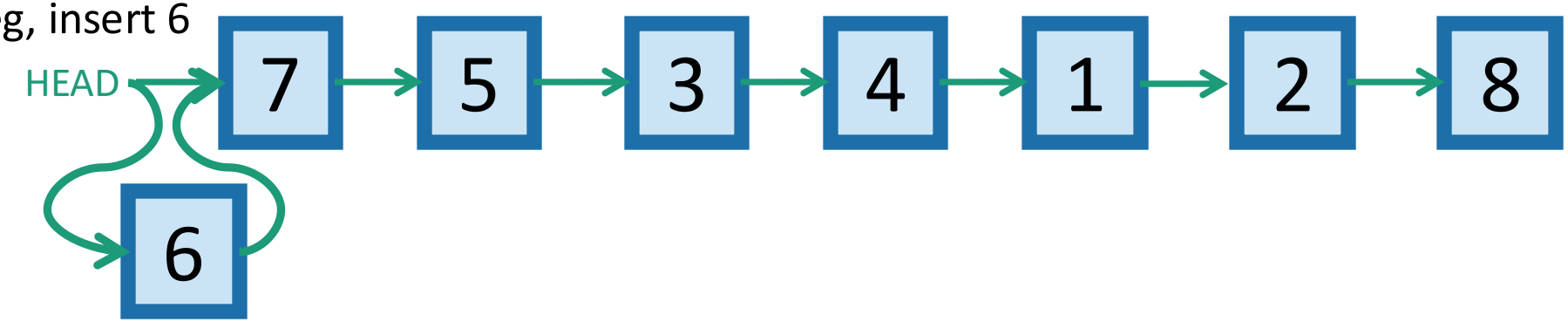
(Not necessarily sorted)

Linked lists

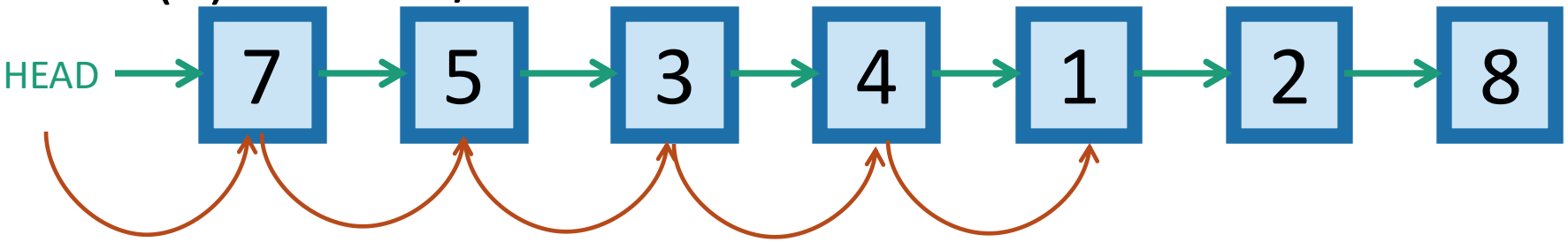


- O(1) INSERT:

eg, insert 6



- O(n) SEARCH/DELETE:



eg, search for 1 (and then you could delete it by manipulating pointers).

Motivation for Binary Search Trees

	Sorted Arrays	Linked Lists
Search	O(log(n))	O(n)
Delete	O(n)	O(n)
Insert	O(n)	O(1)

Motivation for Binary Search Trees

TODAY!

	Sorted Arrays	Linked Lists	(Balanced) Binary Search Trees
Search	O(log(n))	O(n)	O(log(n))
Delete	O(n)	O(n)	O(log(n))
Insert	O(n)	O(1)	O(log(n))

For today all keys are distinct.

Binary tree terminology

Each node has two children.

The left child of 3 is 2

The right child of 3 is 4

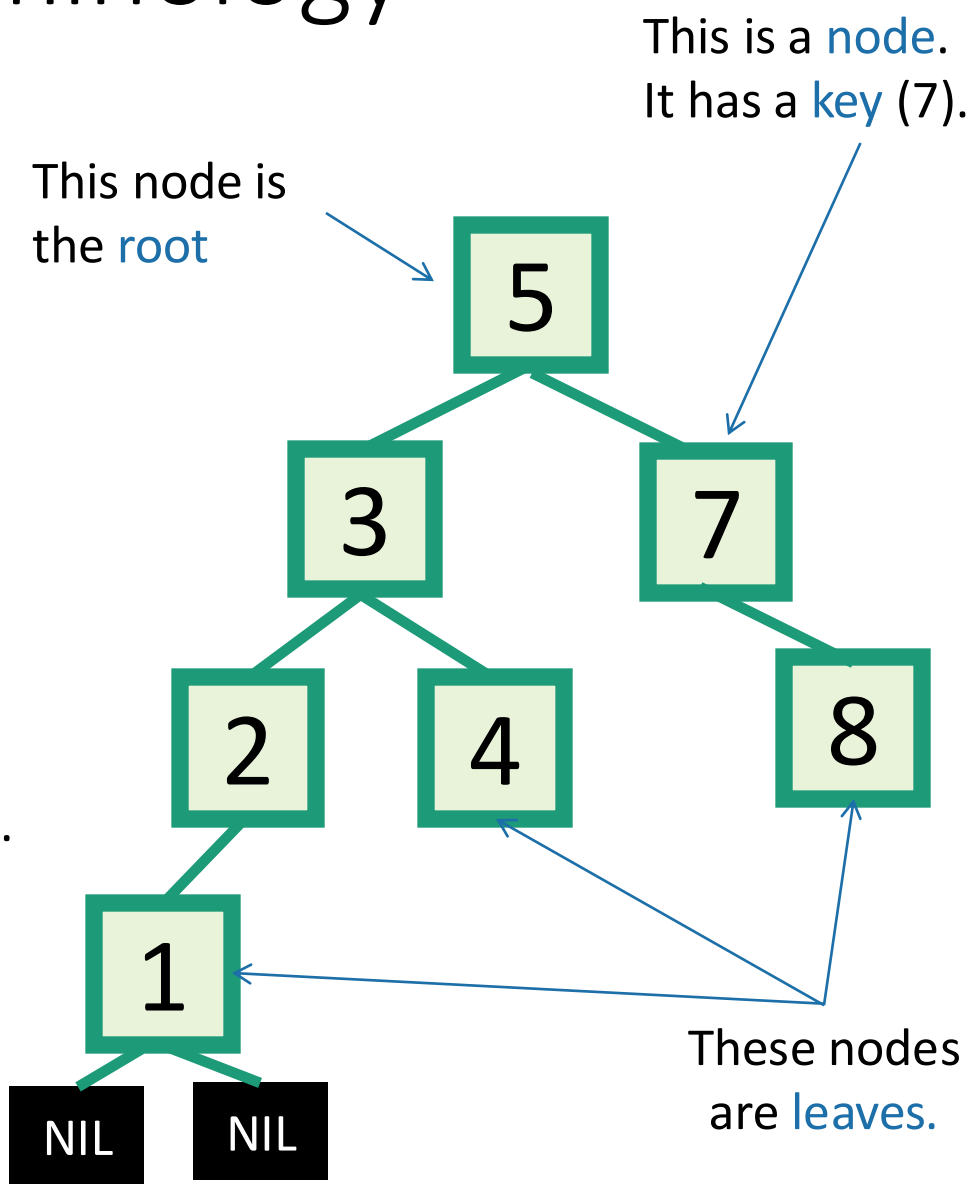
The parent of 3 is 5

2 is a descendant of 5

Each node has a pointer to its left child, right child, and parent.

Both children of 1 are NIL.
(I won't usually draw them).

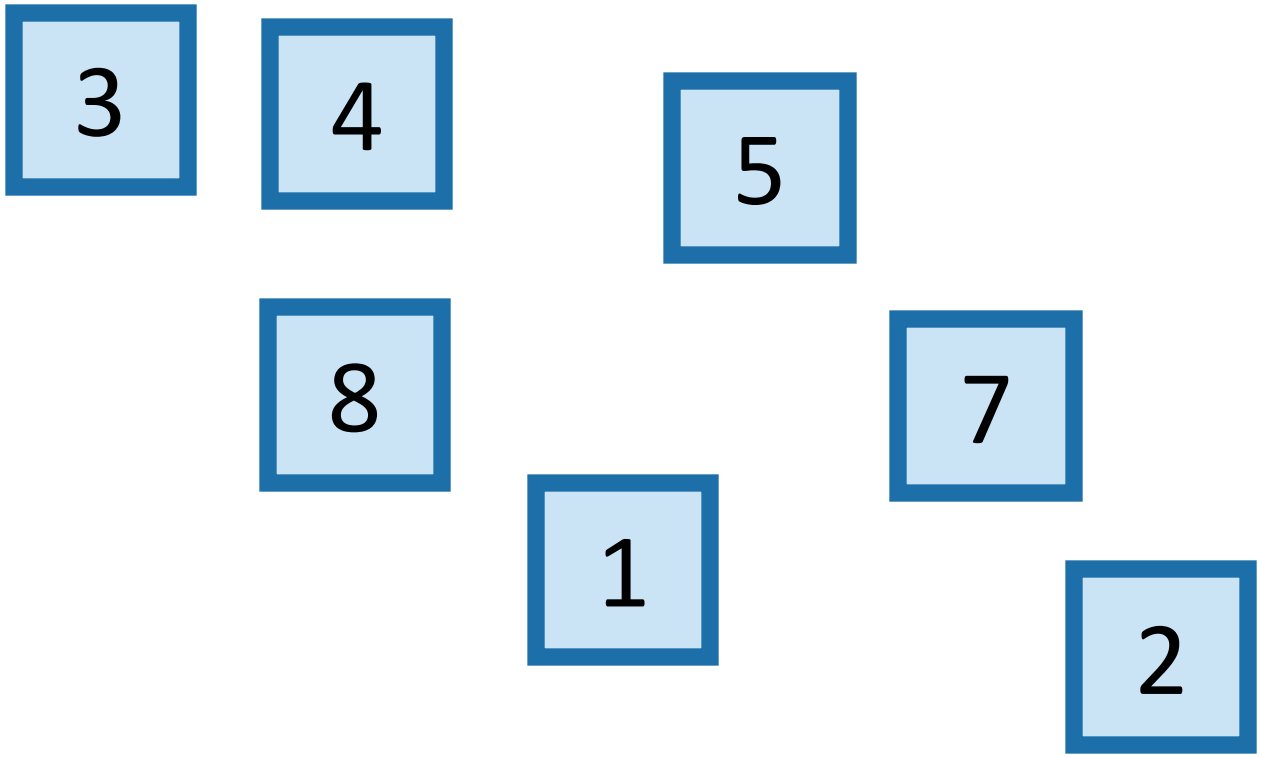
The height of this tree is 3.
(Max number of edges from the root to a leaf).



From your pre-lecture exercise…

Binary Search Trees

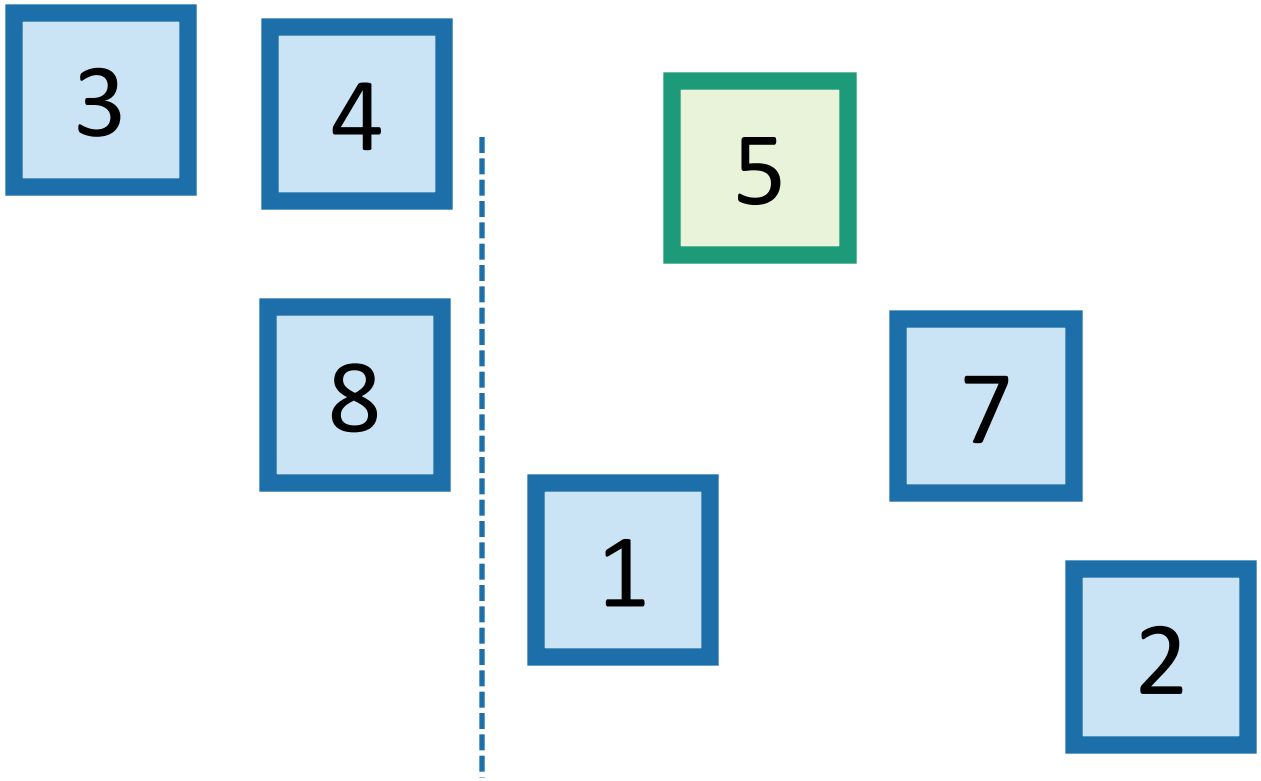
- A BST is a binary tree so that:
  - Every LEFT descendant of a node has key less than that node.
  - Every RIGHT descendant of a node has key larger than that node.
- Example of building a binary search tree:



From your pre-lecture exercise…

Binary Search Trees

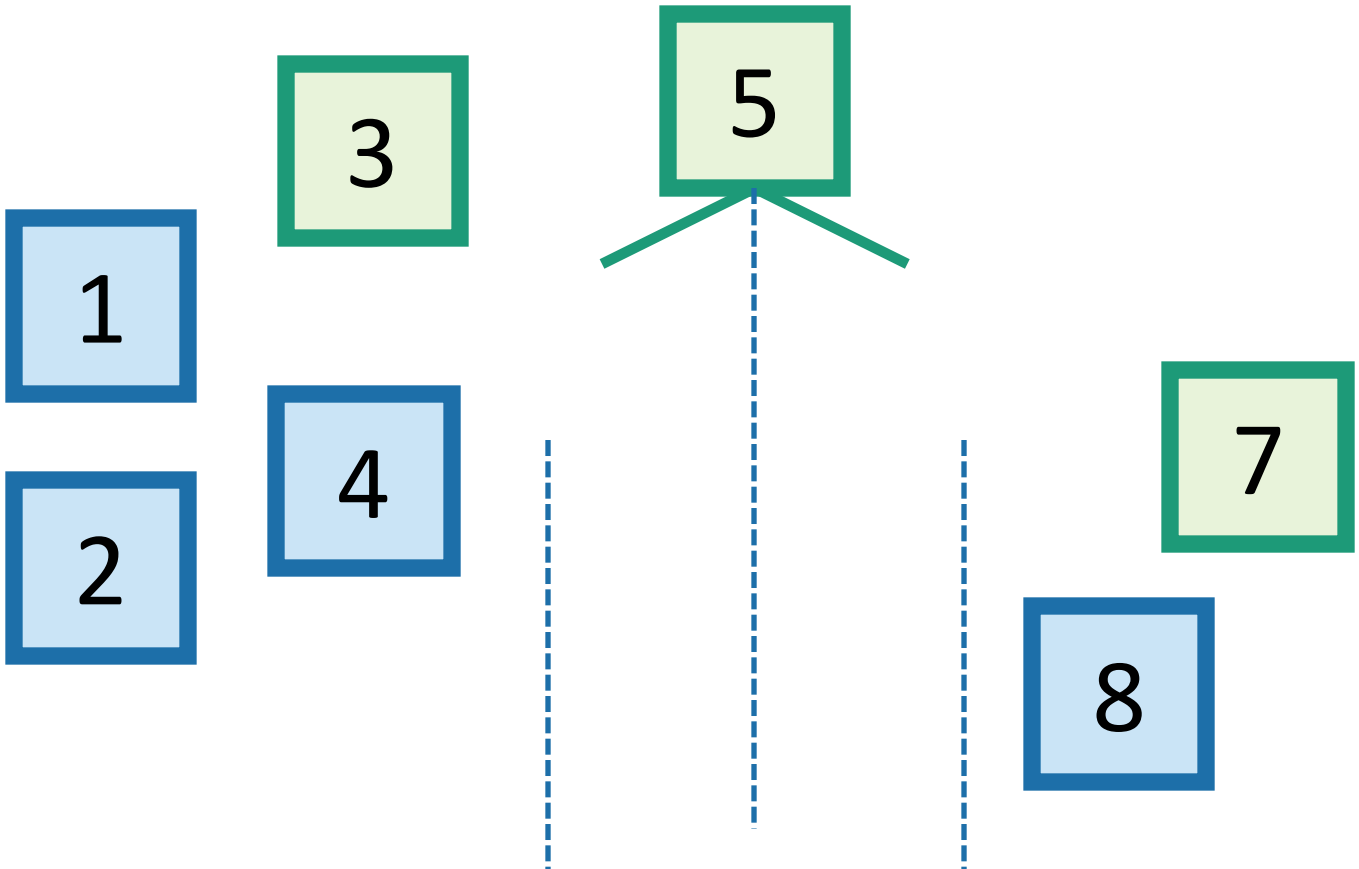
- A BST is a binary tree so that:
  - Every LEFT descendant of a node has key less than that node.
  - Every RIGHT descendant of a node has key larger than that node.
- Example of building a binary search tree:



From your pre-lecture exercise…

Binary Search Trees

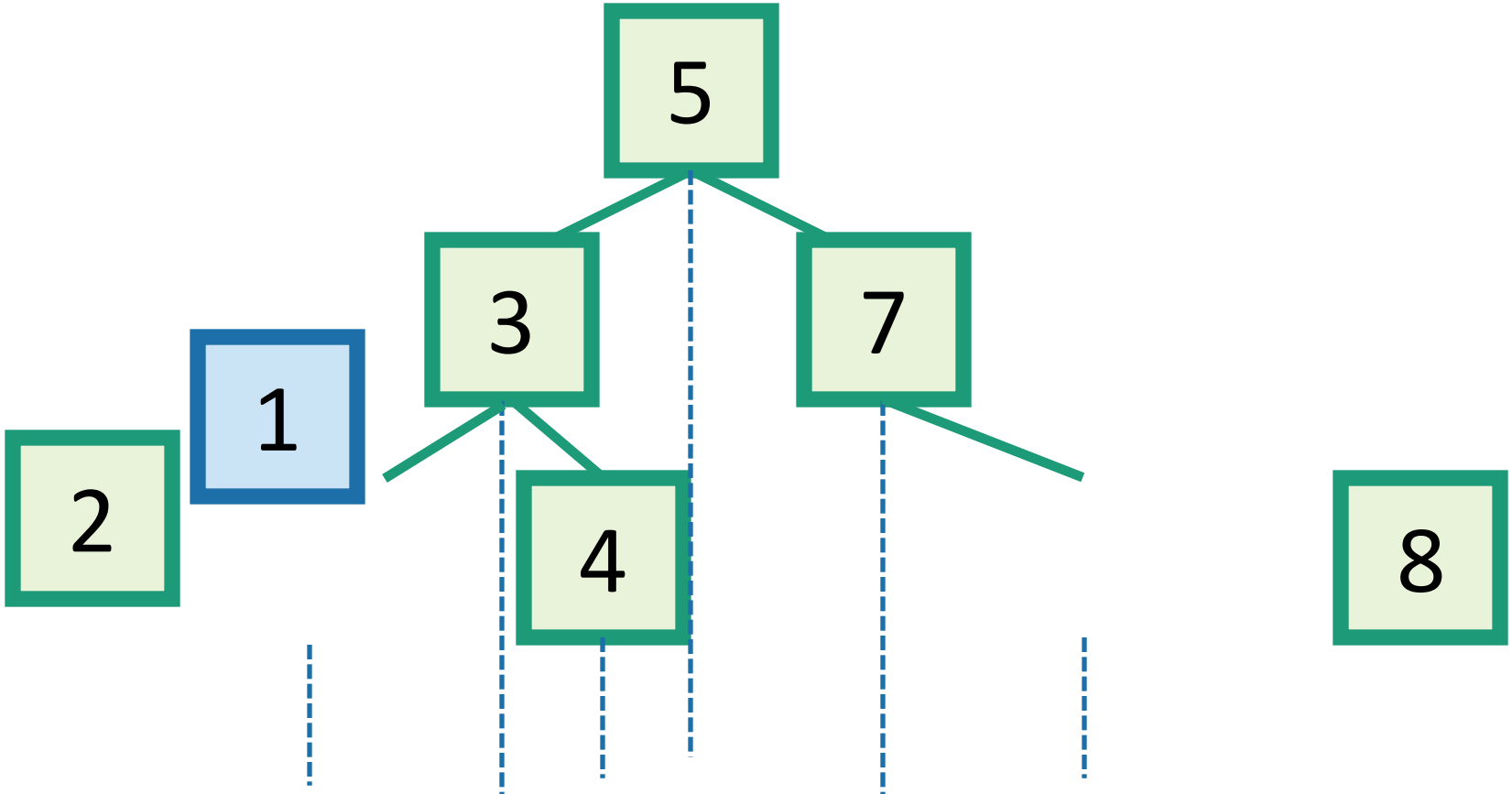
- A BST is a binary tree so that:
  - Every LEFT descendant of a node has key less than that node.
  - Every RIGHT descendant of a node has key larger than that node.
- Example of building a binary search tree:



From your pre-lecture exercise…

Binary Search Trees

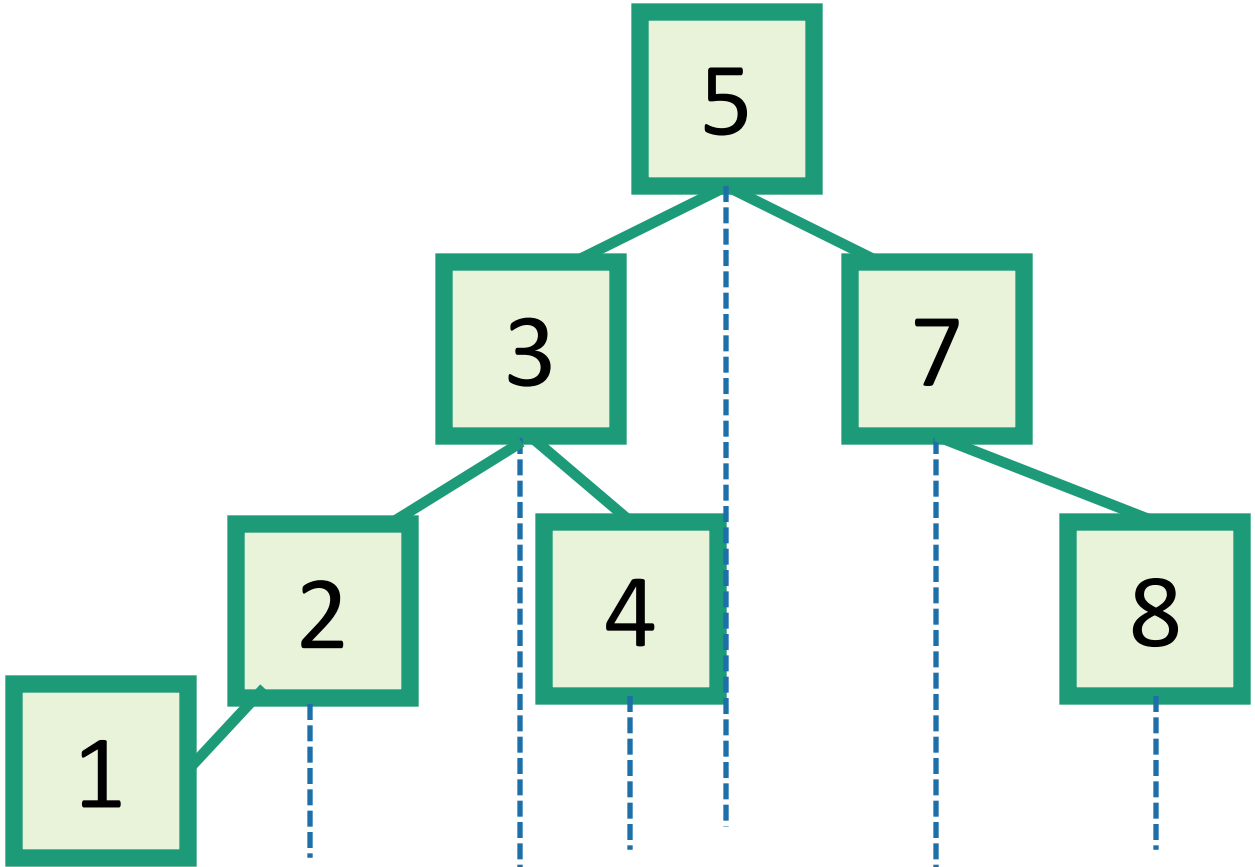
- A BST is a binary tree so that:
  - Every LEFT descendant of a node has key less than that node.
  - Every RIGHT descendant of a node has key larger than that node.
- Example of building a binary search tree:



From your pre-lecture exercise…

Binary Search Trees

- A BST is a binary tree so that:
  - Every LEFT descendant of a node has key less than that node.
  - Every RIGHT descendant of a node has key larger than that node.
- Example of building a binary search tree:

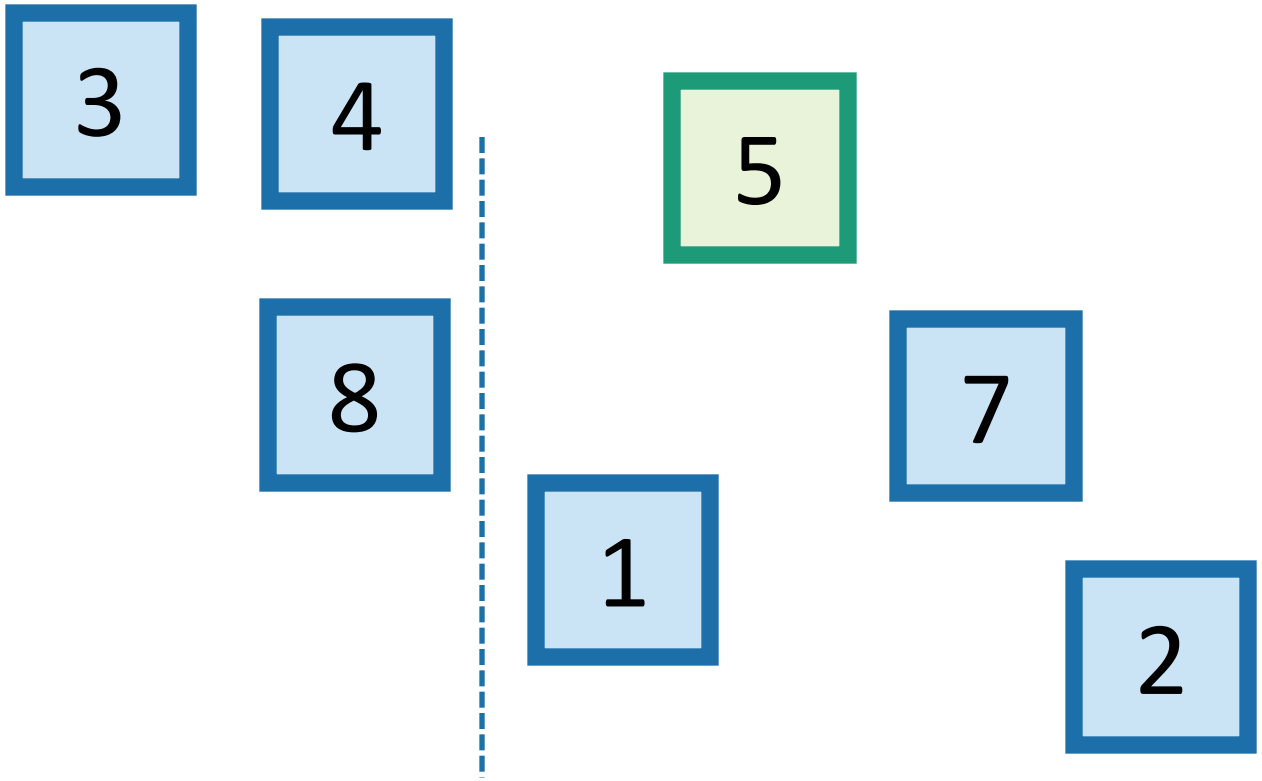


Q: Is this the only binary search tree I could possibly build with these values?

A: **No.** I made choices about which nodes to choose when. Any choices would have been fine.

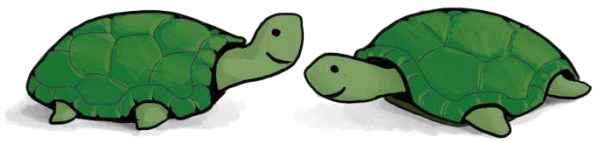
Aside: this should look familiar

kinda like QuickSort

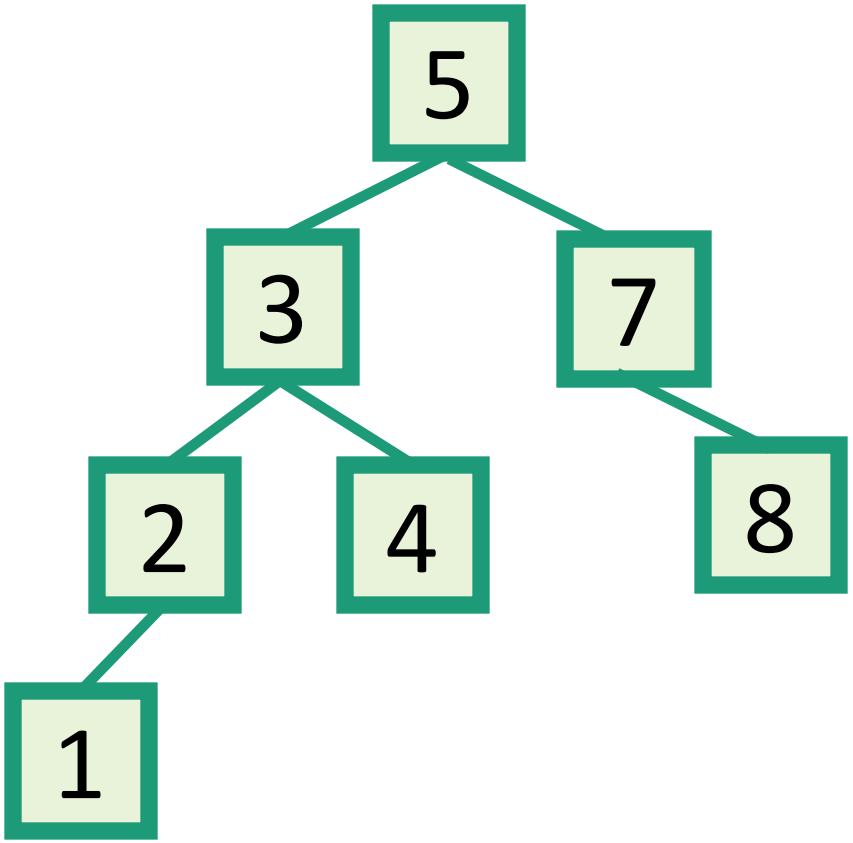


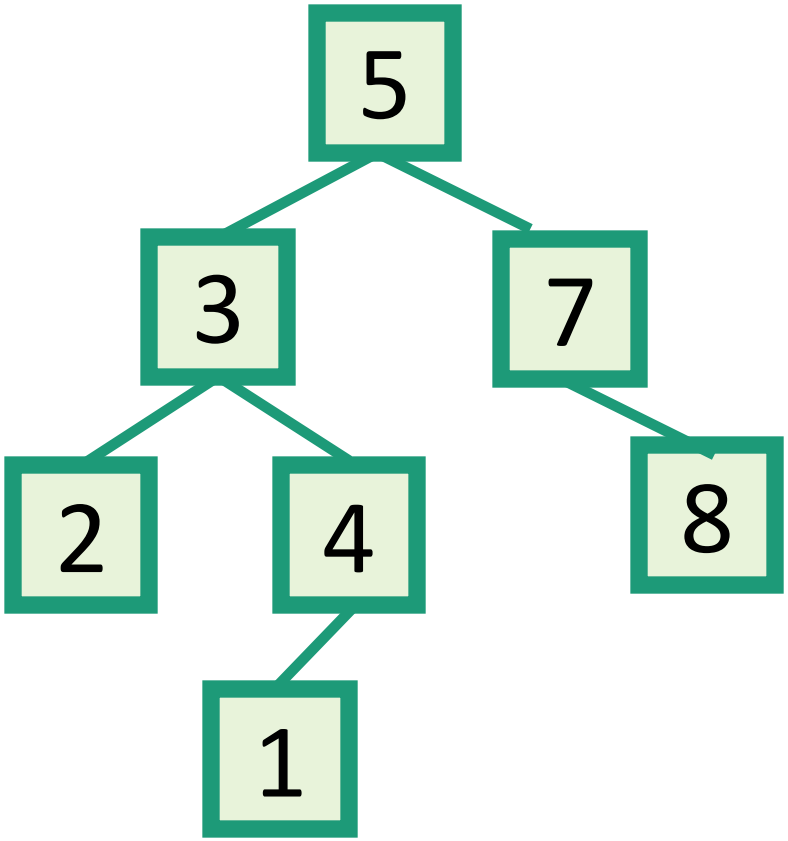
Binary Search Trees

Which of these is a BST?



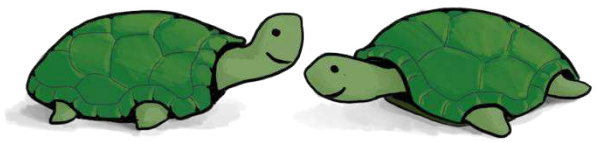
- A BST is a binary tree so that:
  - Every LEFT descendant of a node has key less than that node.
  - Every RIGHT descendant of a node has key larger than that node.



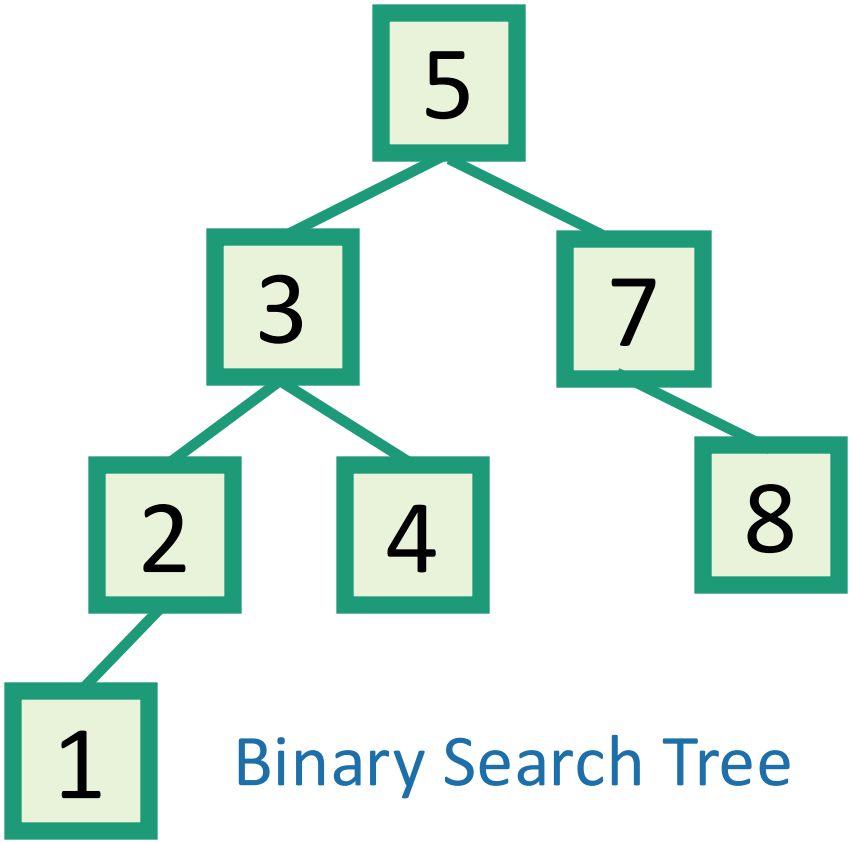


Binary Search Trees

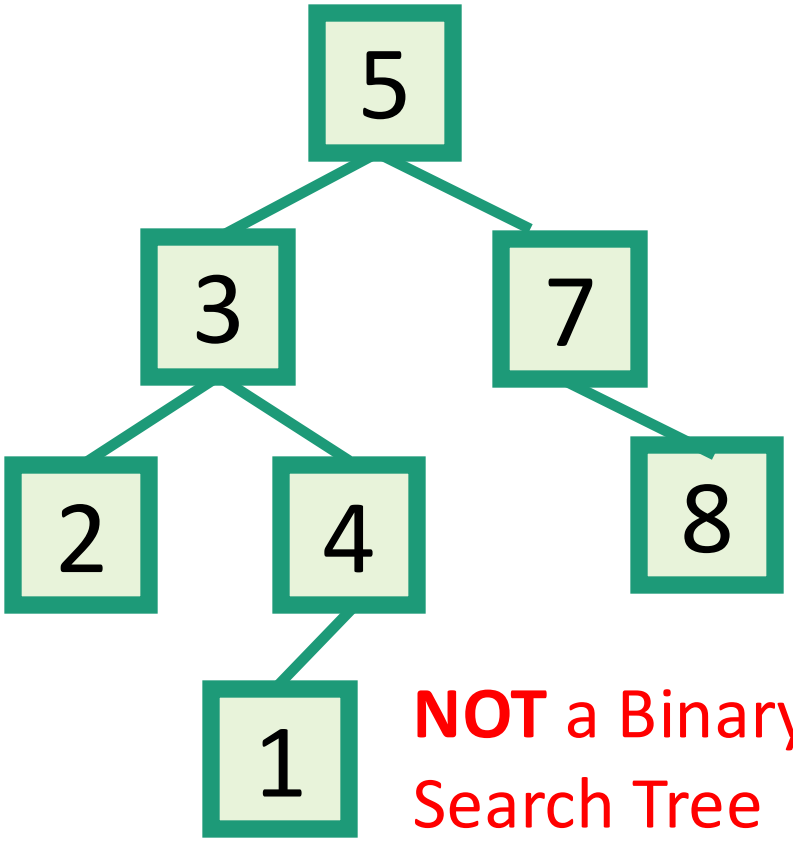
Which of these is a BST?



- A BST is a binary tree so that:
  - Every LEFT descendant of a node has key less than that node.
  - Every RIGHT descendant of a node has key larger than that node.



Binary Search Tree

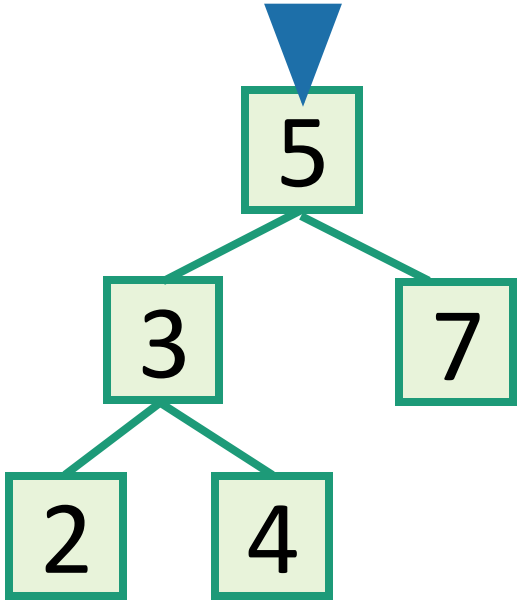


NOT a Binary Search Tree

Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

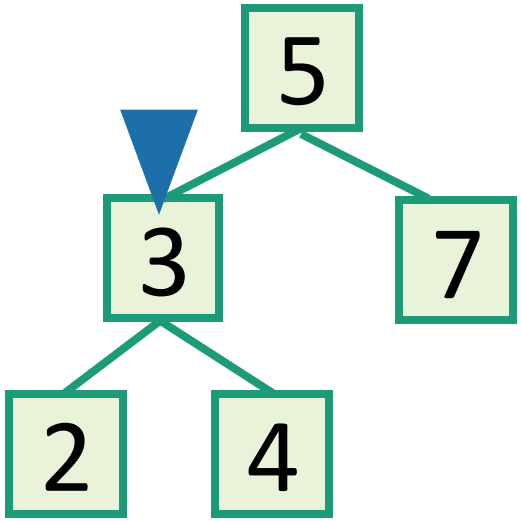
- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)



Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

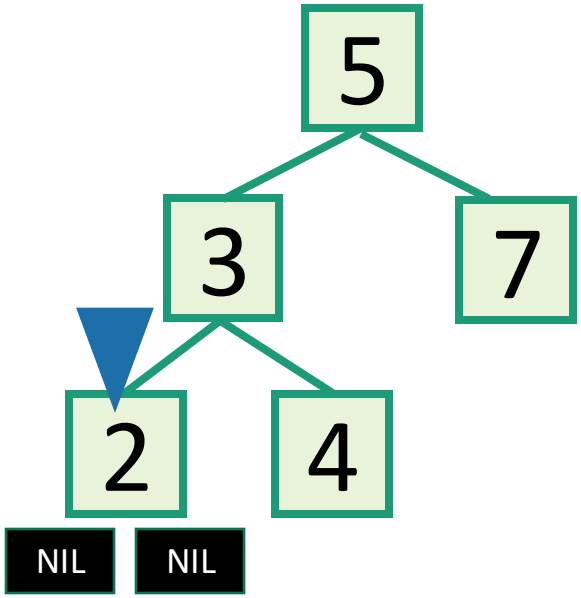
- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)



Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

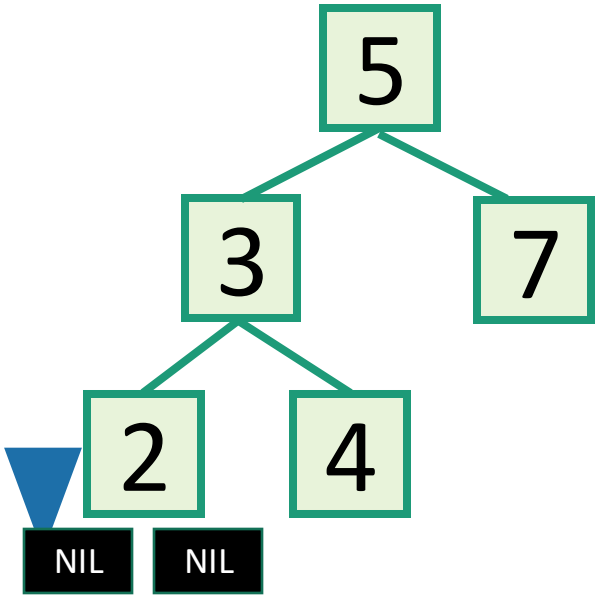
- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)



Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

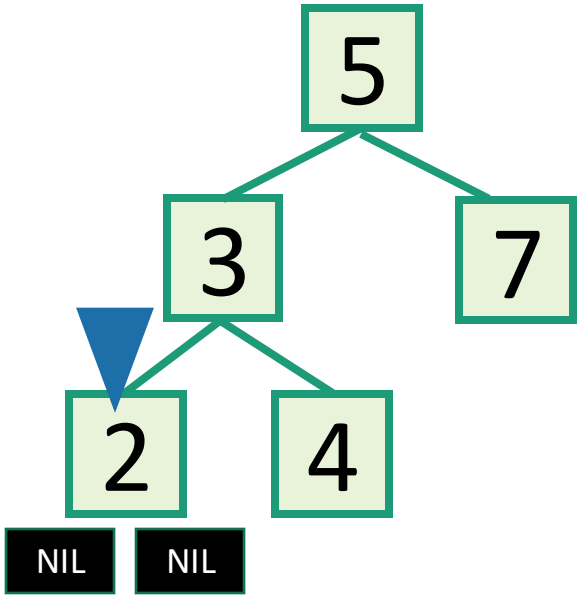
- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)



Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)

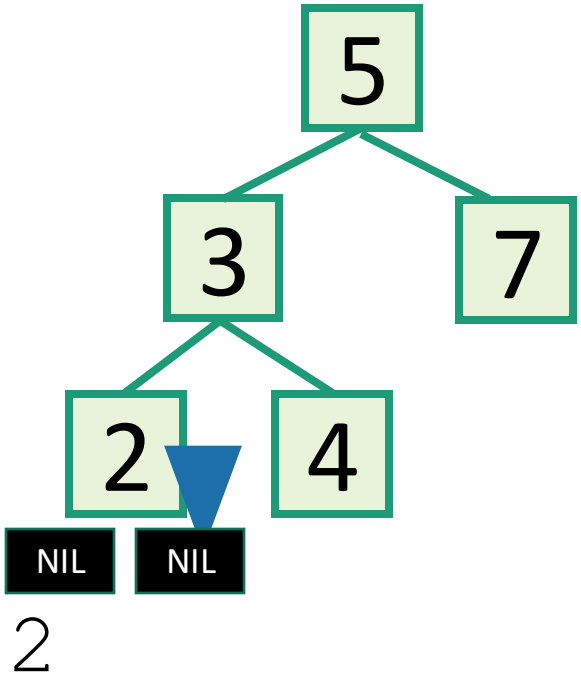


2

Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

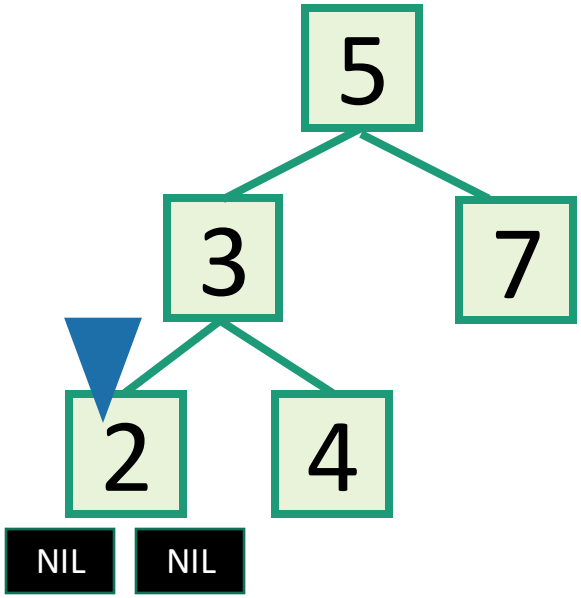
- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)



Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)

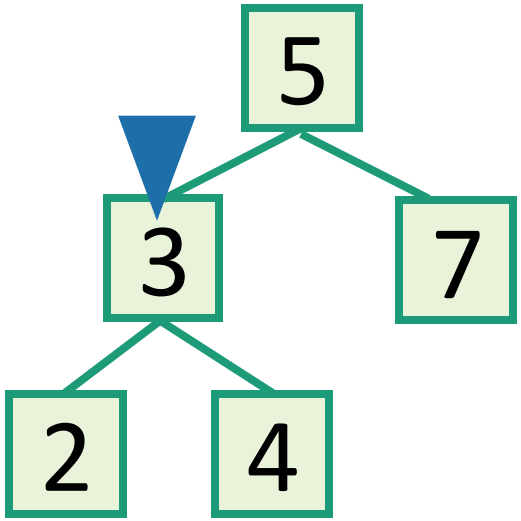


2

Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)

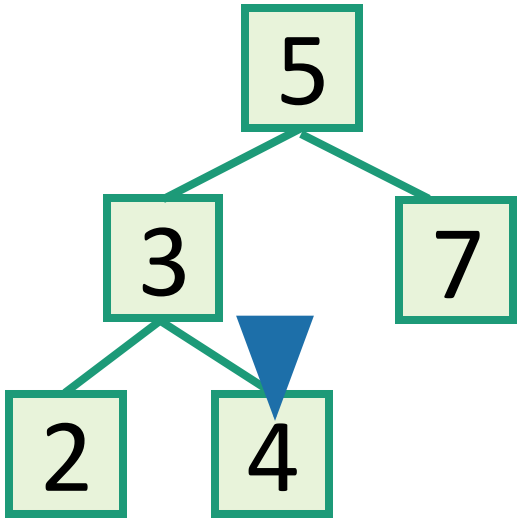


2 3

Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)

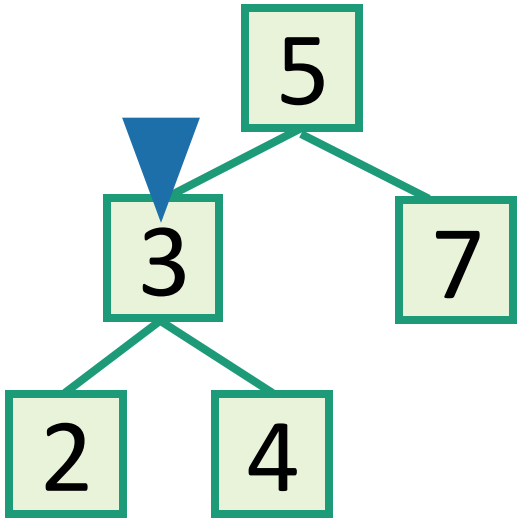


2 3 4

Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)

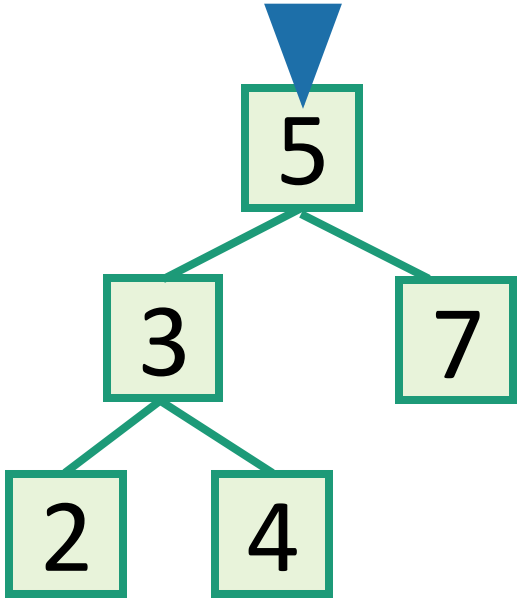


2 3 4

Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)

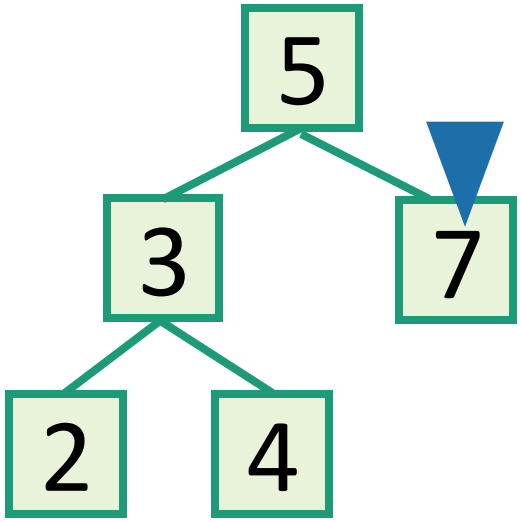


2 3 4 5

Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)

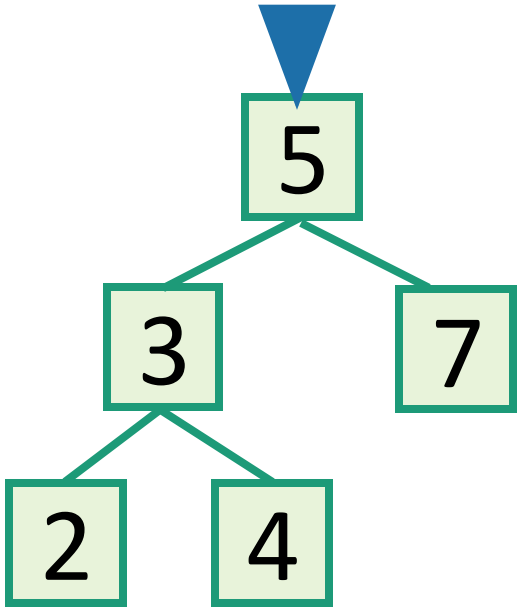


2 3 4 5 7

Aside: In-Order Traversal of BSTs

- Output all the elements in sorted order!

- inOrderTraversal(x):
  - if x!= NIL:
    - inOrderTraversal(x.left)
    - print(x.key)
    - inOrderTraversal(x.right)



- Runs in time O(n).

2 3 4 5 7 Sorted!

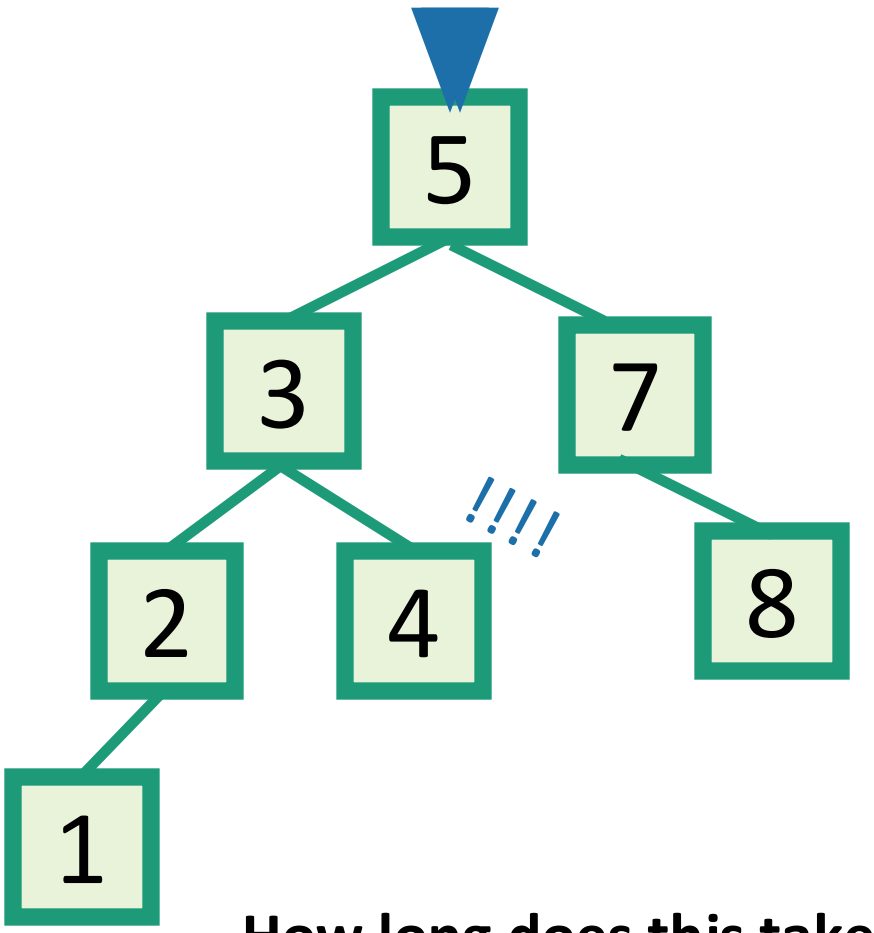
Back to the goal

Fast SEARCH/INSERT/DELETE

Can we do these?

SEARCH in a Binary Search Tree

definition by example

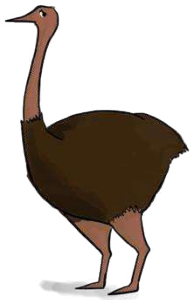


EXAMPLE: Search for 4.

EXAMPLE: Search for 4.5

- It turns out it will be convenient to **return 4** in this case
- (that is, **return** the last node before we went off the tree)

Write pseudocode (or actual code) to implement this!

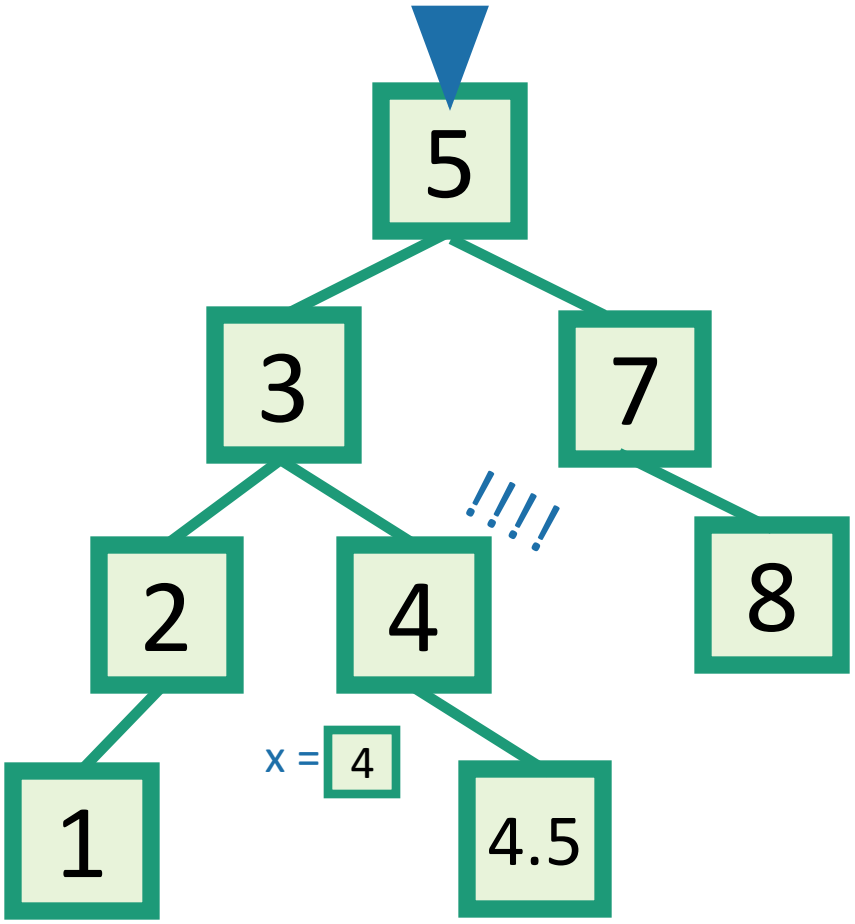


Ollie the over-achieving ostrich

How long does this take?

O(length of longest path) = O(height)

INSERT in a Binary Search Tree



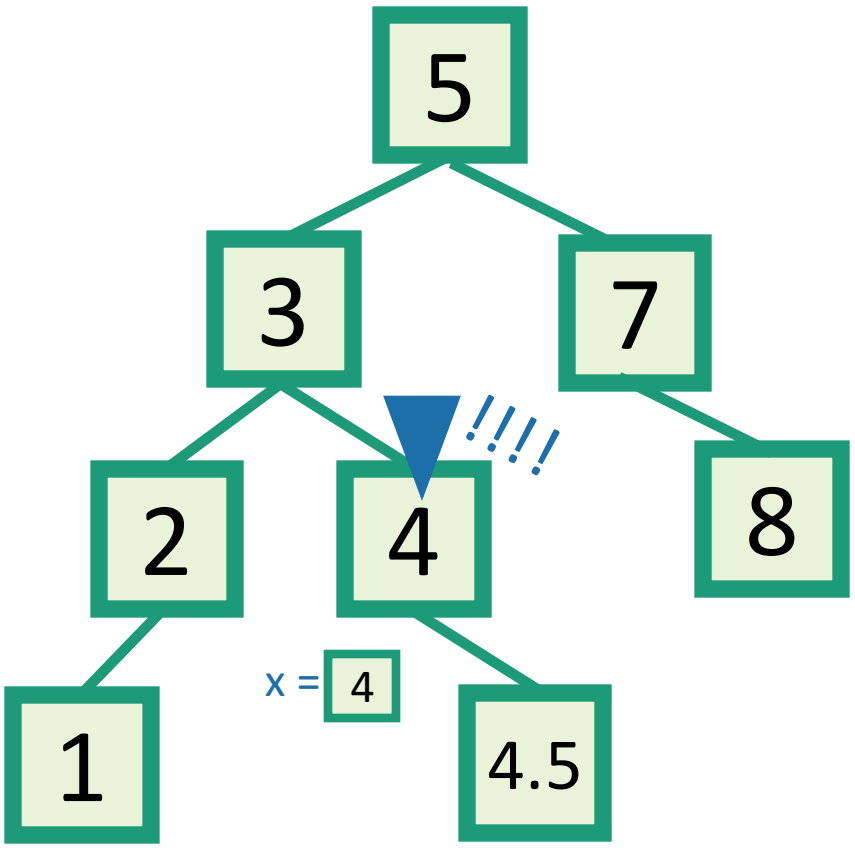
EXAMPLE: Insert 4.5

- INSERT(key):
  - x = SEARCH(key)
  - **Insert** a new node with desired key at x…

You thought about this on your pre-lecture exercise! (See skipped slide for pseudocode.)

INSERT in a Binary Search Tree

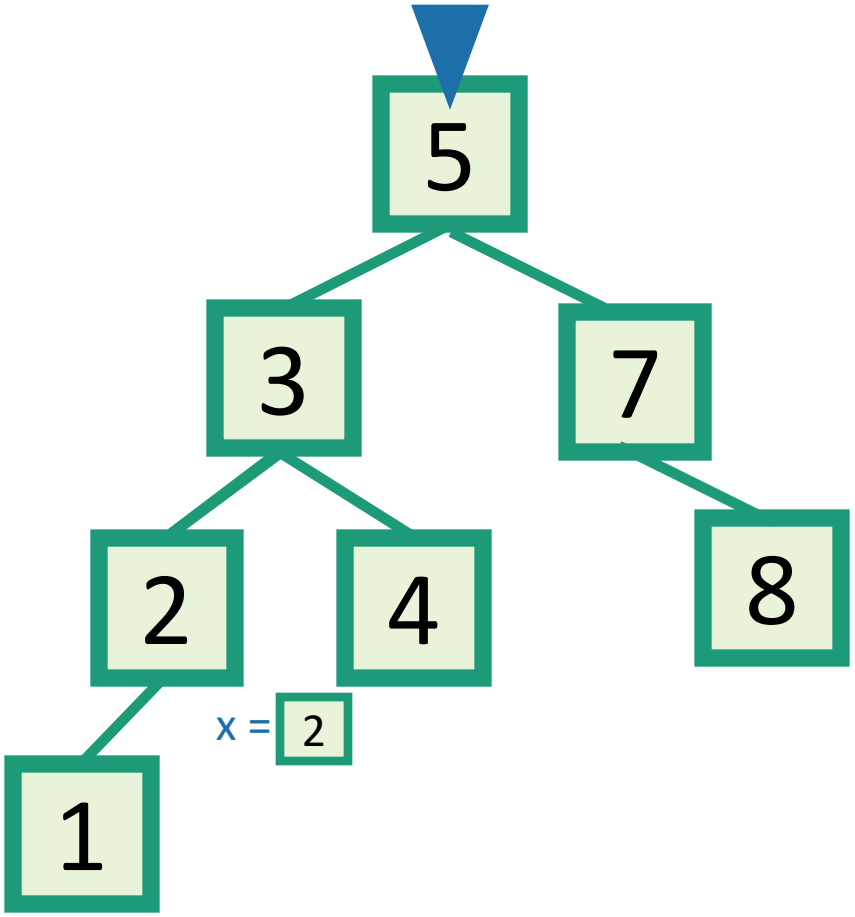
This slide skipped in class – here for reference



EXAMPLE: Insert 4.5

- INSERT(key):
  - x = SEARCH(key)
  - **if** key > x.key:
    - Make a new node with the correct key, and put it as the right child of x.
  - **if** key < x.key:
    - Make a new node with the correct key, and put it as the left child of x.
  - **if** x.key == key:
    - **return**

DELETE in a Binary Search Tree



EXAMPLE: Delete 2

- DELETE(key):
  - x = SEARCH(key)
  - **if** x.key == key:
    - ….delete x….

You thought about this in your pre-lecture exercise too!

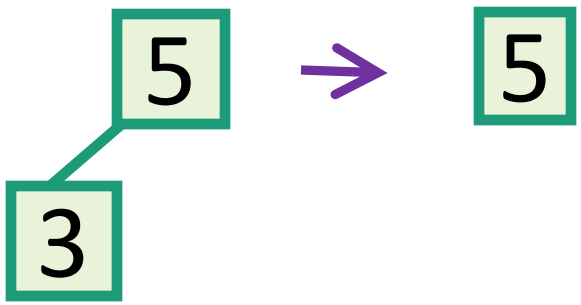
This is a bit more complicated…see the skipped slides for some pictures of the different cases.

DELETE in a Binary Search Tree

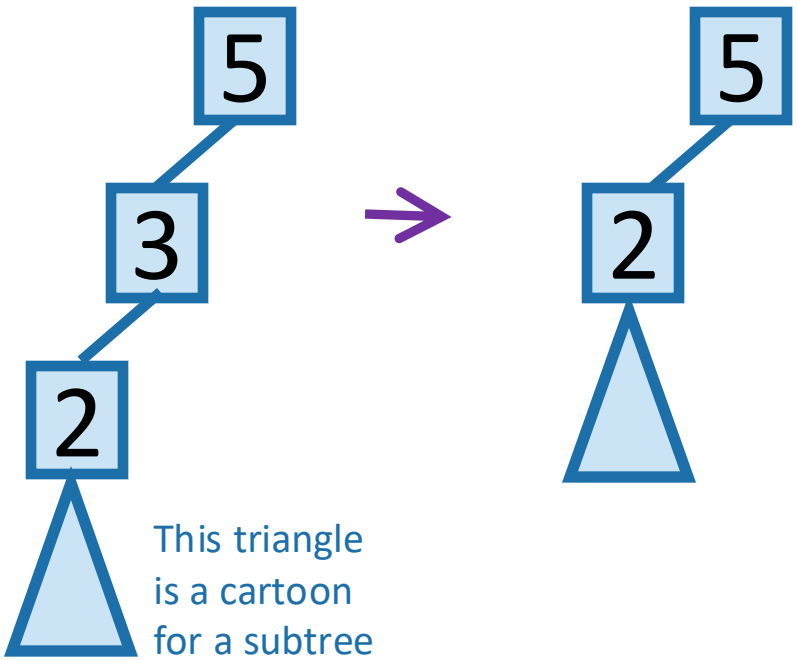
This slide skipped in class – here for reference!

several cases (by example)

say we want to delete 3



Case 1: if 3 is a leaf, just delete it.



This triangle is a cartoon for a subtree

Case 2: if 3 has just one child, move that up.

Write pseudocode for all of these!

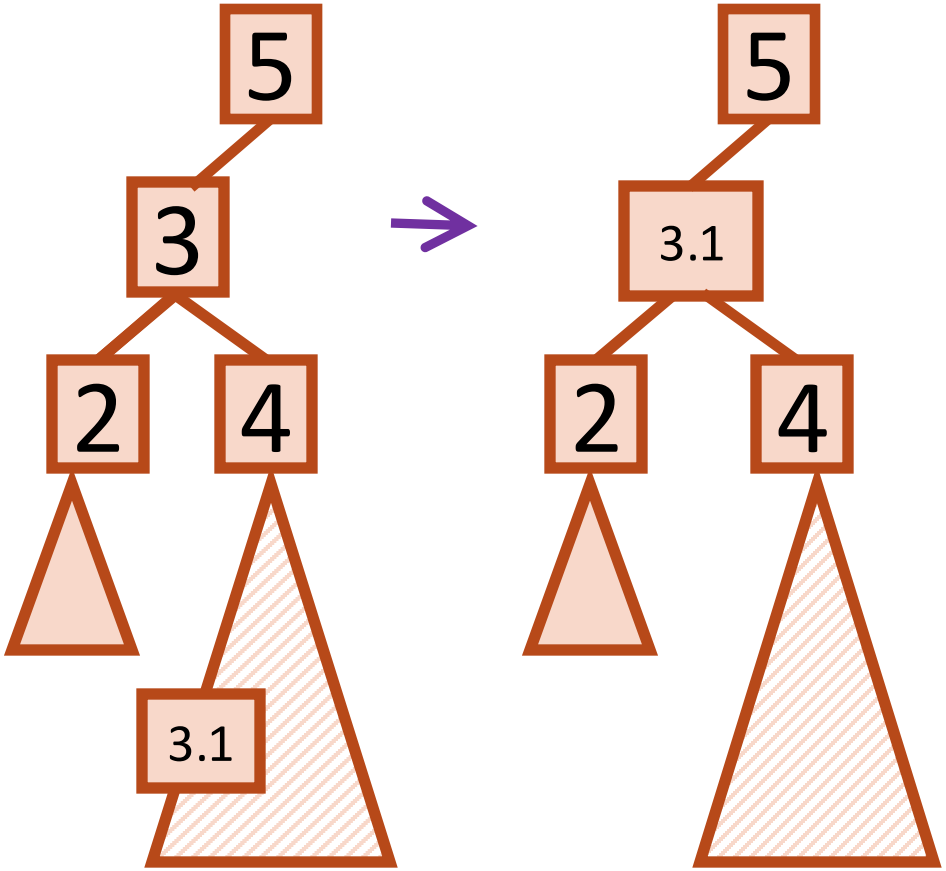
Siggi the Studious Stork

DELETE in a Binary Search Tree

ctd.

This slide skipped in class – here for reference!

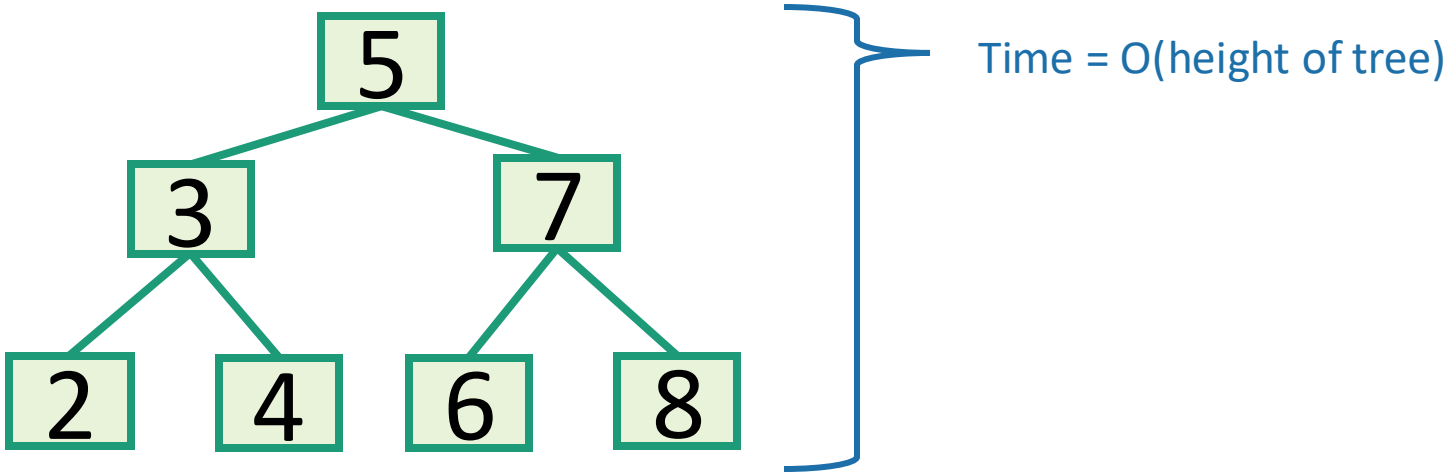
Case 3: if 3 has two children,
replace 3 with it's immediate successor.
(aka, next biggest thing after 3)



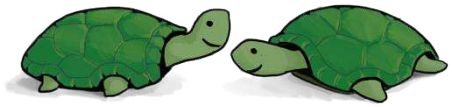
- Does this maintain the BST property?
  - Yes.
- How do we find the immediate successor?
  - SEARCH for 3 in the subtree under 3.right
- How do we remove it when we find it?
  - If [3.1] has 0 or 1 children, do one of the previous cases.
- What if [3.1] has two children?
  - It doesn't.

How long do these operations take?

- SEARCH is the big one.
  - Everything else just calls SEARCH and then does some small O(1)-time operation.

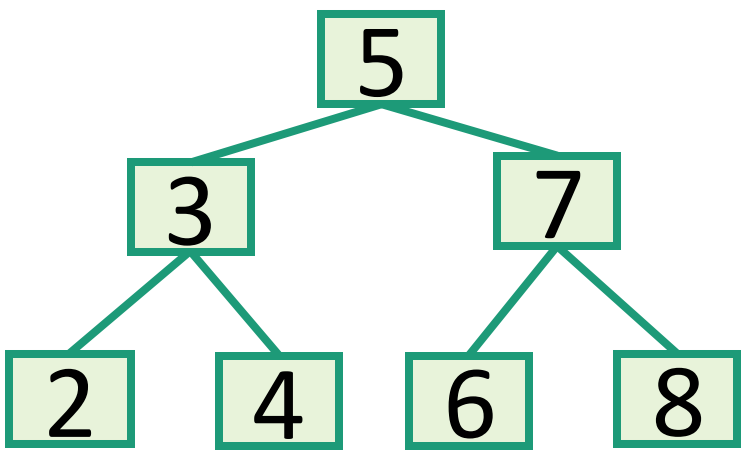


How long does search take?



How long do these operations take?

- SEARCH is the big one.
  - Everything else just calls SEARCH and then does some small O(1)-time operation.



Time = O(height of tree)

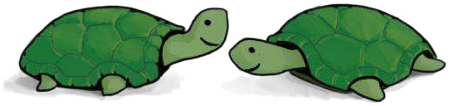
Trees have depth O(log(n)). **Done!**

Lucky the lackadaisical lemur.

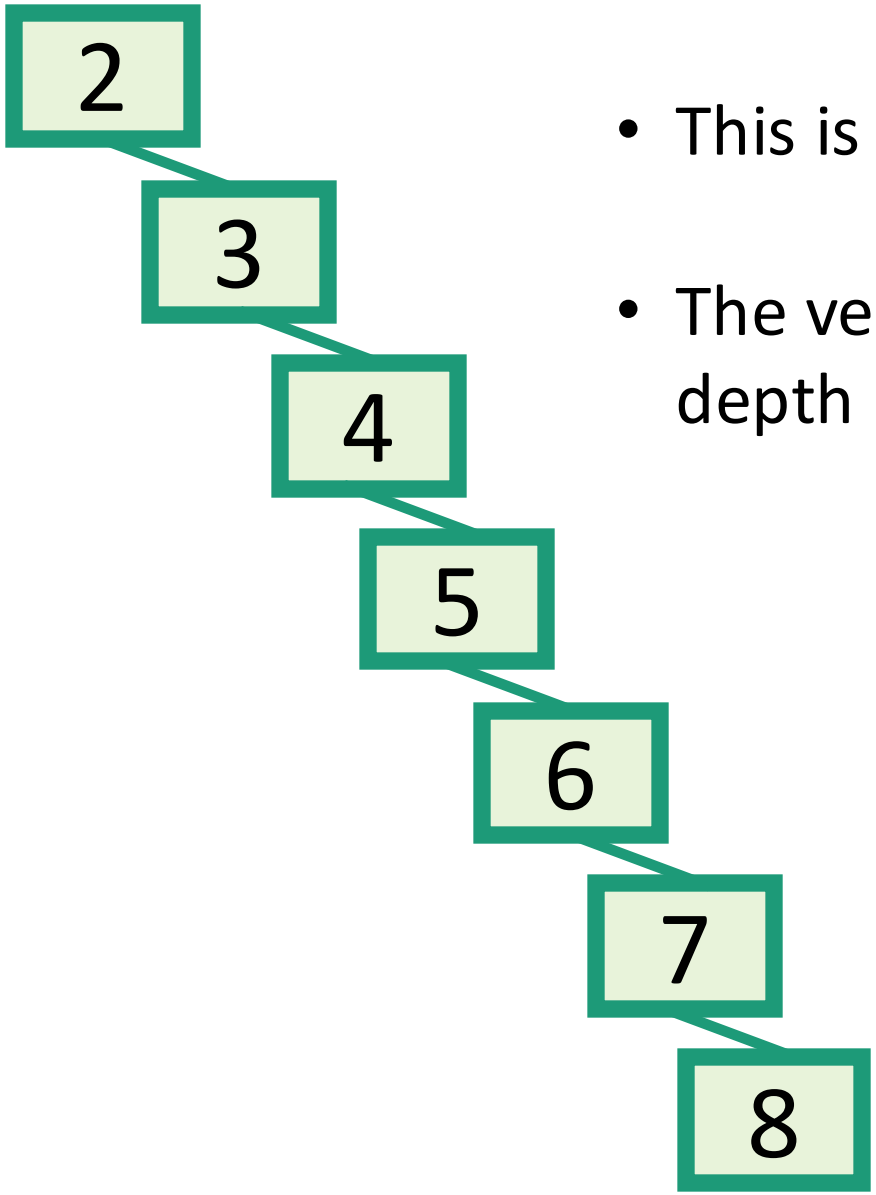
Wait a second…

Plucky the Pedantic Penguin

How long does search take?



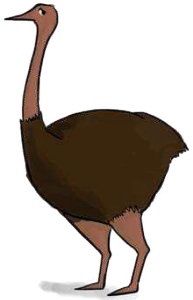
Search might take time O(n).



- This is a valid binary search tree.
- The version with n nodes has depth n, **not** O(log(n)).

What to do?

How often is "every so often" in the worst case? It's actually pretty often!



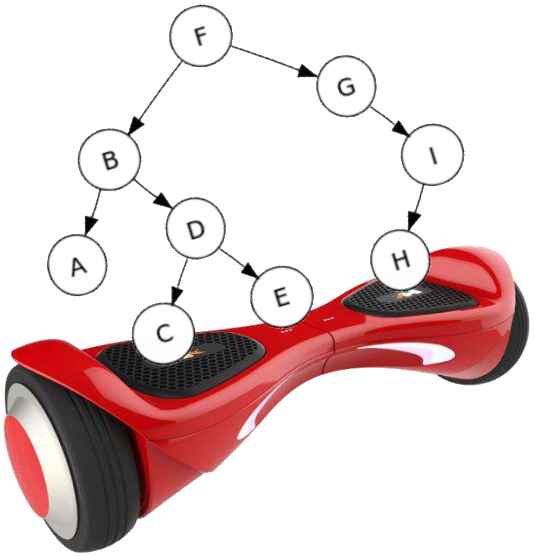
Ollie the over-achieving ostrich

- Goal: Fast SEARCH/INSERT/DELETE
- All these things take time O(height)
- And the height might be big!!! ☹

- Idea 0:
  - Keep track of how deep the tree is getting.
  - If it gets too tall, re-do everything from scratch.
    - At least $\Omega(n)$ every so often….

- Turns out that's not a great idea. Instead we turn to…

Self-Balancing Binary Search Trees

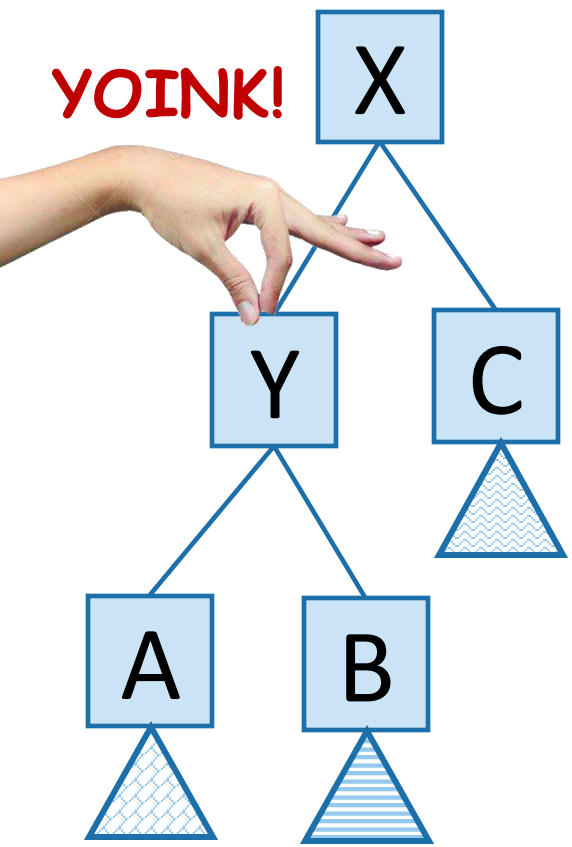


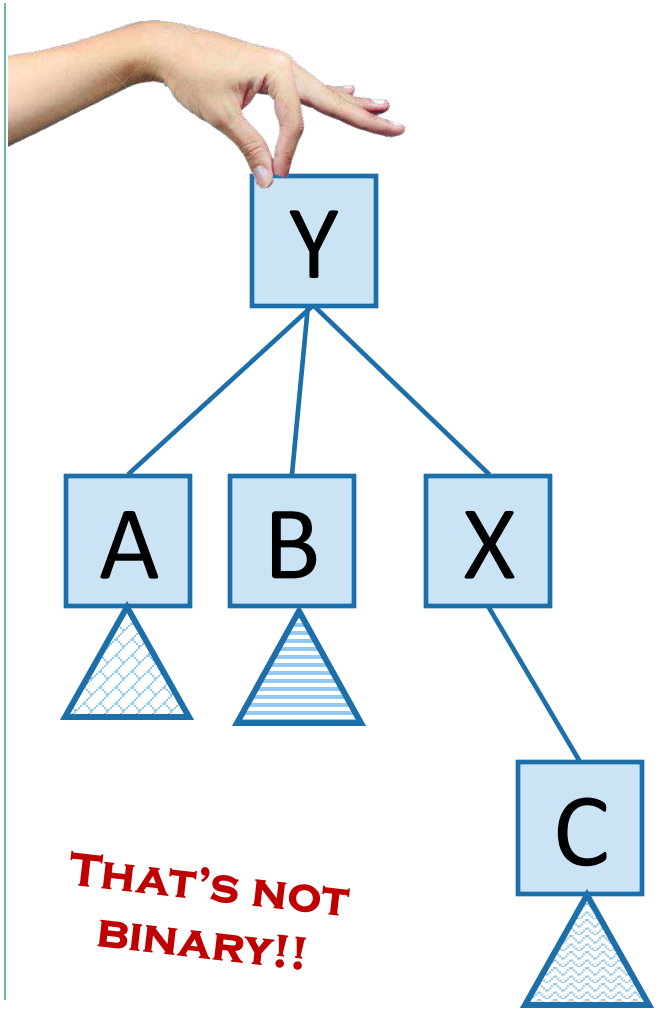
Idea 1: Rotations

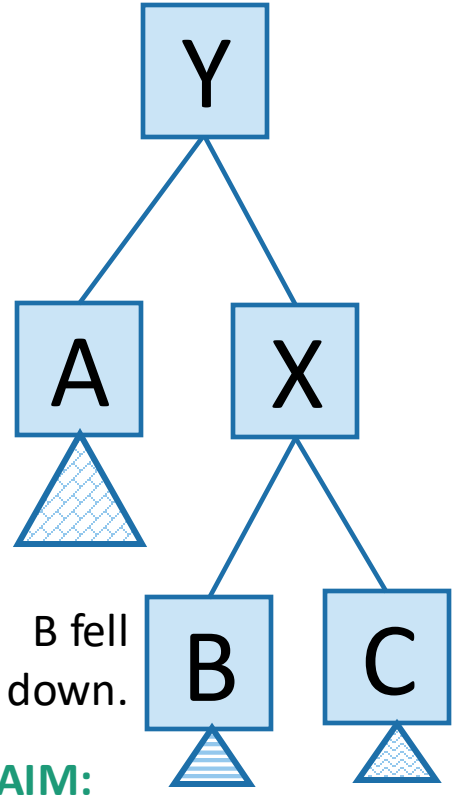
No matter what lives underneath A,B,C, **this takes time O(1).** (Why?)

- Maintain Binary Search Tree (BST) property, while moving stuff around.

Note: A, B, C, X, Y are variable names, not the contents of the nodes.

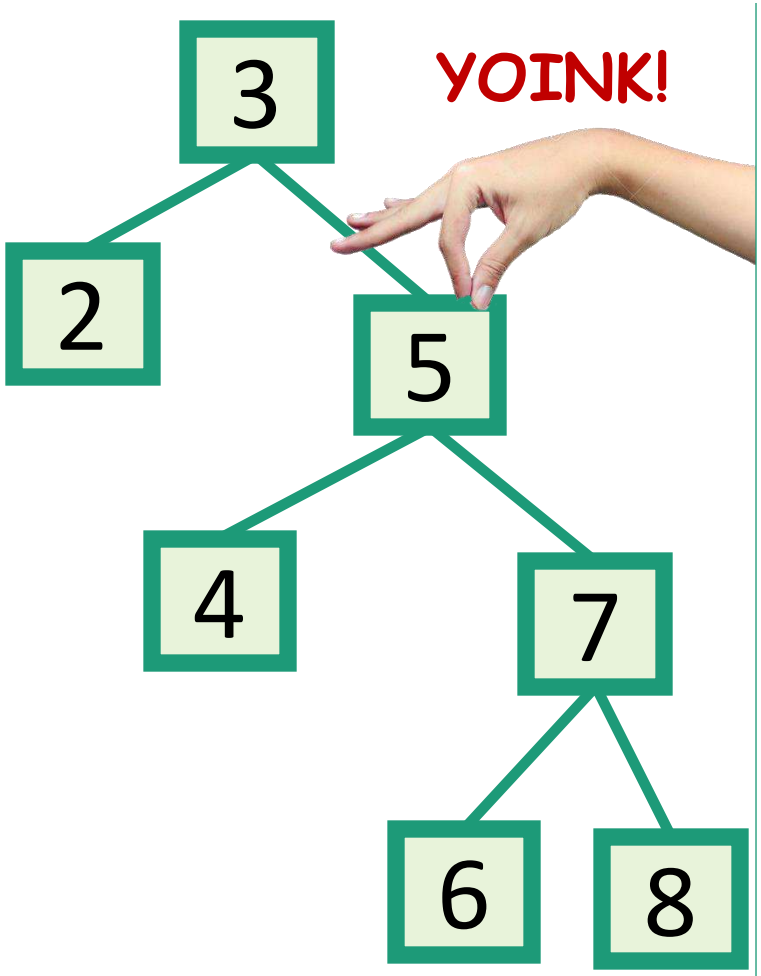


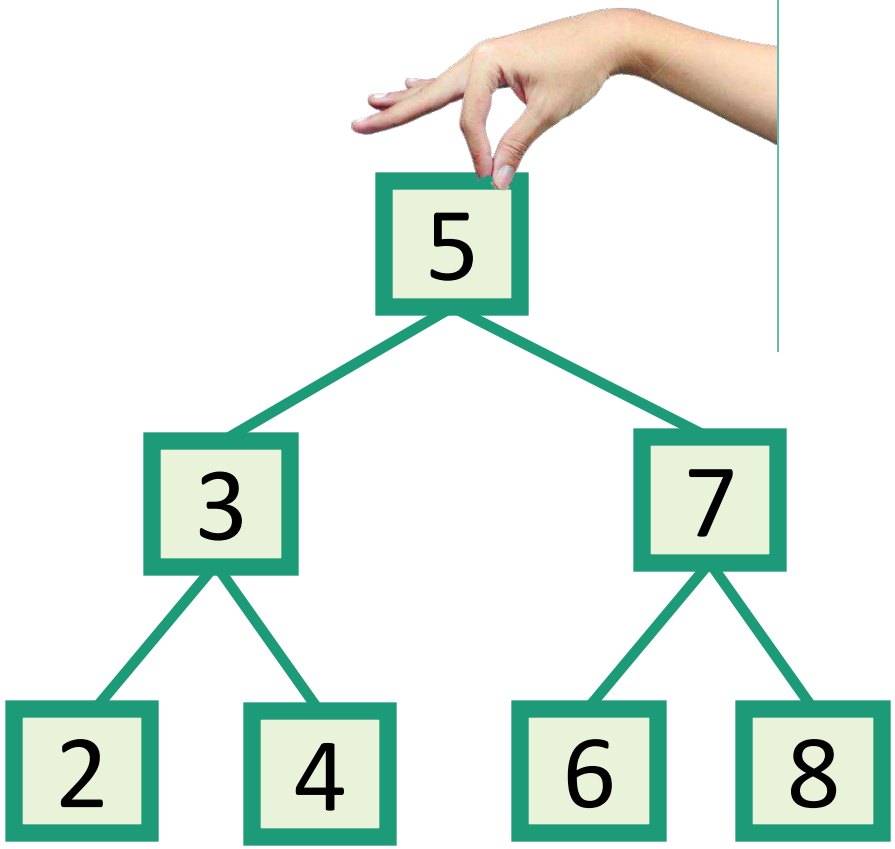




CLAIM: this still has BST property.

This seems helpful





Strategy?

- Whenever something seems unbalanced, do rotations until it's okay again.

Lucky the Lackadaisical Lemur

Even for Lucky this is pretty vague. What do we mean by "seems unbalanced"? What's "okay"?

Idea 2: have some proxy for balance

- Maintaining perfect balance is too hard.
- Instead, come up with some proxy for balance:
  - If the tree satisfies [SOME PROPERTY], then it's pretty balanced.
  - We can maintain [SOME PROPERTY] using rotations.

There are actually several ways to do this, but today we'll see…

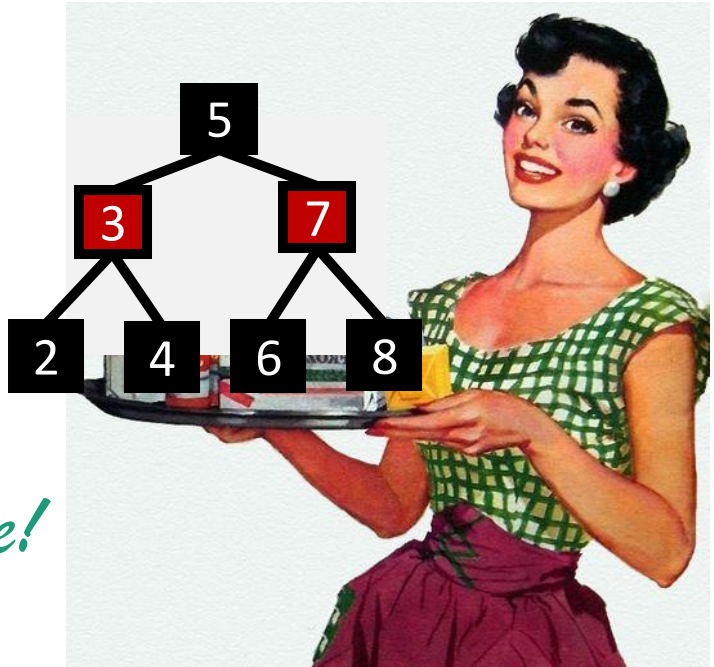
Red-Black Trees

- A Binary Search Tree that balances itself!
- No more time-consuming by-hand balancing!
- Be the envy of your friends and neighbors with the time-saving…

Red-Black tree!

Maintain balance by stipulating that **black nodes** are balanced, and that there *aren't too many* **red nodes**.

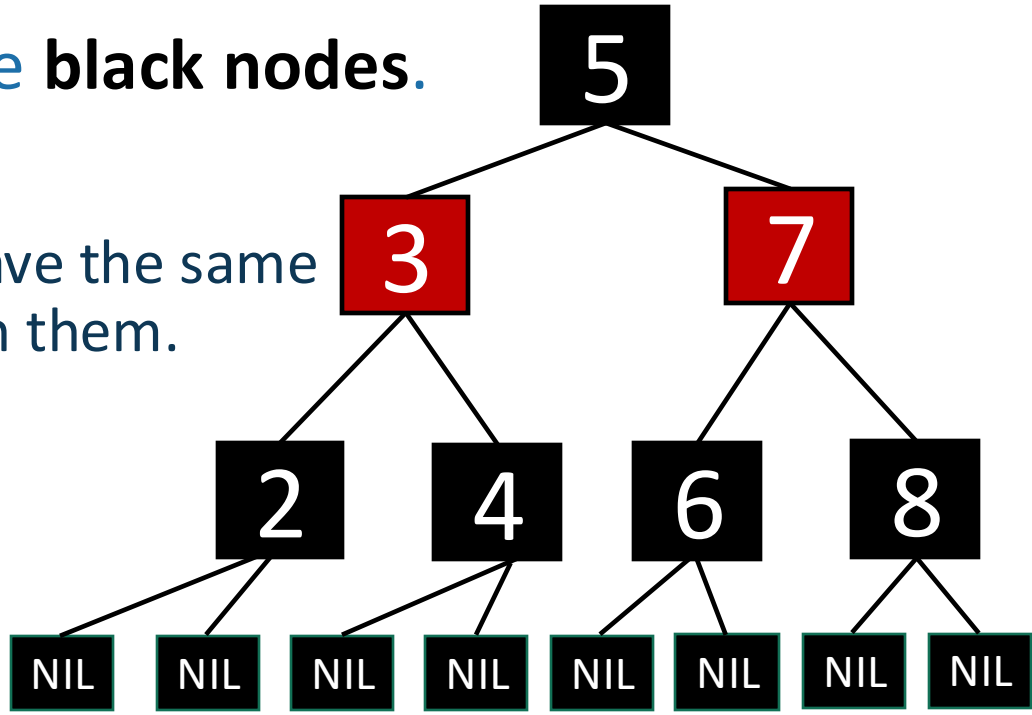
It's just good sense!



Red-Black Trees

obey the following rules (which are a proxy for balance)

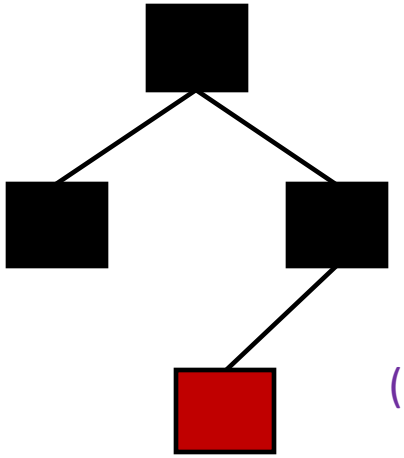
- Every node is colored **red** or **black.**
- The root node is a **black node**.
- NIL children count as **black nodes**.
- Children of a **red node** are **black nodes**.
- For all nodes x:
  - all paths from x to NIL's have the same number of **black nodes** on them.



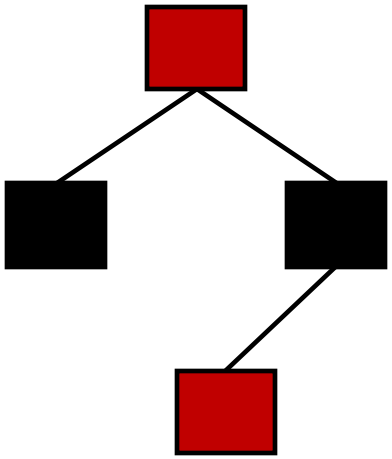
I'm not going to draw the NIL children in the future, but they are treated as black nodes.

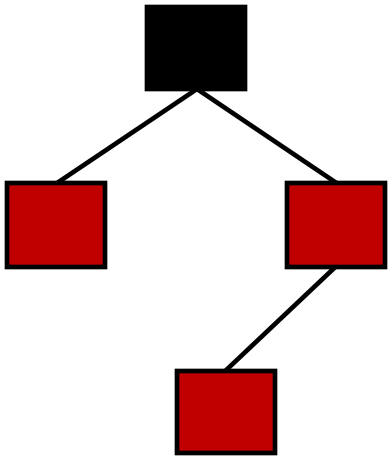
Examples(?)

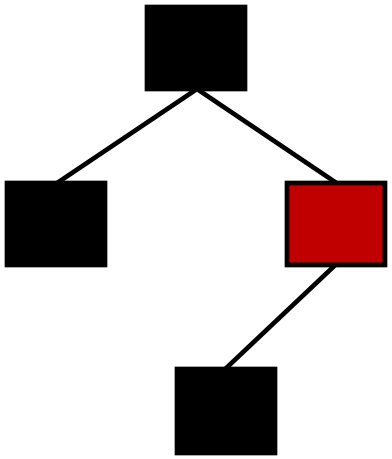
- Every node is colored **red** or **black.**
- The root node is a **black node**.
- NIL children count as **black nodes**.
- Children of a **red node** are **black nodes**.
- For all nodes x:
  - all paths from x to NIL's have the same number of **black nodes** on them.



Which of these
are red-black trees?
(NIL nodes not drawn)

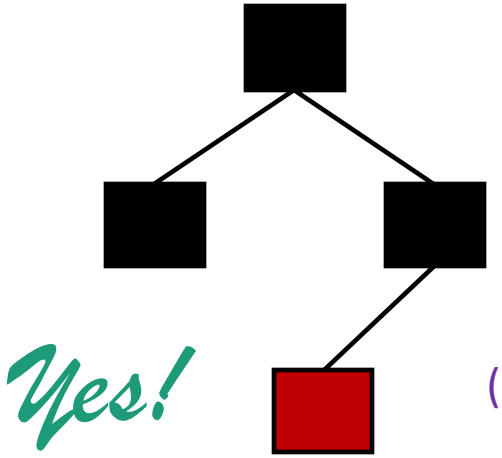






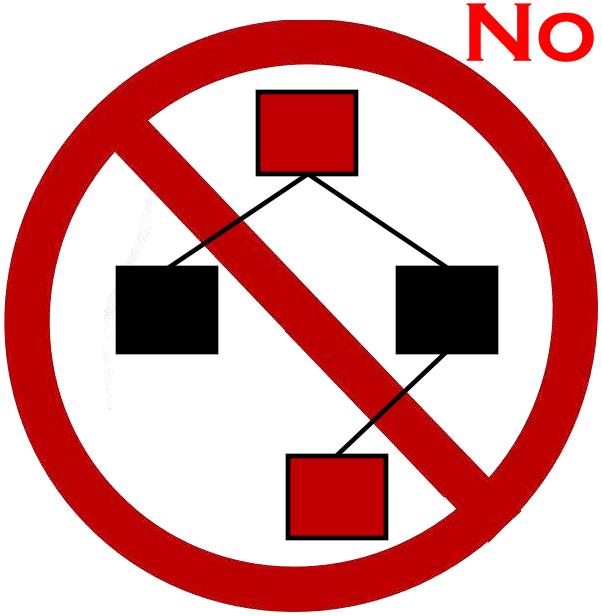
Examples(?)

- Every node is colored **red** or **black.**
- The root node is a **black node**.
- NIL children count as **black nodes**.
- Children of a **red node** are **black nodes**.
- For all nodes x:
  - all paths from x to NIL's have the same number of **black nodes** on them.

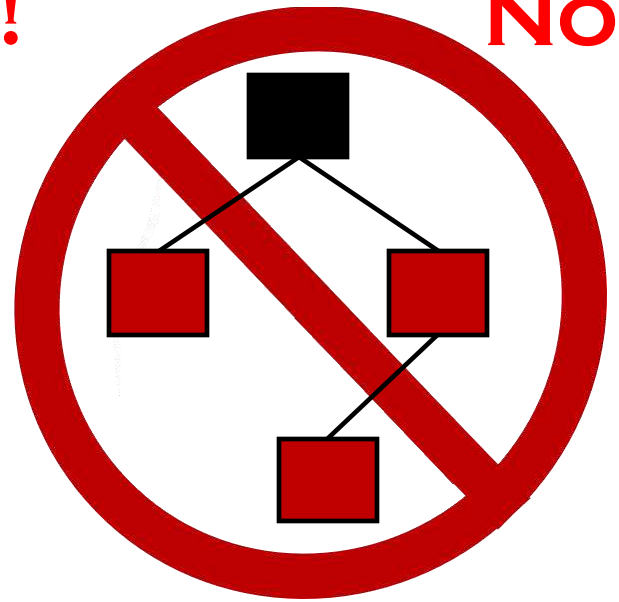


Yes!

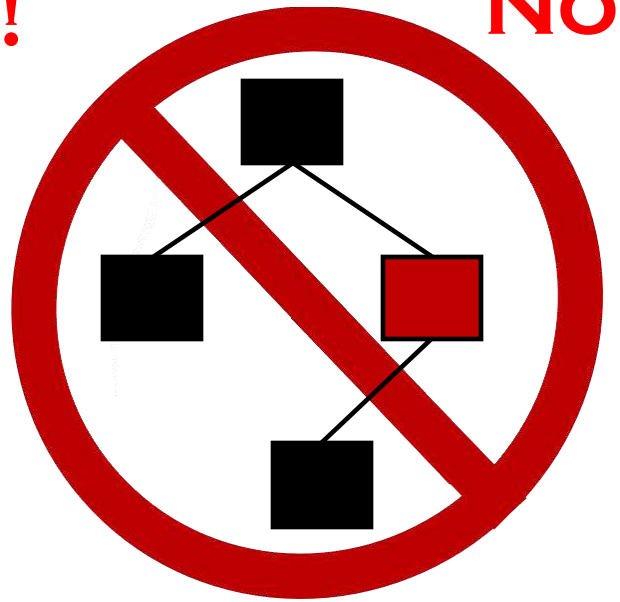
Which of these are red-black trees? (NIL nodes not drawn)



No!



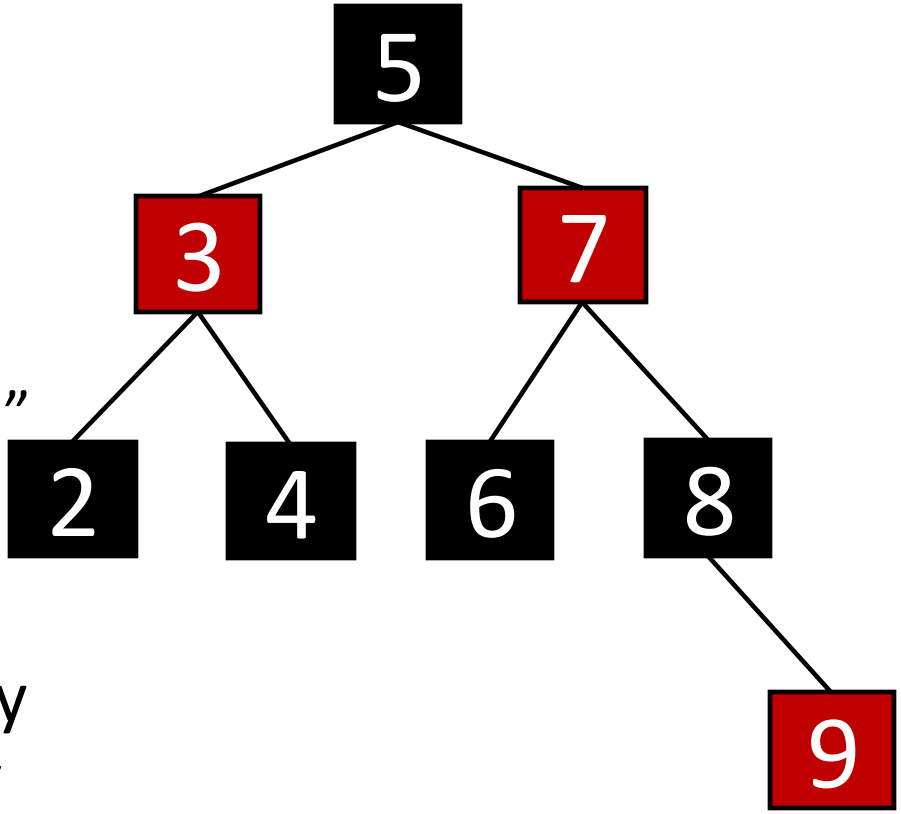
No!



No!

Why **these** rules??????

- Intuition: red-black trees are "pretty balanced"
  - The **black nodes** are balanced
  - The **red nodes** are "spread out" so they don't mess things up too much.



- We can maintain this property as we insert/delete nodes, by using rotations.

This is the really clever idea!

This **Red-Black** structure is a proxy for balance.

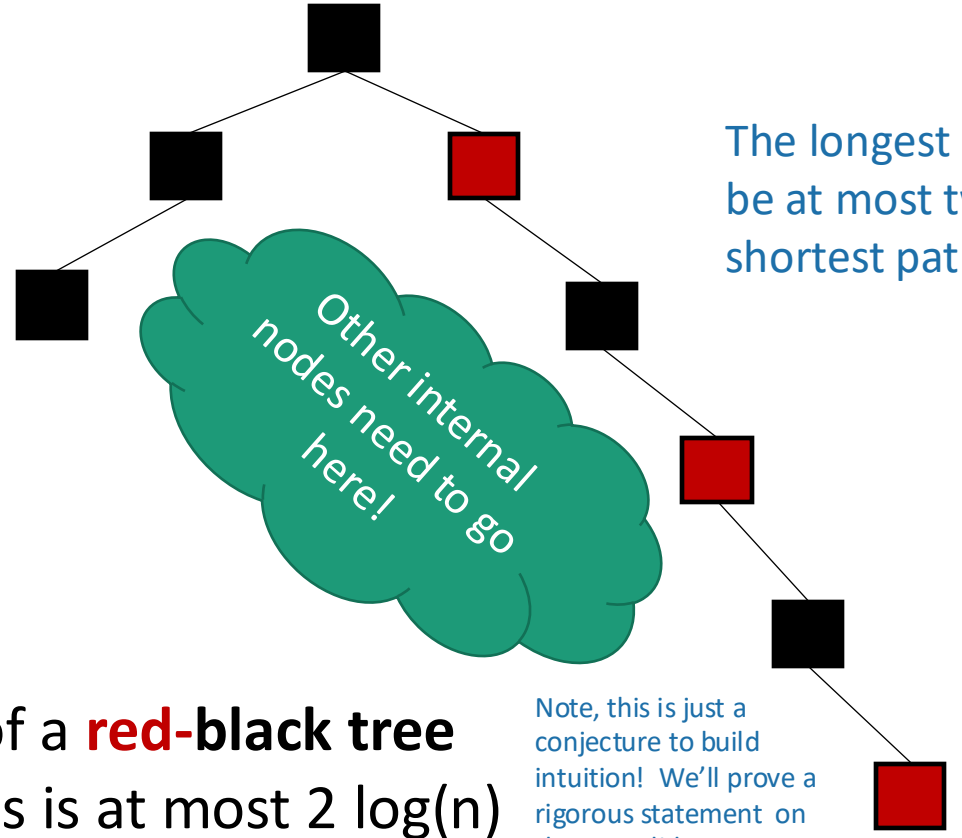
It's just a smidge weaker than perfect balance, but we can actually maintain it!

Let's build some intuition!

Lucky the lackadaisical lemur

This is "pretty balanced"

- To see why, intuitively, let's try to build a Red-Black Tree that's unbalanced.



The longest root-to-NIL path can be at most twice as long as the shortest path.

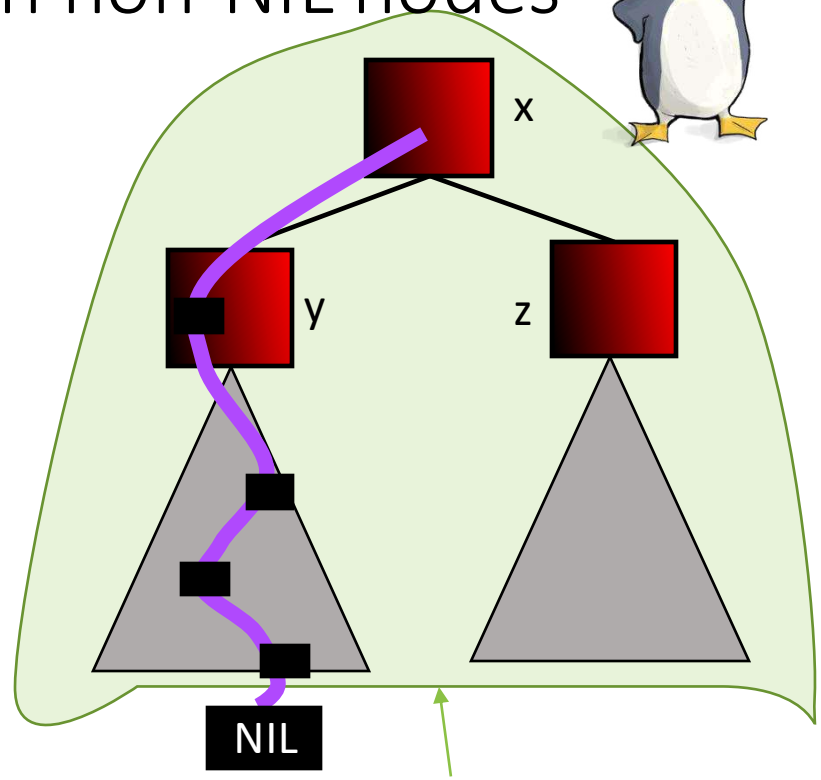
Conjecture:
the height of a **red-black tree** with n nodes is at most 2 log(n)

Note, this is just a conjecture to build intuition! We'll prove a rigorous statement on the next slide.

The height of a RB-tree with n non-NIL nodes is at most $2\log(n+1)$

- Define b(x) to be the number of black nodes in any path from x to NIL.
  - (excluding x, including NIL).
- Claim:
  - There are at least $2^{b(x)} - 1$ non-NIL nodes in the subtree underneath x. (Including x).
- [Proof by induction – sketch on board if time]

(You aren't responsible for this proof in particular…but it's good to see more examples of pf by induction!)



Claim: at least $2^{b(x)} - 1$ nodes in this WHOLE subtree (of any color).

Then:

$$n \geq 2^{b(\text{root})} - 1$$ using the Claim

$$\geq 2^{\text{height}/2} - 1$$ $b(root) \geq \frac{\text{height}}{2}$ because of RBTree rules.

Rearranging:

$$n + 1 \geq 2^{\text{height}/2} \Rightarrow \text{height} \leq 2\log(n+1)$$

This is great!

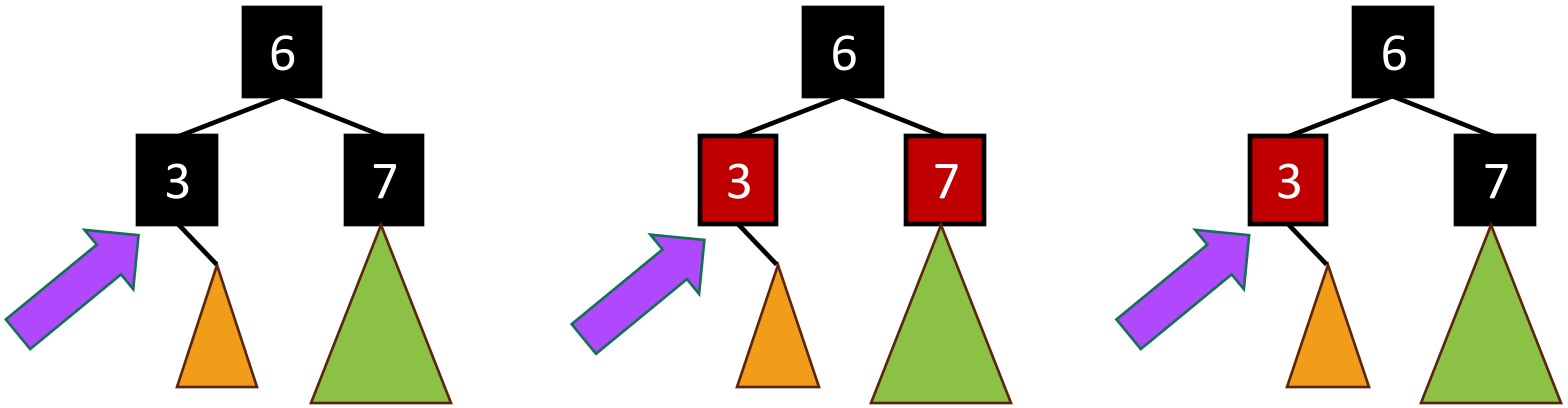
- SEARCH in an RBTree is immediately O(log(n)), since the depth of an RBTree is O(log(n)).

- What about INSERT/DELETE?
  - Turns out, you can INSERT and DELETE items from an RBTree in time O(log(n)), while *maintaining* the RBTree property.
  - That's why this is a good property!

INSERT/DELETE

- I expect we are out of time…
  - There are some slides which you can check out to see how to do INSERT/DELETE in RBTrees if you are curious.
  - See CLRS Ch 13. for even more details.
- You are **not responsible** for the details of INSERT/DELETE for RBTrees for this class.
  - You should know what the “proxy for balance” property is and why it ensures approximate balance.
  - You should know **that** this property can be efficiently maintained, but you do not need to know the details of how.

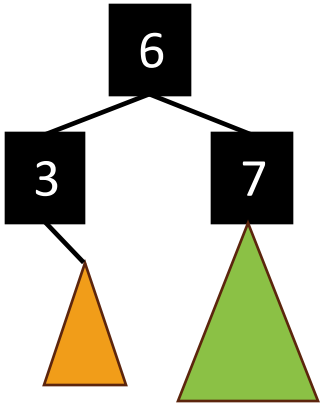
INSERT: Many cases



- Suppose we want to insert 0 **here**.
- There are 3 “important” cases for different colorings of the existing tree, and there are 9 more cases for all of the various symmetries of these 3 cases.

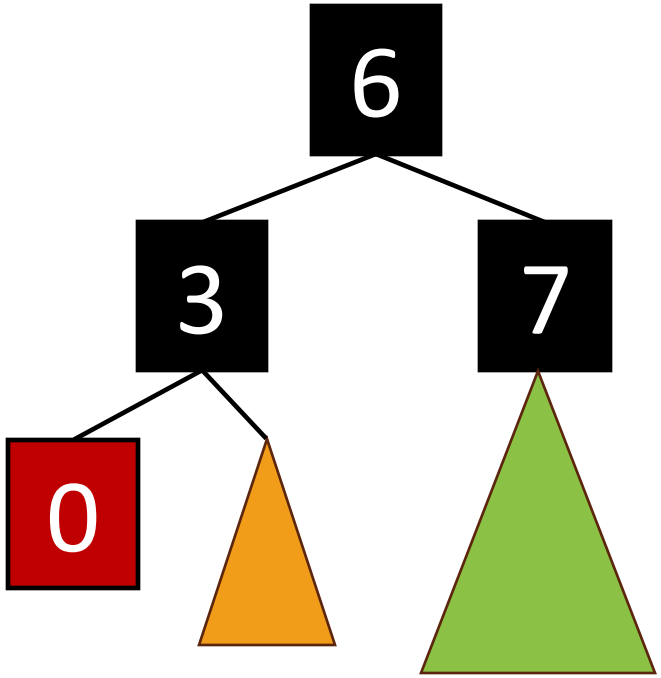
INSERT: Case 1

- Make a new **red node**.
- Insert it as you would normally.

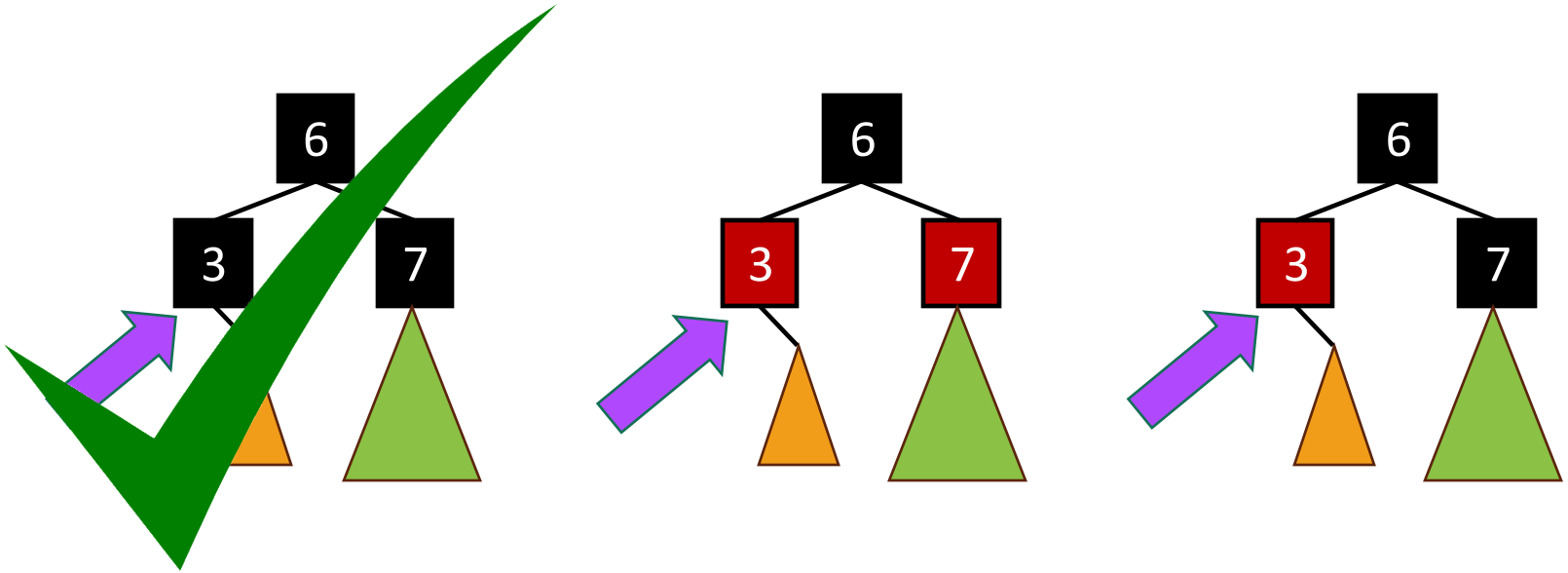


What if it looks like this?

Example: insert 0



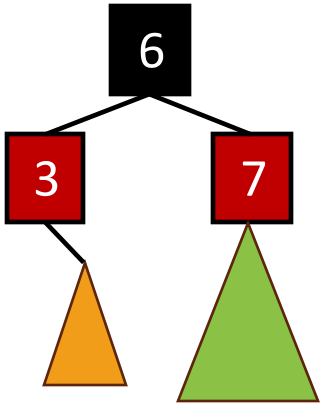
INSERT: Many cases



- Suppose we want to insert 0 **here**.
- There are 3 “important” cases for different colorings of the existing tree, and there are 9 more cases for all of the various symmetries of these 3 cases.

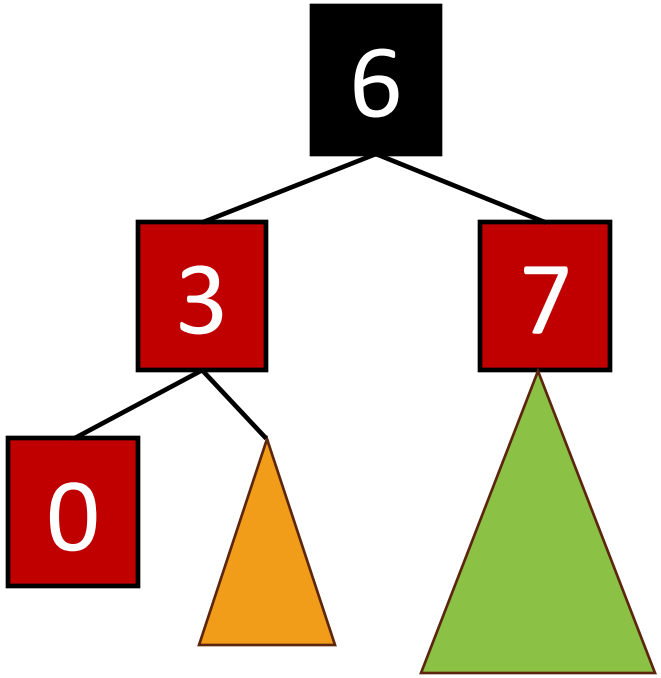
INSERT: Case 2

- Make a new **red node**.
- Insert it as you would normally.
- Fix things up if needed.



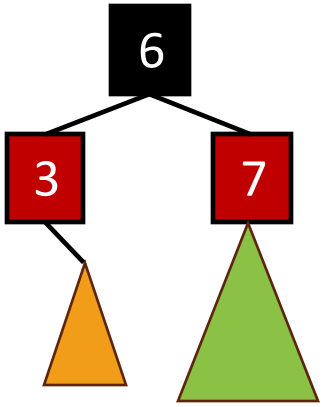
What if it looks like this?

Example: insert 0

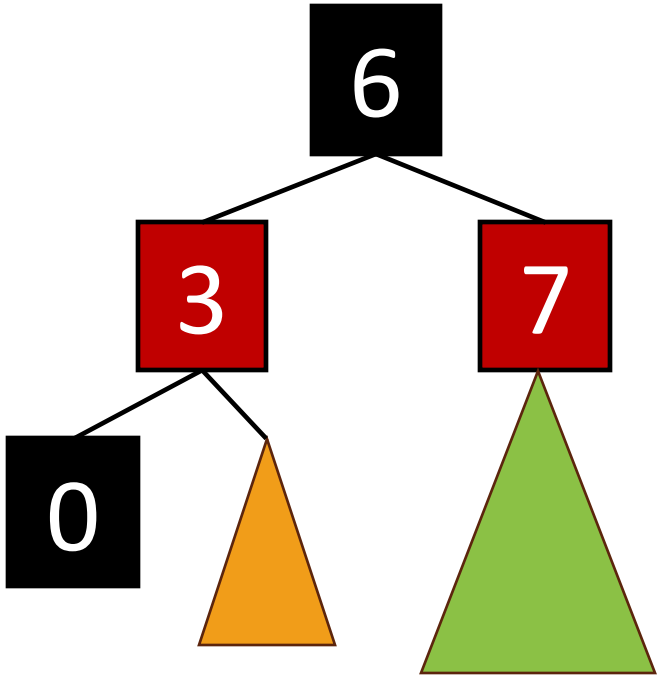


INSERT: Case 2

- Make a new **red node**.
- Insert it as you would normally.
- Fix things up if needed.

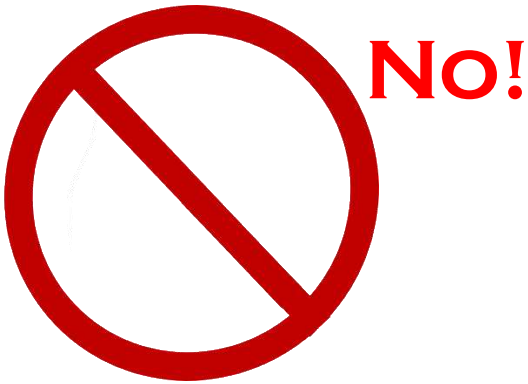


What if it looks like this?

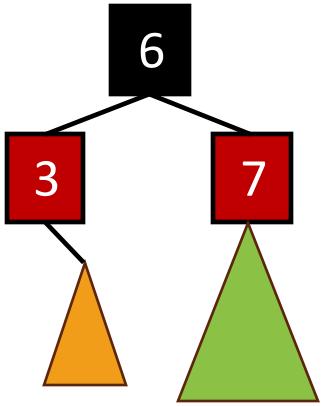


Example: insert 0

Can't we just insert 0 as a **black node?**

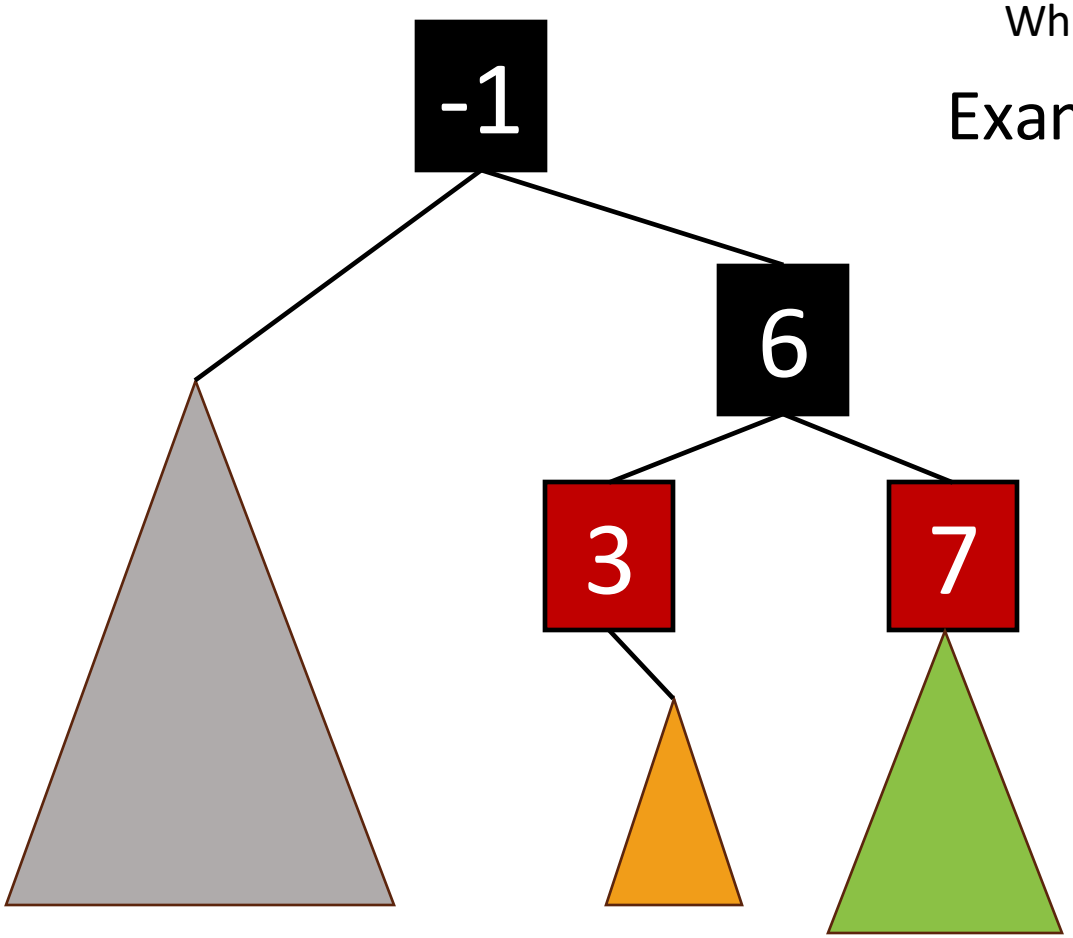


We need a bit more context



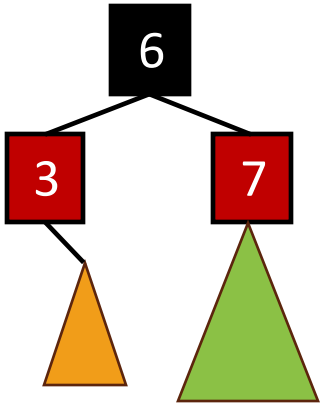
What if it looks like this?

Example: insert 0



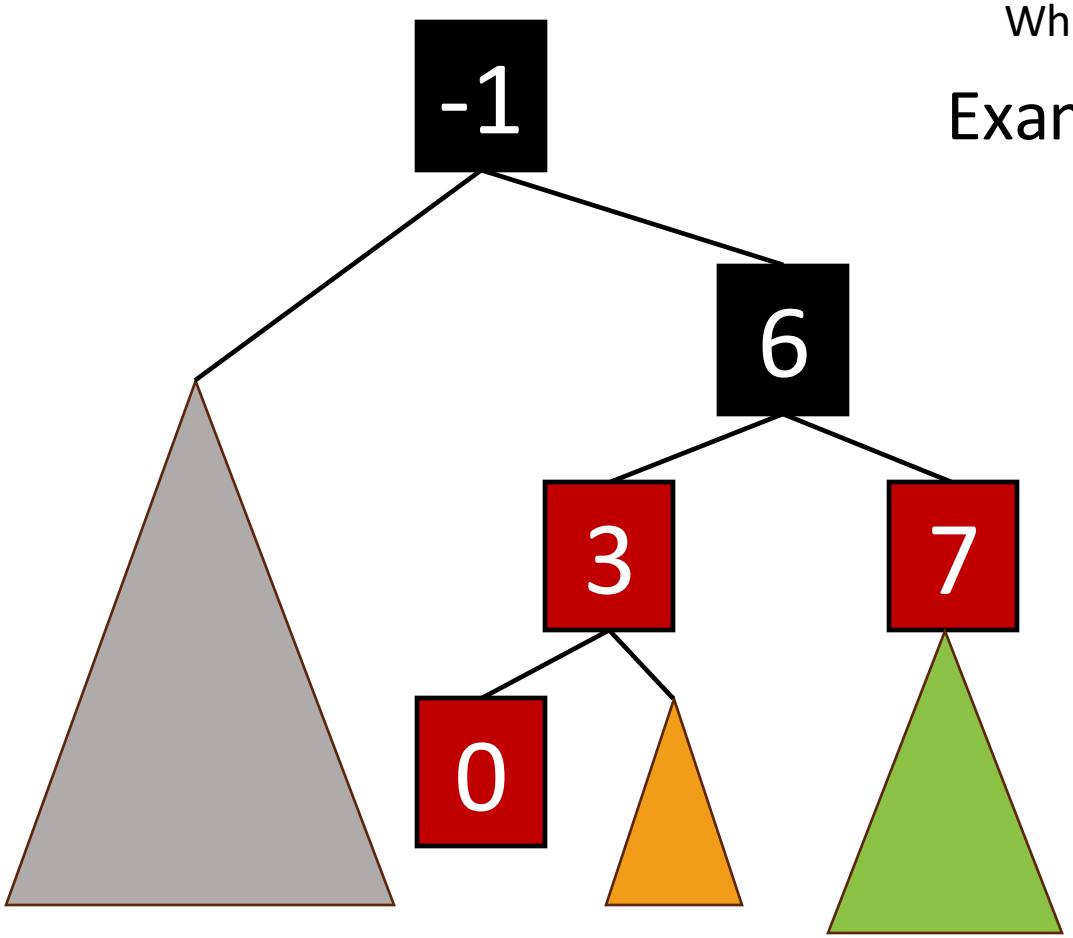
We need a bit more context

- Add 0 as a red node.



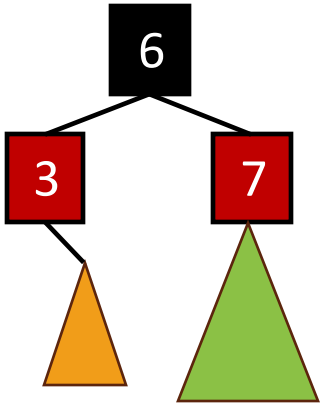
What if it looks like this?

Example: insert 0



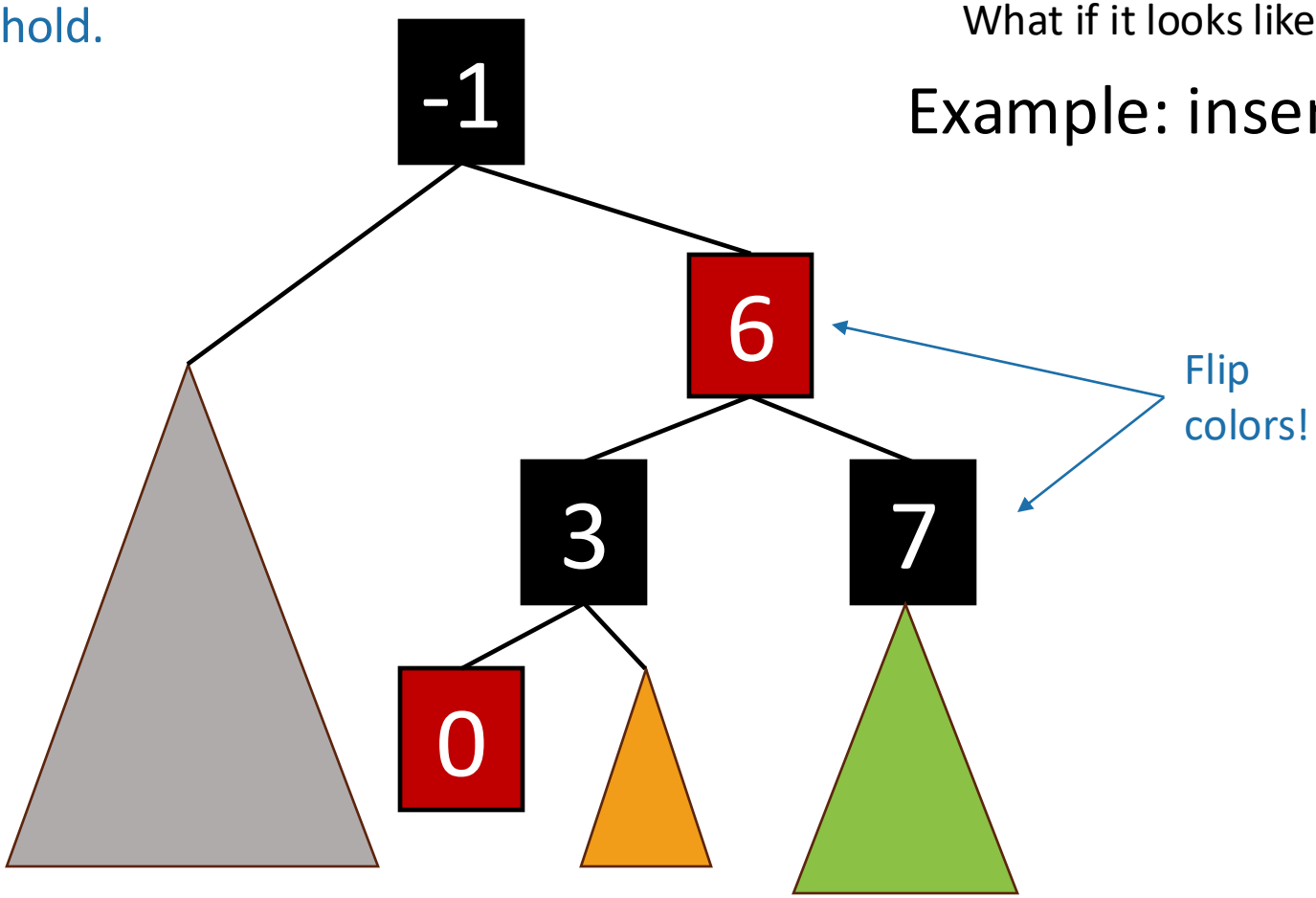
We need a bit more context

- Add 0 as a red node.
- **Claim:** RB-Tree properties still hold.

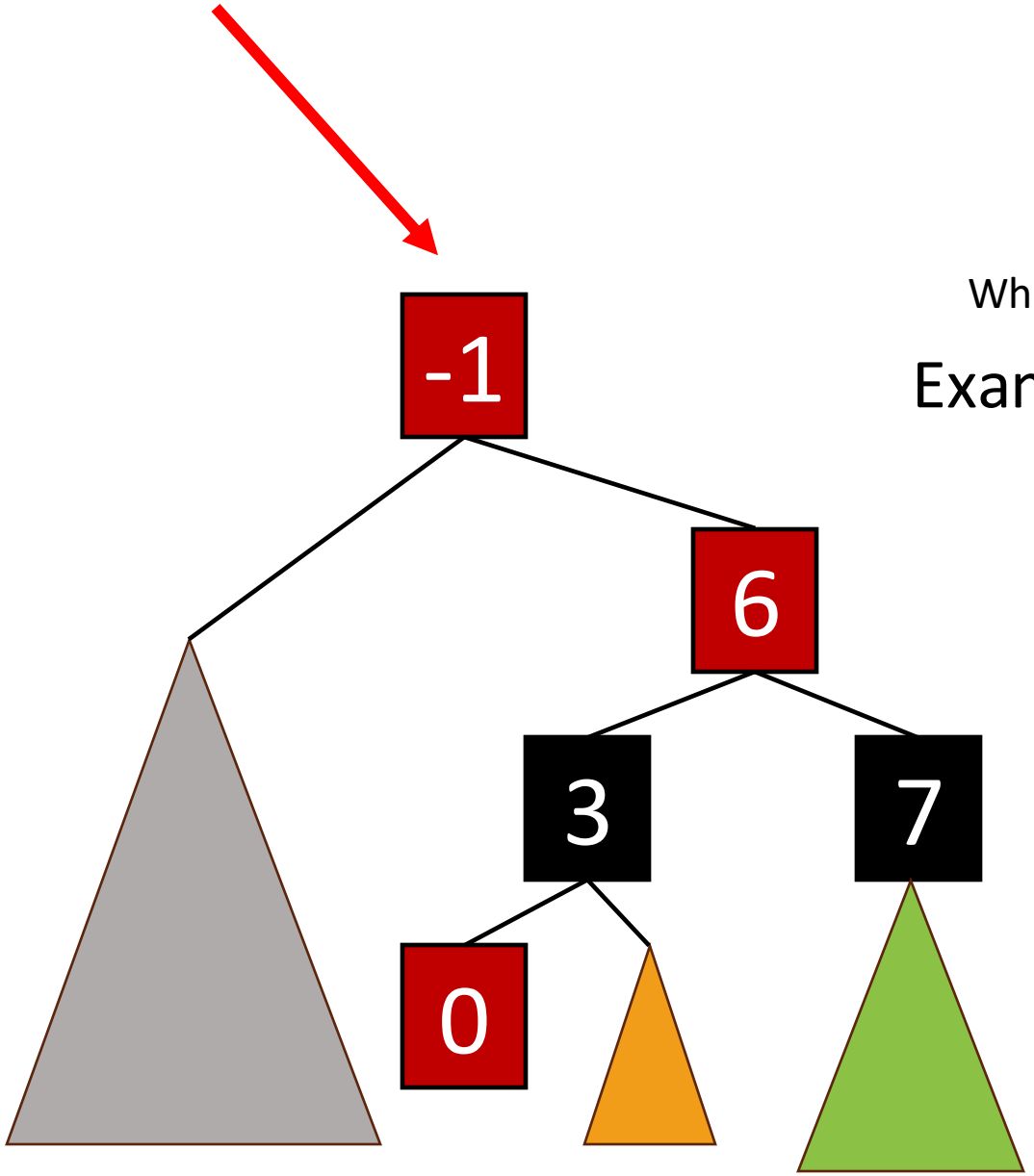


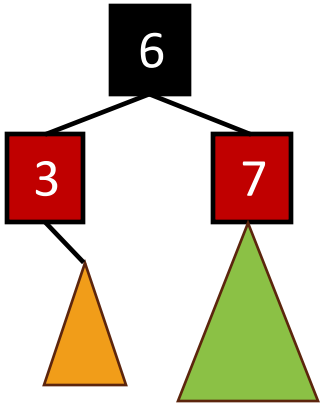
What if it looks like this?

Example: insert 0



But what if that was red?

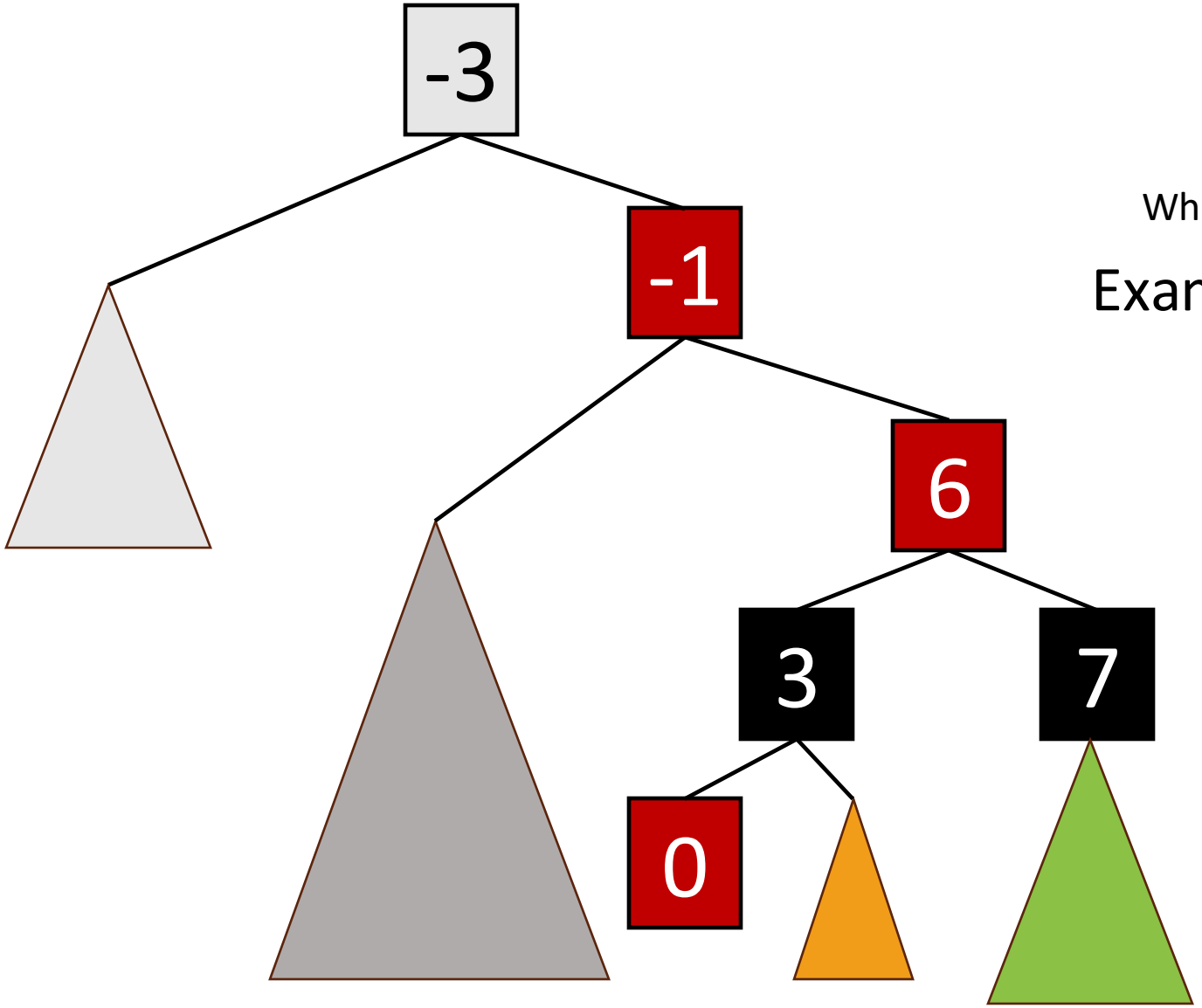


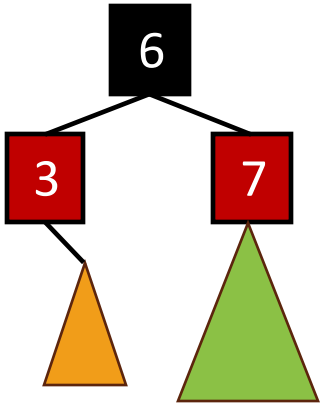


What if it looks like this?

Example: insert 0

More context…

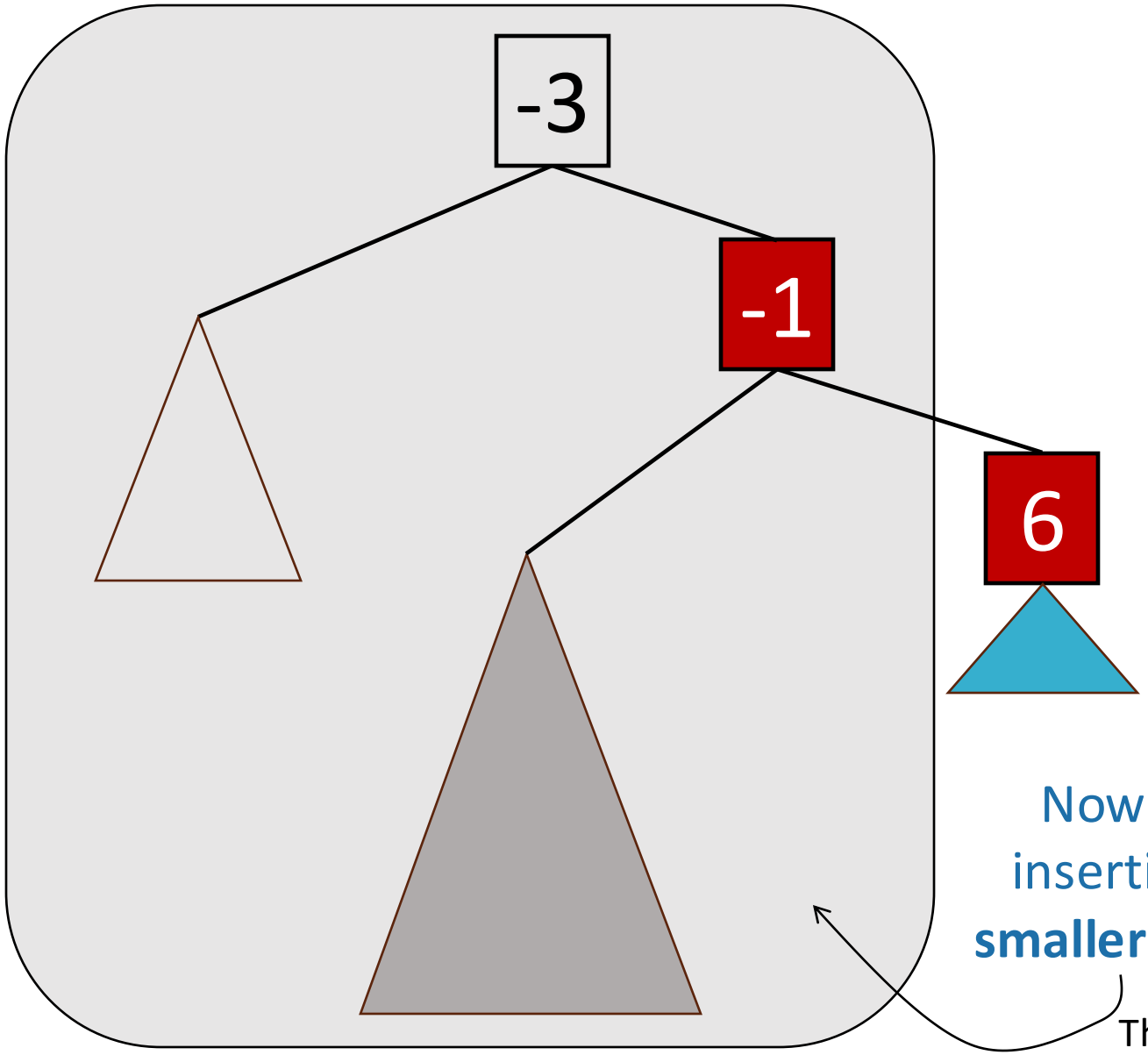


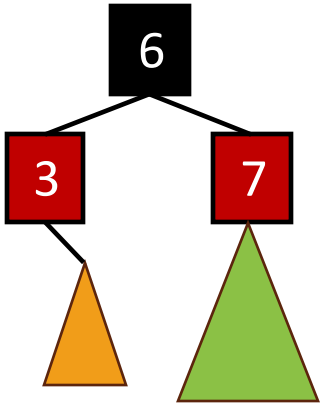


What if it looks like this?

Example: insert 0

More context…





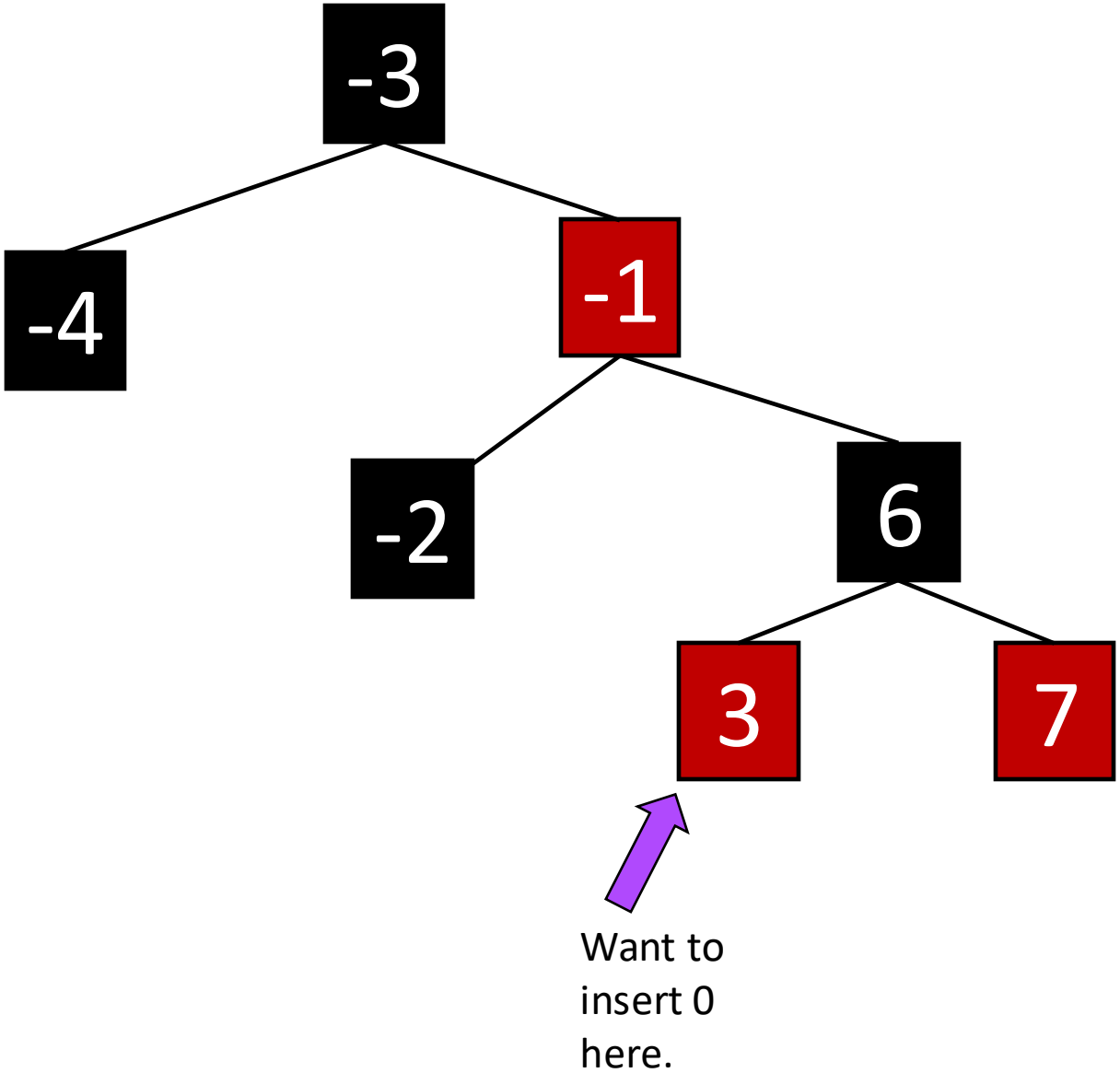
What if it looks like this?

Example: insert 0

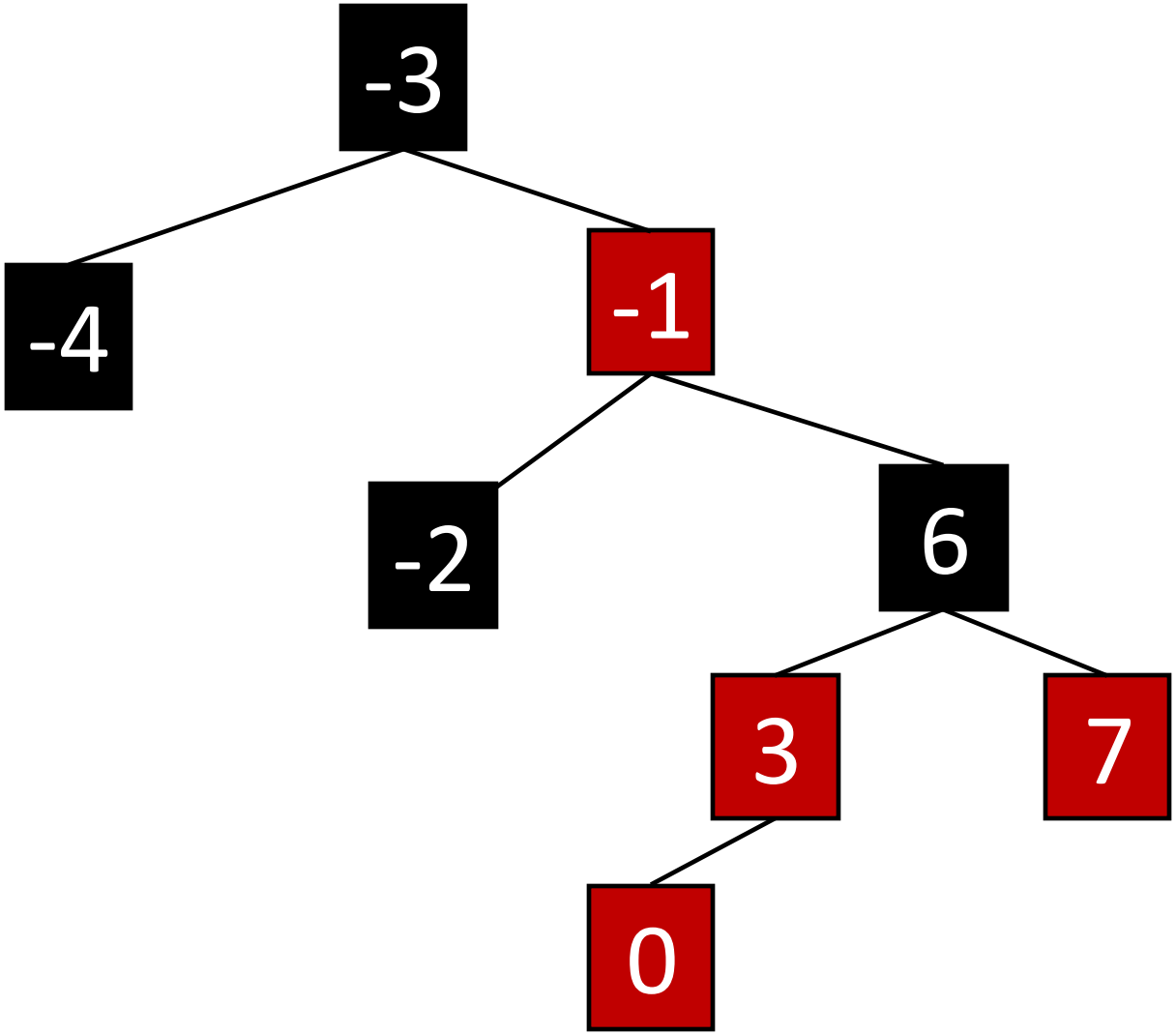
Now we're basically inserting 6 into some **smaller tree**. Recurse!

This one!

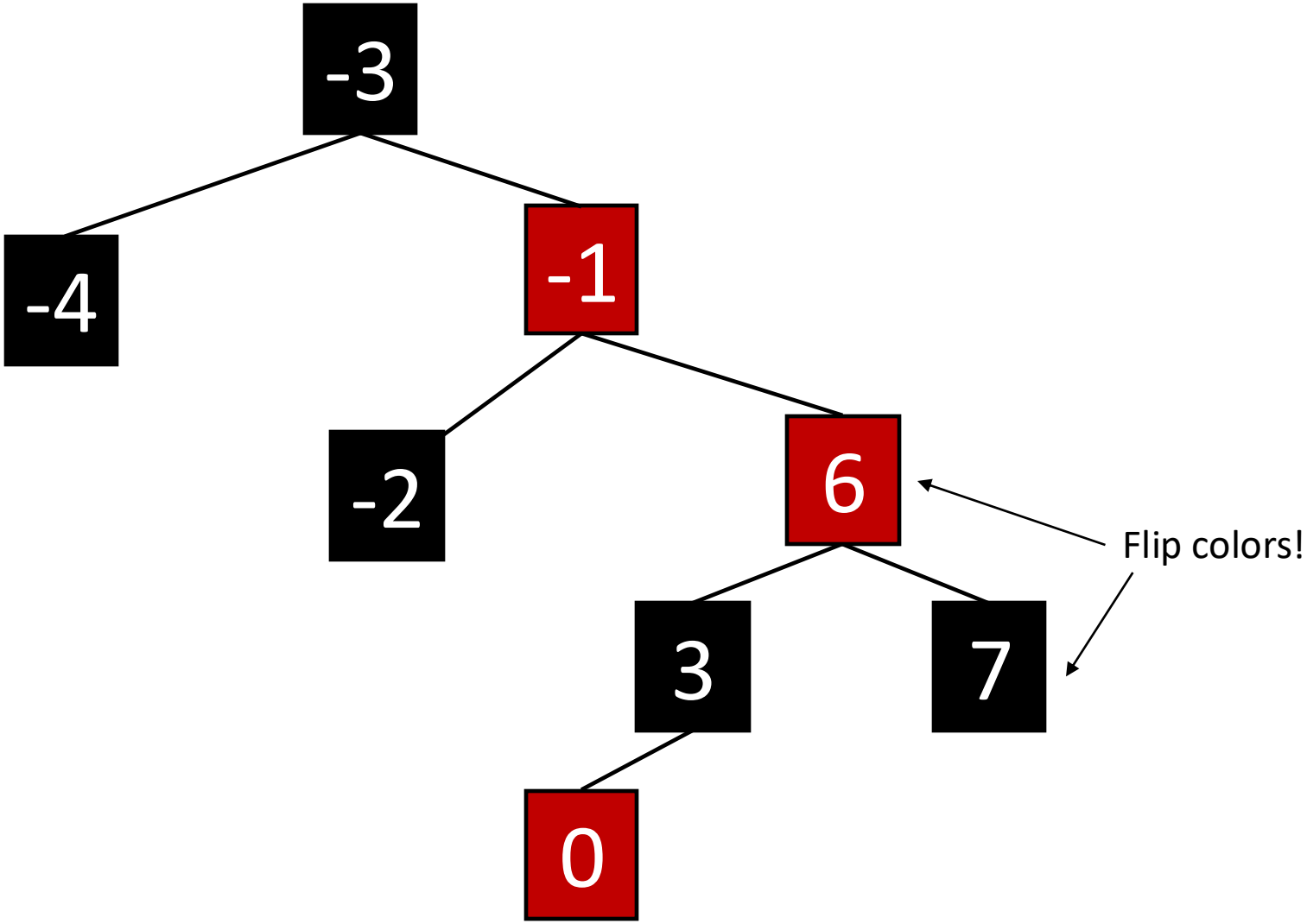
Example, part I



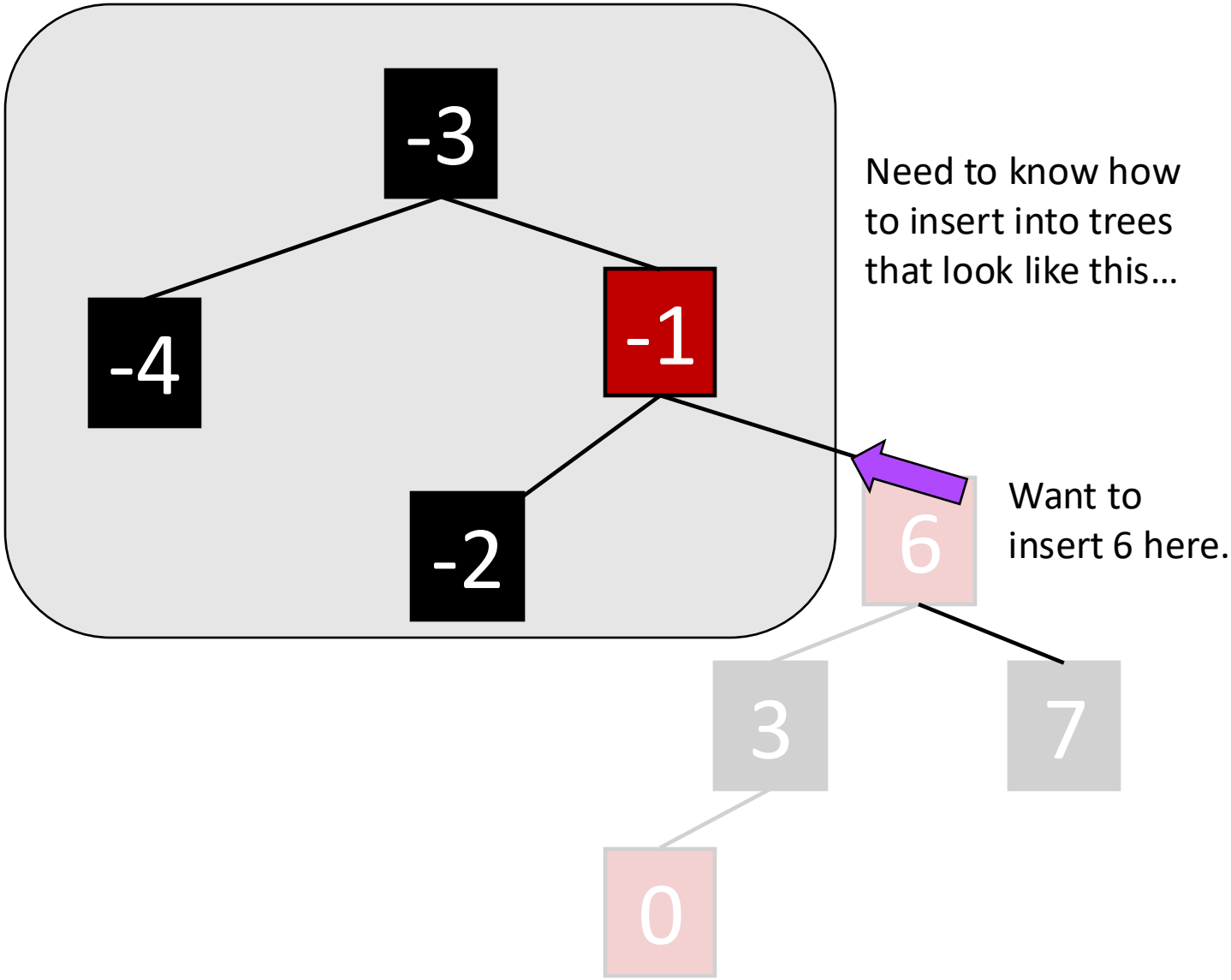
Example, part I



Example, part I

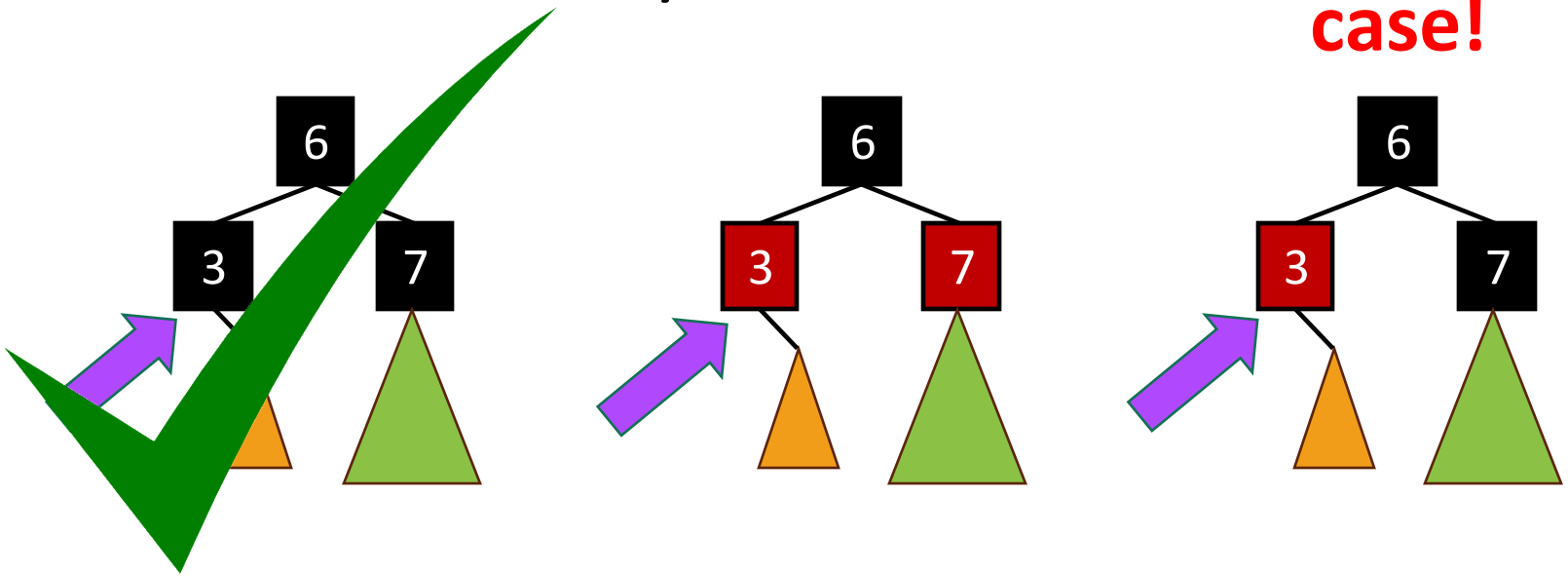


Example, part I



INSERT: Many cases

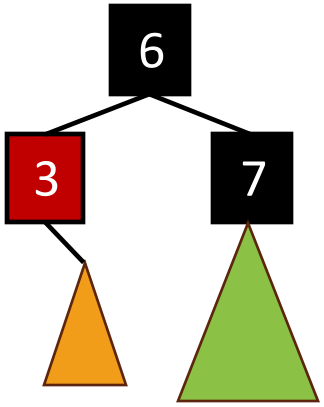
That's this case!



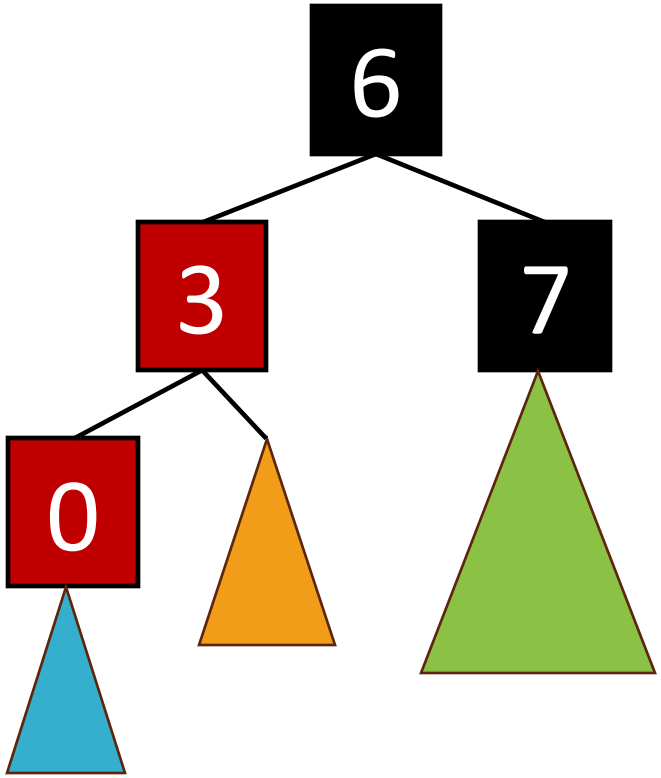
- Suppose we want to insert 0 **here**.
- There are 3 "important" cases for different colorings of the existing tree, and there are 9 more cases for all of the various symmetries of these 3 cases.

INSERT: Case 3

- Make a new **red node**.
- Insert it as you would normally.
- Fix things up if needed.



What if it looks like this?

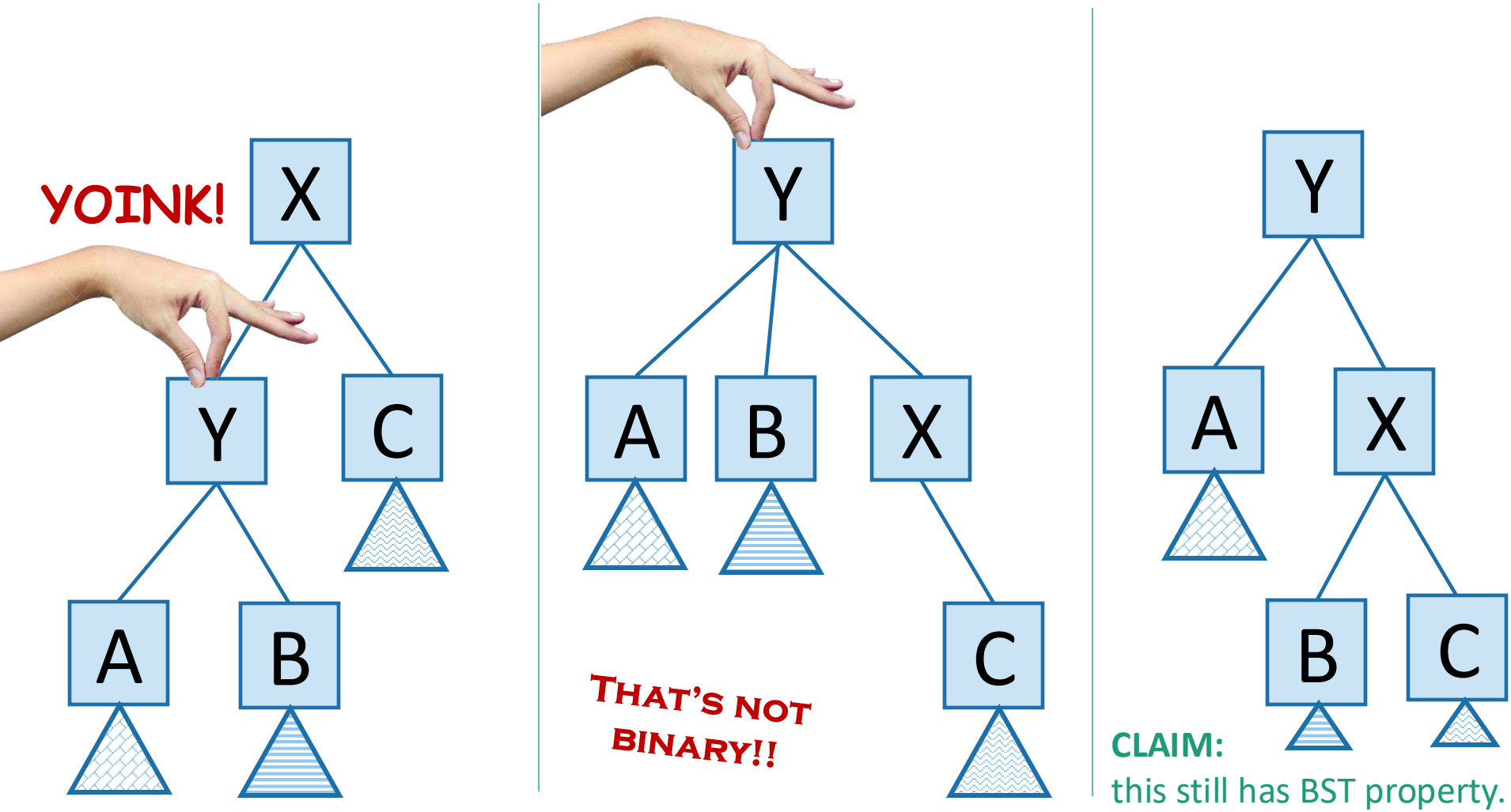


Example: Insert 0.

- Maybe with a subtree below it.

Recall Rotations

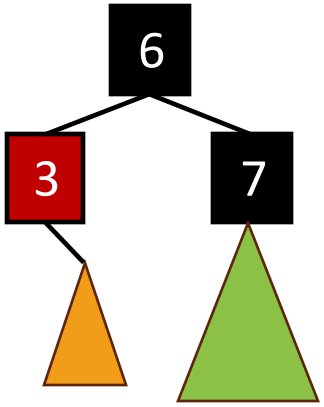
- Maintain Binary Search Tree (BST) property, while moving stuff around.



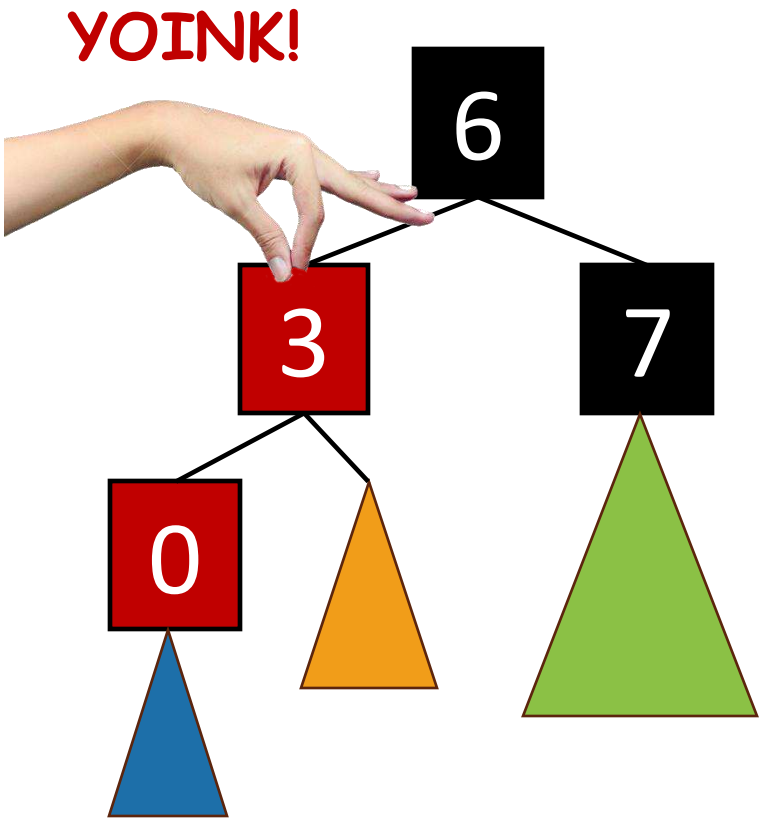
CLAIM:
this still has BST property.

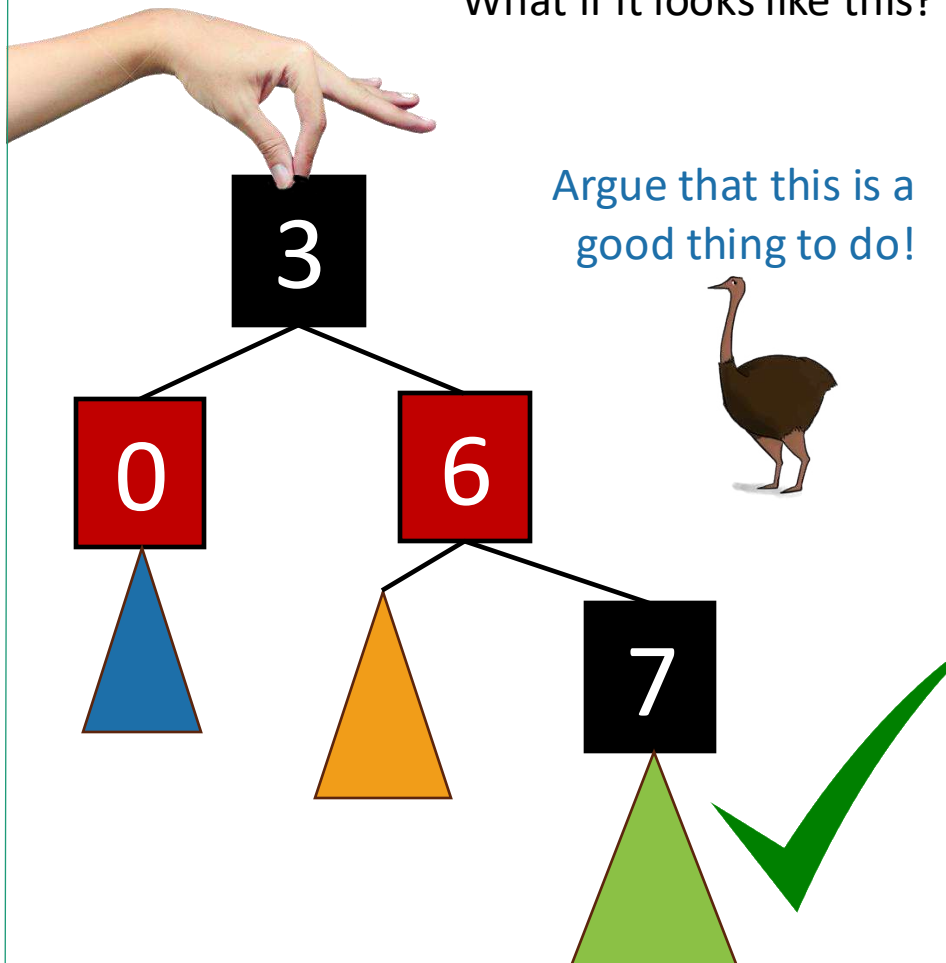
Inserting into a Red-Black Tree

- Make a new **red node**.
- Insert it as you would normally.
- Fix things up if needed.



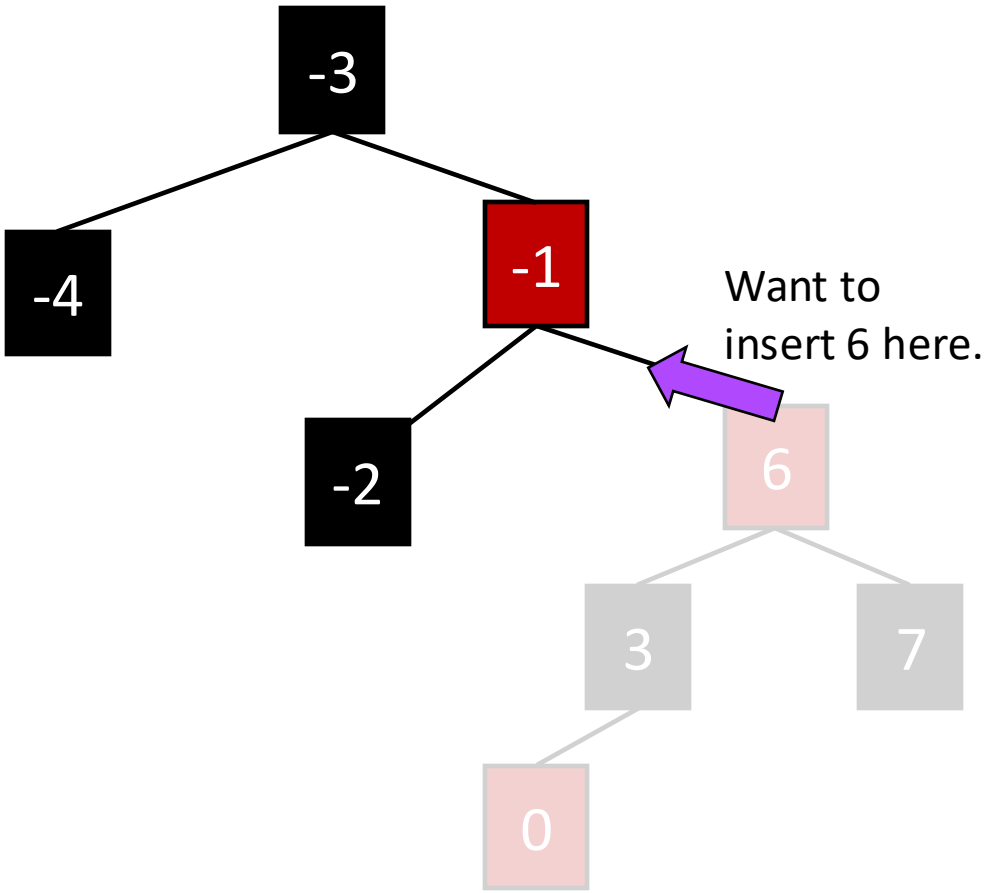
What if it looks like this?



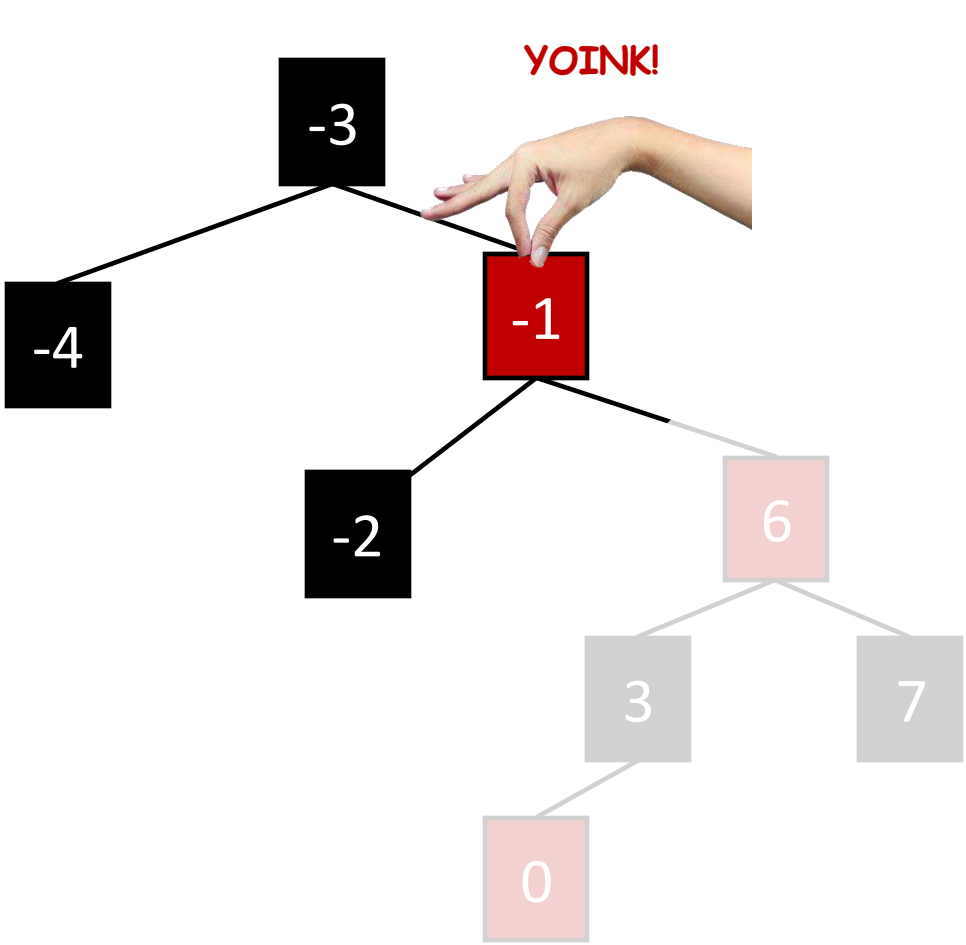


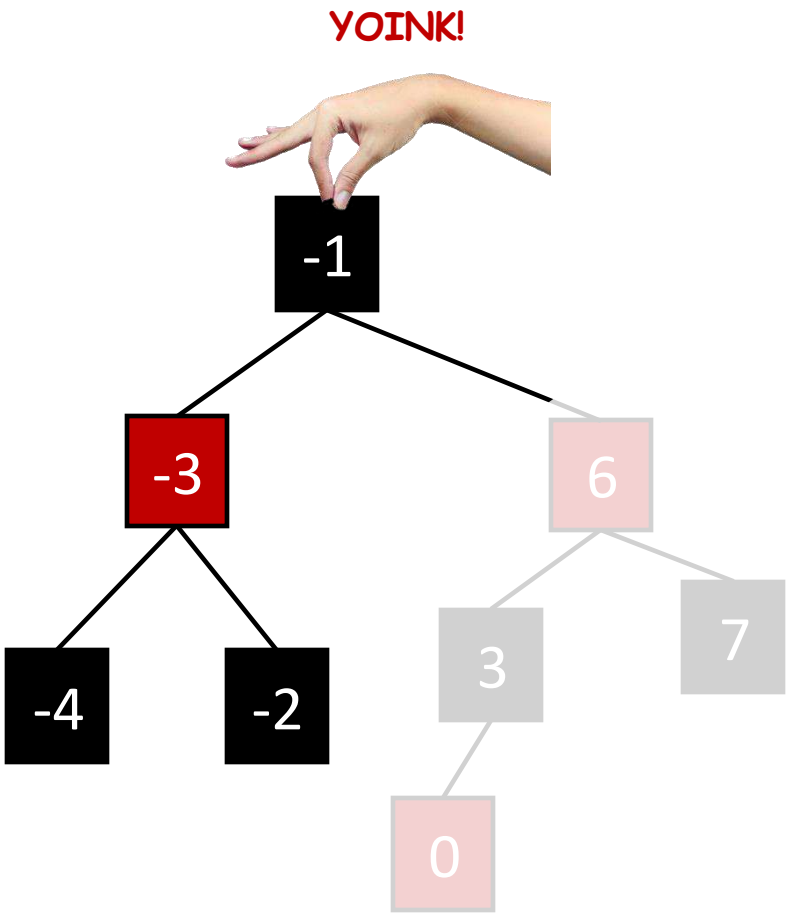
Argue that this is a good thing to do!

Example, part 2

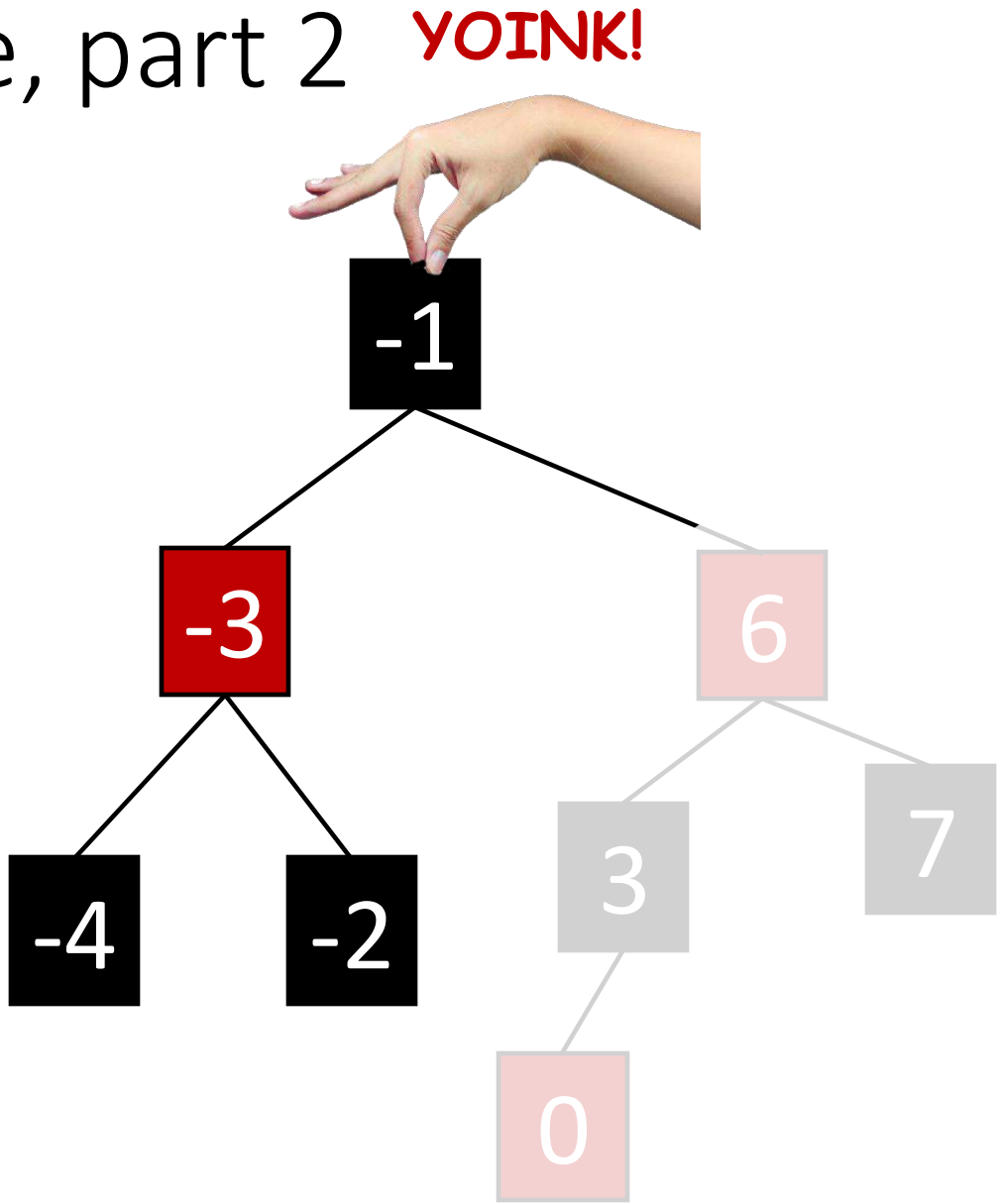


Example, part 2

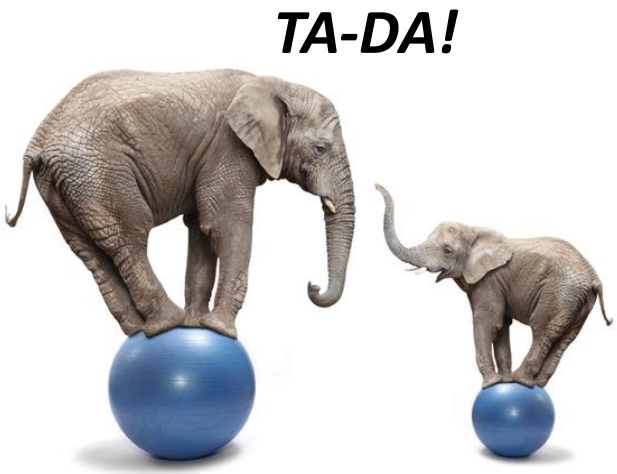


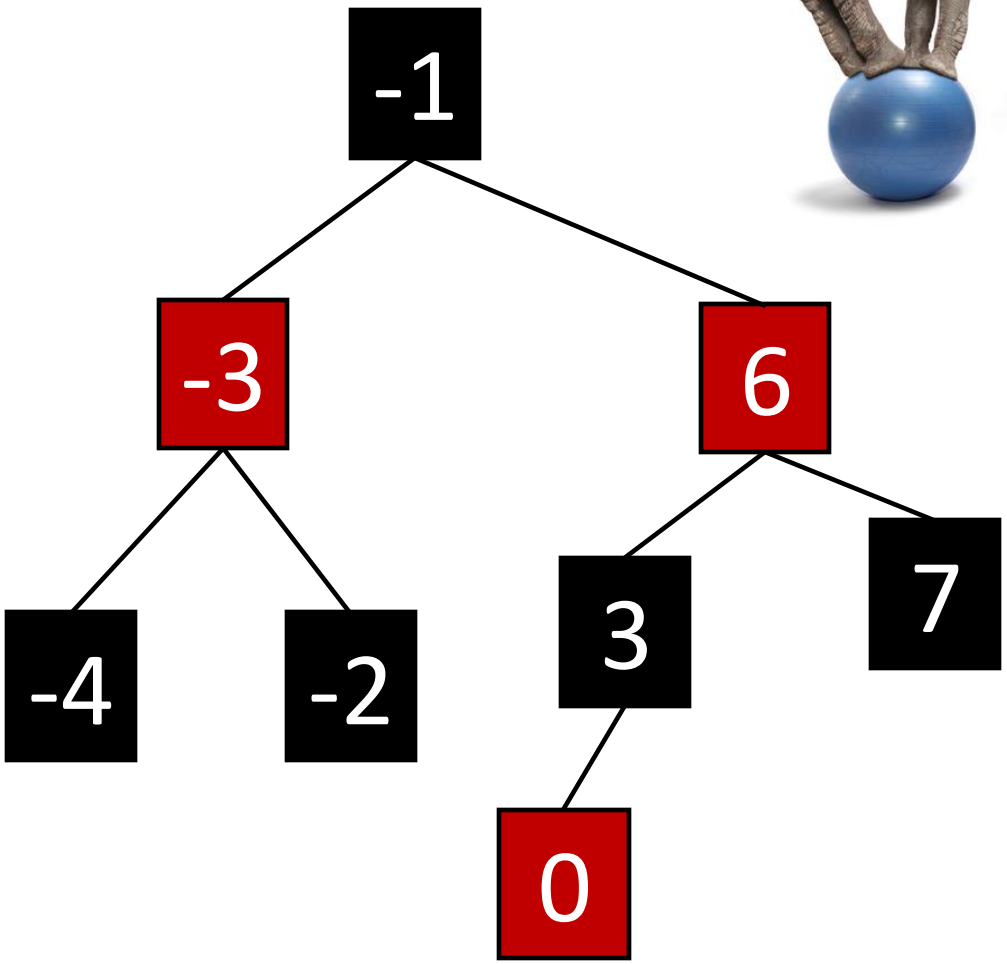


Example, part 2

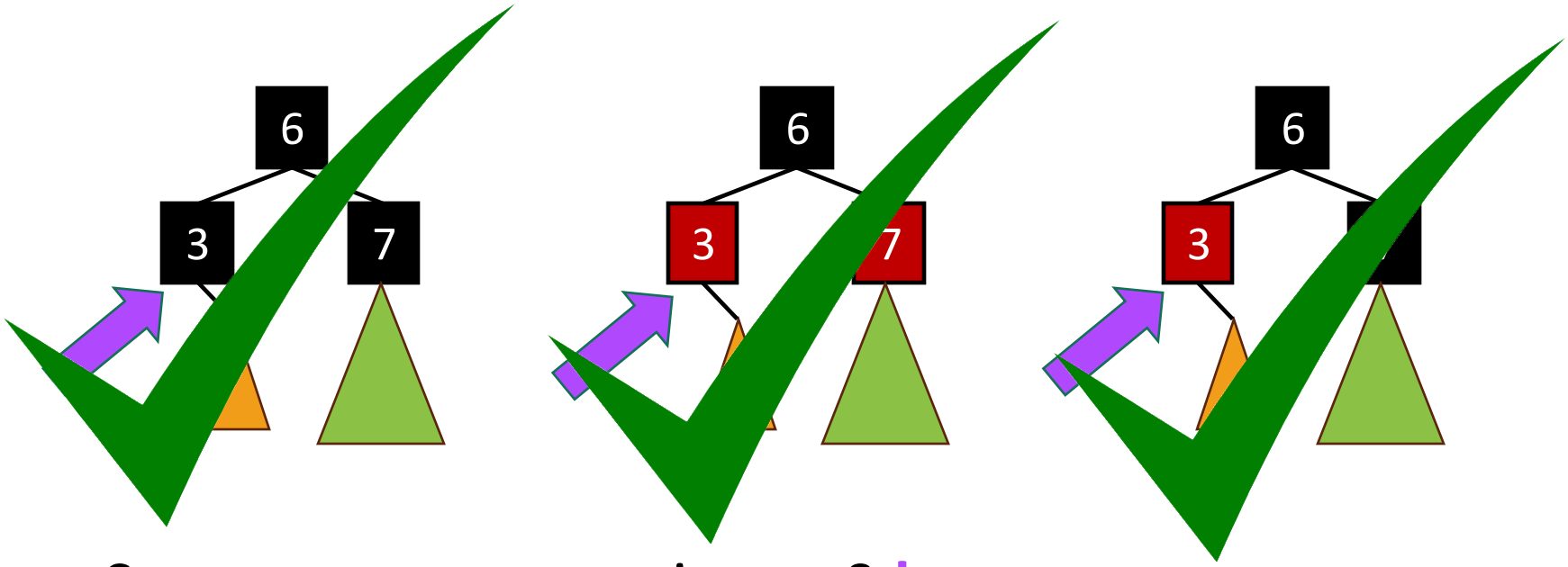


Example, part 2





Many cases

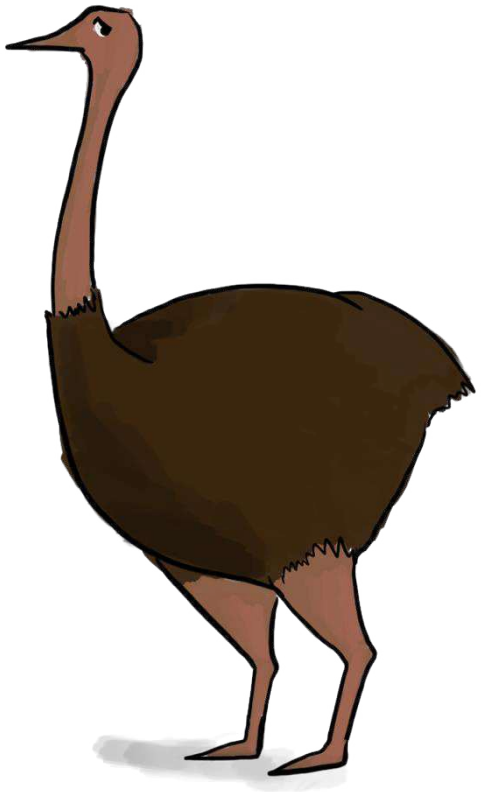


- Suppose we want to insert 0 **here**.

- There are 3 “important” cases for different colorings of the existing tree, and there are 9 more cases for all of the various symmetries of these 3 cases.

Deleting from a Red-Black tree

Fun exercise!



Ollie the over-achieving ostrich

That's a lot of cases!

- You are **not responsible** for the nitty-gritty details of Red-Black Trees. (For this class)
  - Though implementing them is a great exercise!
- You should know:
  - What are the properties of an RB tree?
  - And (more important) why does that guarantee that they are balanced?

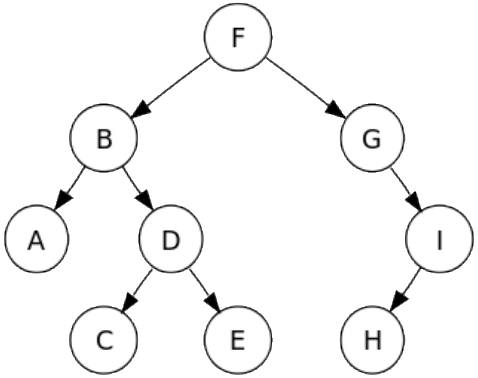
What have we learned?

- Red-Black Trees always have height at most 2log(n+1).
- As with general Binary Search Trees, all operations are O(height)
- So all operations with RBTrees are O(log(n)).

Conclusion: The best of both worlds

	Sorted Arrays	Linked Lists	Binary Search Trees*
Search	O(log(n))	O(n)	O(log(n))
Delete	O(n)	O(n)	O(log(n))
Insert	O(n)	O(1)	O(log(n))

Today

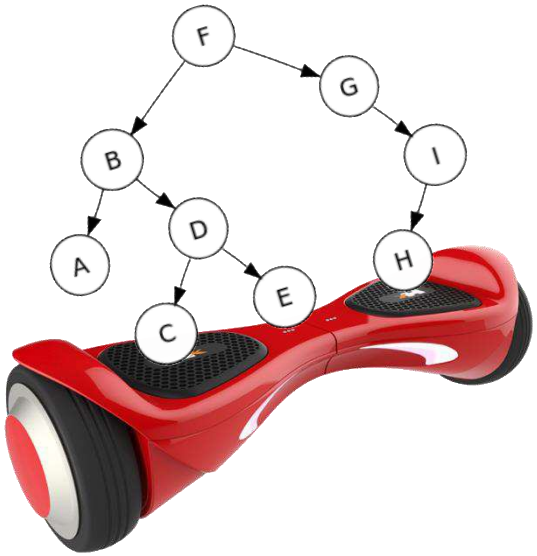


- Begin a brief foray into data structures!
  - See CS 166 for more!
- Binary search trees
  - You may remember these from CS 106B
  - They are better when they're balanced.

this will lead us to…

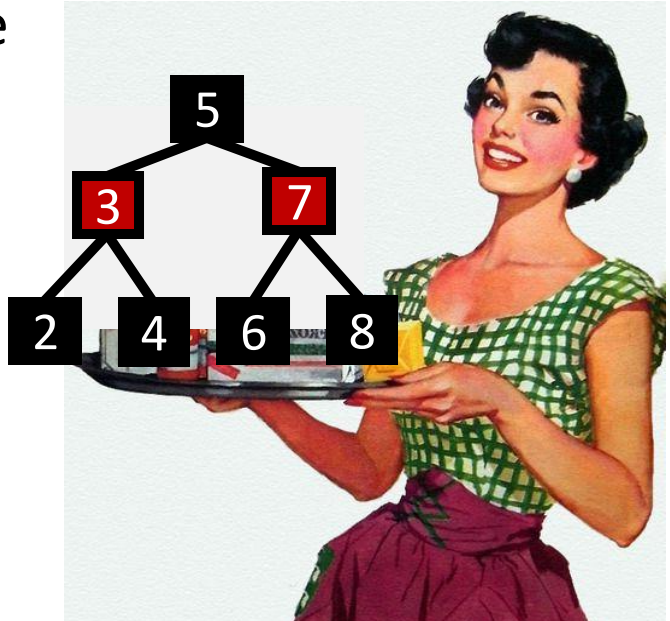
- Self-Balancing Binary Search Trees
  - **Red-Black** trees.

Recap



Recap

- Balanced binary trees are the best of both worlds!
- But we need to keep them balanced.
- **Red-Black Trees** do that for us.
  - We get O(log(n))-time INSERT/DELETE/SEARCH
  - Clever idea: have a proxy for balance



Next time

- **Midterm!**
- **(After that, Hashing!)**

Before next time

- Study for and take the exam!
- After that, pre-lecture Exercise for Lecture 8 (on Tuesday)
  - Yay more probability!